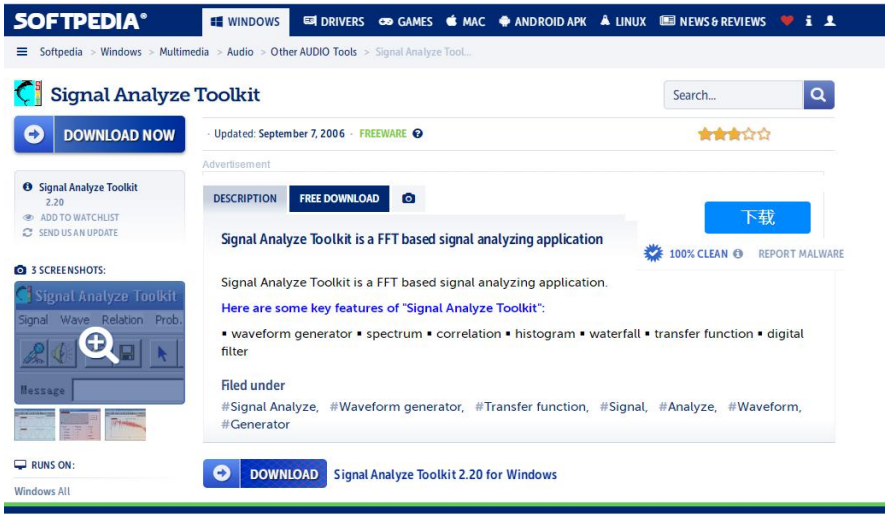
Python DRVI 可重构虚拟仪器资源包编程说明

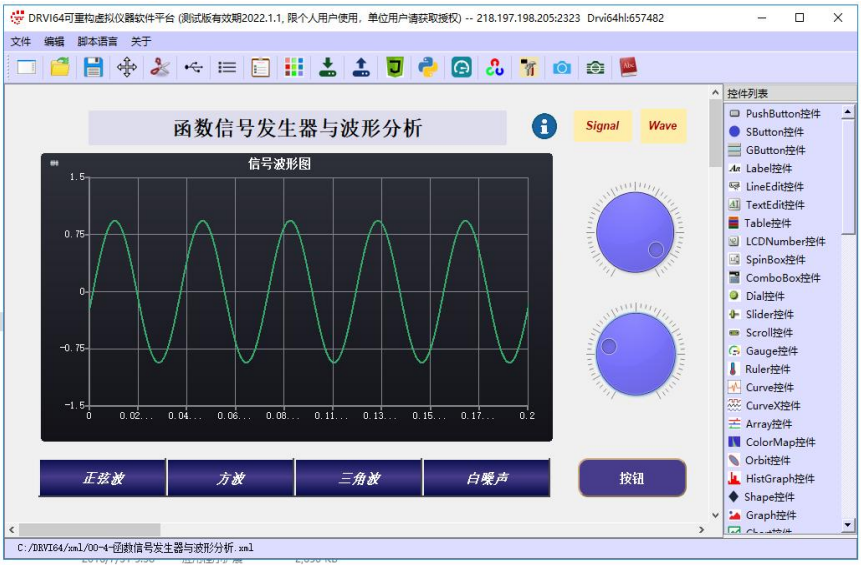
何岭松
华中科技大学机械学院仪器系

一、关于

最近十年，一直用美国的 Matlab 进行教学和科研工作，十分方便；但 Matlab，需要隔三差五的激活后，才能授权使用；别人不让你继续用，那是分分钟的事。我的主业是计算机虚拟仪器，C++、Java、Asp、Php，懂的计算机编程语言多且杂，也会那么几招软件开发功夫，HaHa，共享软件的江湖上现在还能查到早年闯荡时留下的痕迹。



一直想写一个 Mini 版的 Matlab 软件，可以放在 U 盘中，随身携带到教室，用来进行测控类课程教学。N 年前，学习 QT_C++，将过去开发的 DRVI 虚拟仪器软件从 Visual_C++过渡到了 QT_C++。发挥 QT 优点的 DRVI 界面酷炫，速度快，体积小，总体效果不错。



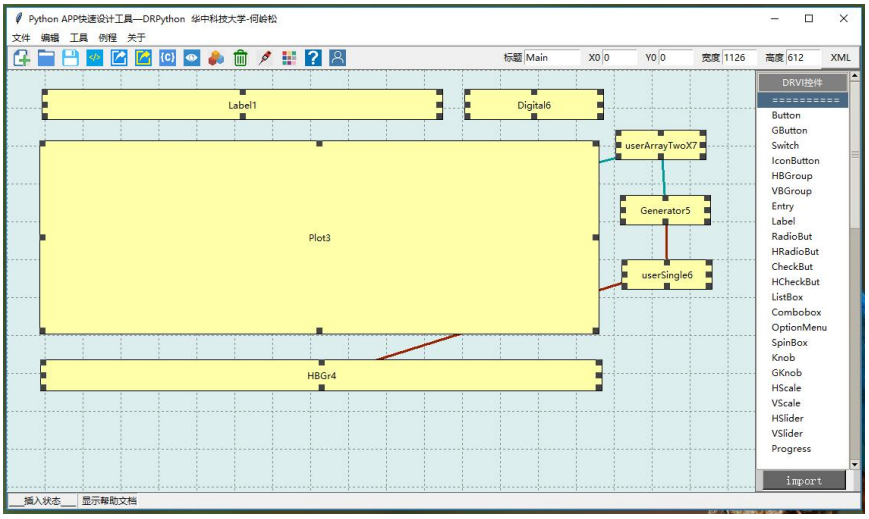
但 C++开发的软件有个硬伤，就是信号分析部分编程很难、功能扩展很难，功能实现非一般用户可以掌握。QT 版 DRVI 用于科研项目开发可以，但做为大众用途的工具软件，此路不通。

疫情仍未完全平息的这个暑假，异常炎热，无法外出观鸟，只能终日宅在空调屋里胡思乱想。忽然一天来了灵感，用 DRVI 中的软件总线和数据流驱动模型+Python 和 Python 旗下的 Numpy、SciPy、

Matplotlib 科学三剑客,来打造一款信号分析和测控技术领域的开源“Matlab”。

闭门造车一个月，终于将 DRVI 数据流驱动模型嫁接到了 Python 和其自带的 Tkinter 库，以及第三方的 Matplotlib 库上，形成了 Python 可重构虚拟仪器界面库 DRVI(内核大约 3M)，并赶在研究生《数字信号分析理论与实践》开课前完成了 Python 版的 APP_Designer——DRPython。

学生可以用 Python 语言快速设计虚拟仪器风格的 Python 程序，从实践中探索信号分析的奥秘。这是用 DRPython 的图形化编程界面布局的一个小电子琴程序的界面：



布局结果仍然采用笔者提出的可重构虚拟仪器装配描述语言 XVIML 进行保存、加载、修改和运行：

```
<?xml version="1.0" encoding="utf-8"?>
<drvi4.1>
    <Main>
        <p1 v="0,0,0,1344,756"/>
        <p2 v=""/>
    </Main>
    <Label1>
        <p1 v="2,46,57,529,41"/>
        <p2 v="'#003355','#ffffff','小电子琴例程',20"/>
    </Label1>
    <Plot3>
        <p1 v="51,43,125,738,256"/>
        <p2 v="','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.1,0,0,0,0"/>
    </Plot3>
    <HBGr4>
        <p1 v="101,44,414,741,42"/>
        <p2 v="'#003355','#ffffff','1,2,3,4,5,6,7,8,9',12,0"/>
    </HBGr4>
    <Generator5>
        <p1 v="79,808,197,120,40"/>
        <p2 v="0,44100,22050,0.6,100,0"/>
    </Generator5>
    <userSingle6>
        <p1 v="200,810,282,120,40"/>
        <p2 v=""/>
<![CDATA[v=v*100
v=v+200
mDigital6.setValueSingle(v)]]>      </userSingle6>
    <Digital6>
        <p1 v="36,603,57,184,41"/>
        <p2 v="'#000000','#00ffaa',0,2,22"/>
```

```
        </Digital6>
        <userArrayTwoX7>
            <p1 v="203,802,111,120,40"/>
            <p2 v=""/>
<![CDATA[sa.play(1,44100,xDataArray,0)]]>  </userArrayTwoX7>
        <ProbeLine>
            <p1 v="HBGr4,addCallBackSingle,userSingle6,setValueSingle,1"/>
        </ProbeLine>
        <ProbeLine>
            <p1 v="userSingle6,addCallBackSingle,Generator5,setVSFre,1"/>
        </ProbeLine>
        <ProbeLine>
            <p1 v="Generator5,addCallBack2DX,userArrayTwoX7,setValue2DX,3"/>
        </ProbeLine>
        <ProbeLine>
            <p1 v="userArrayTwoX7,addCallBack2DX,Plot3,setValue2DX,3"/>
        </ProbeLine>
        <Import>
<![CDATA[#==import related package====
import tkinter as tk
import tkinter.messagebox as msgbox
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
]]>   </Import>
</drvi4.1>
```

对普通用户来说，XVIML 只是保存 Python 应用程序 GUI 布局设计的后台助记符，并不需要直接与其打交道。完成 GUI 布局和控件间的数据流连线后，就可以用 DRPython 的 GUI 界面导出功能，将 XVLML 助记符记录的 GUI 界面设计结果和为 DRVI 可重构虚拟仪器 GUI 界面库支持的用户可看懂的 Python 程序源代码框架。这时，用户就可以在其基础上用 Python 编辑器进行设计和修改，与一般的 Python 程序并无区别。

下面是直接导出的一个可运行的 Python 小电子琴程序:

```
import tkinter as tk
import tkinter.messagebox as msgbox
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa

#==CallBack Function of MenuBar==============
def mExit():
    win.destroy()
    return 1

def mAbout():
    k=msgbox.askyesno('About','This is a APP frame by DRPython !')
    return 1

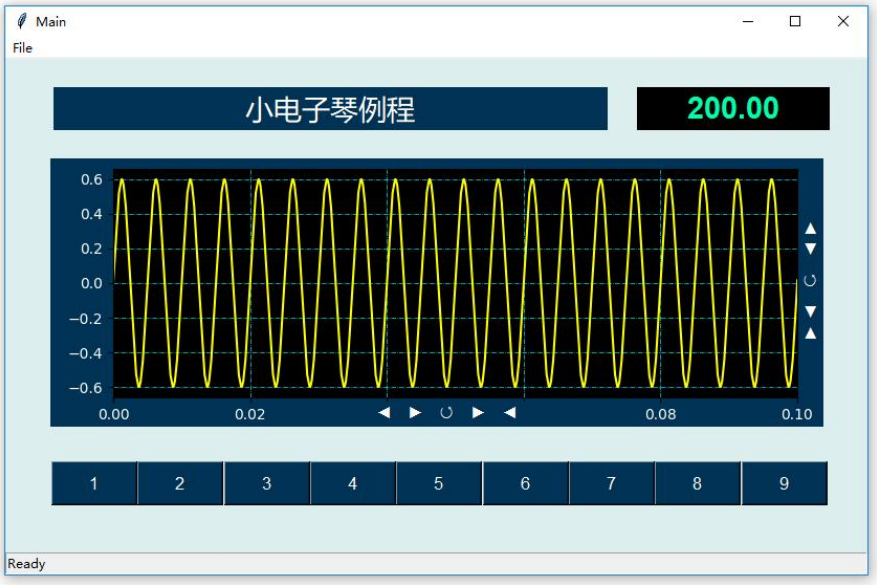
def muserSingle6(v):
  v=v*100
```

```
  v=v+200
  mDigital6.setValueSingle(v)
  mGenerator5.setVSFre(v)
  print("data ready")

def muserArrayTwoX7(dx,xDataArray):
  sa.play(1,44100,xDataArray,0)
  mPlot3.setValue2DX(dx,xDataArray)
  print("data ready")

#==Main Window=====================
win= tk.Tk()
win.wm_title('Main')
winWidthCur=986
winHeightCur=572
win.geometry('986x572')
win.config(bg="#ddeeee")
#==MenuBar========================
menubar=tk.Menu(win)
fmenu=tk.Menu(menubar,tearoff=False)
fmenu.add_command(label="About",command=mAbout)
fmenu.add_command(label="Close",command=mExit)
menubar.add_cascade(label="File",menu=fmenu)
win.config(menu=menubar)
#==StatusBar======================
sbar=tk.Label(win,text="Ready",bd=1,relief=tk.SUNKEN,anchor=tk.W)
sbar.pack(side=tk.BOTTOM,fill=tk.X)
#==Layout of controls=============
mLabel1=dr.DRLabel(win,46,27,529,41,'#003355','#ffffff','小电子琴例程',20)
mPlot3=dr.DRPlot(win,43,95,738,256,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.1,0,0,0,0)
mHBGr4=dr.DRHButtonGroup(win,44,384,741,42,'#003355','#ffffff','1,2,3,4,5,6,7,8,9',12,0)
mGenerator5=dsp.DRGenerator(0,44100,22050,0.6,100,0)
mDigital6=dr.DRDigital(win,603,27,184,41,'#000000','#00ffaa',0,2,22)
#=User defined data probe==========
mHBGr4.addCallBackSingle(muserSingle6)
mGenerator5.addCallBack2DX(muserArrayTwoX7)
#==Main Loop======================
win.mainloop()
```

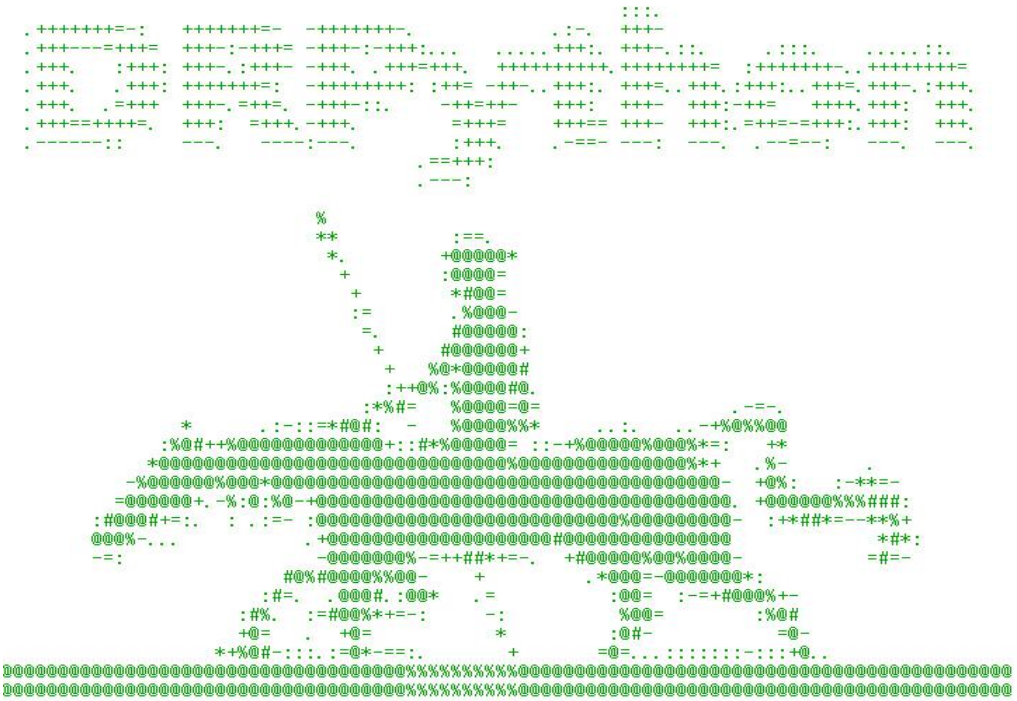
这是运行后的程序界面，初创作品的 GUI 界面虽然还不是很漂亮，但核心的图形化编程和数据流驱动的程序设计思想近乎完美的实现，十分满意。



虽然图形化编程创建出的一个带 GUI 界面的 Python 信号分析程序就几十行代码，但它背后站着的是 Numpy、SciPy、Matplotlib 科学三剑客，在数学运算和曲线显示方面的深厚功力与 Matlab 可以说是旗鼓相当。这样学生就可以将精力专注在信号分析问题本身，而不是在程序的设计实现上。

DRVI/DRPython 有 Numpy 、 SciPy 、 Matplotlib 科 学 三 剑 客 助 剑 ， 相 信 一 定 可 以 象 Don_Quijote_de_la_Mancha 一样，在共享软件的江湖上闯上出一小片自己的天地。HaHa，不自量力的 DRVI/DRPython 和我！

但天有不测风云，AI 的出现完全颠覆了传统编程技术发展路径，图形化编程多半会被 AI Coding 直接取代，费了很大劲实现的 DRPython 图形化编程工具，刚出道，就 Out 了；但做为基石，DRVI 库中的虚拟控件还是很有用的，可以让 AI 组装出的 Python 程序更简明、直观。

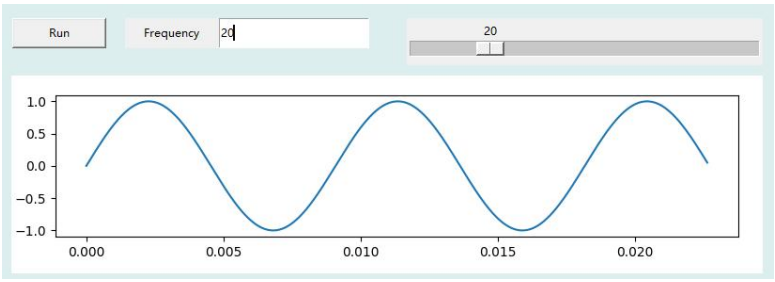


二、虚拟仪器控件直连和数据流驱动原理

2.1 直连编程问题的提出

DRVI 的内核是 Python 自带的 GUI 库 Tkinter 或 pyqt 等第三方 GUI 库。Tkinter GUI 控件采用的是回调函数工作机制，用户通过 GUI 界面的键盘或鼠标等操作触发某一 Tkinter 控件后，则调用该控件绑定的自定义回调函数来完成相应功能。程序控制流程很清晰，与一般的带 GUI 界面的 Matlab、C++等的编程原理相似。

回调函数工作机制编程简单，但对多控件联动操作时，其编程效率有点低。例如，下面是一个简单的正弦波信号发生器程序，其信号频率即可以用文本框输入，也可以用滑块输入，也就是说数字型文本框和滑块的数值应该是绑定和联动的。



下面是实现上述功能和这两个控件联动的样例代码。我们需要在各自的回调函数中编程调用另一控件的 set 方法来改变使其属性值，使它们的值同步改变。

```
import tkinter as tk
#定义回调函数
def entry1Func(e):
    scale1.set(str(e.widget.get()))

def scalFunc(value):
    entry1.delete(0,"end")
    entry1.insert(0,float(value))

# 创建主窗口（Evidence 10,11）
win = tk.Tk()
win.geometry("1100x740")
# Insert a Entry
entry1_var=tk.StringVar(win,'10')
entry1=tk.Entry(win,textvariable=entry1_var)
entry1.place(x=200,y=20,width=100,height=32)
entry1.bind("<Return>",entry1Func)
# Insert a scale
svar1 = tk.DoubleVar(win,10)
scale1 = tk.Scale(win,variable=svar1,from_=0, to=100, orient=tk.HORIZONTAL,command=scalFunc)
scale1.place(x=320,y=20,width=500,height=50)
# 运行主循环
win.mainloop()
```

如果能够将数字型文本框的输出端绑定到滑块控件的输入端，同时把滑块控件的输出端绑定到数字型文本框的输入端，则这两个回调函数就可以不需要；也就是可以用一个控件的输出数据流去直接驱动和修改另一个控件的属性。这就是我们提出的 DRVI 虚拟仪器控件直连和数据流驱动思想。

下面是 DRVI 控件实现相同功能的代码：

```
import tkinter as tk
import drvi.drviControlls as dr
#==Main Window====================
```

```
win= tk.Tk()
win.geometry('1204x716')
#==Layout of controls==================
mEntry1=dr.DREntry(win,180,114,100,30,'#ffffff','#000000','100',12)
mHScale2=dr.DRHScale(win,359,101,200,50,'#002266','#000000',0,100,50,11)
mEntry1.addCallBackSingle(mHScale2.setValueSingle)
mHScale2.addCallBackSingle(mEntry1.setValueSingle)
#==Main Loop=====================
win.mainloop()
```

由于直接用控件的回调函数对另一控件的属性进行设定，这样就把外部的 2 个回调函数给省掉了，程序更加简明、高效。但省代码并不是我们的主要目的，根本好处是这样处理后的控件可以实现直连编程，也就是可以用控件装配的图形化编程方式来实现 Python 框架程序设计。

为实现控件直连，必须满足一个前提条件，就是上一控件的输出函数参数个数和类型必须和下一控件的输入函数一致。因此，我们按照信号分析类程序的特点，对 Tkinter 和 Matplotlib 基础上生成的 DRVI 虚拟仪器控件间直连传递的参数进行了标准化处理，并分为如下几个常用的类别：

String：字符串，用于在控件间交换一段文字信息。

Single：数字，用于在控件间交换一个浮点数字。

2DArray：2 个数组，用于在控件间交换二个一维的浮点数组，如二维曲线的 x 坐标和 y 坐标数据。

2DXArray：数字+一维数组，用于在控件间传递一个浮点数字和一个一维数组，如一条二维曲线中的采样间隔和采样信号数组值。在传递等间距信号，如采样信号波形、FFT 频谱等时，可只传递一个一维数组，其数据传输效率比 2DArray 高。

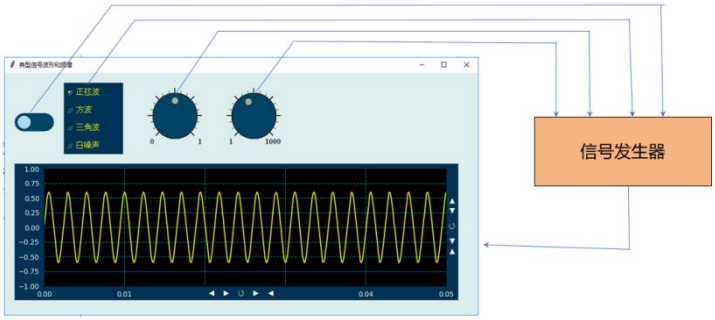
3DArray：3 个数组，用于在控件间交换三个数组对象。如谱阵图、语谱图数据，这时 x、y 为一维数组，z 通常二维的阵图数据。

Image：图象，用于在控件间传递一幅图象的数据块。

其它：用户也可以自定义其它组合形式的参数传递组合，只要输出方和输入方的参数个数和类型一致即可实现控件 set 属性函数和 get 属性函数的直连和驱动。

做了参数传递的标准化处理后，只要控件间 get/set 方法的参数个数和类型一致，就可以通过直接将上一控件的输出方法(get)，作为输入传递给下一控件的属性设定方法(set)。这样可以删除因控件取值、赋值所需的回调函数；同时，通过控件的直连和实现数据流驱动，程序框架也变的更易于实现装配式图形化编程。

例如，下面是用 DRVI 虚拟仪器控件设计的一个简单的信号发生器程序。通过一个按钮控件、一个单选按钮组、两个旋钮控件、一个曲线控件和一个数字信号发生器控件间的直连关系绑定，不用编写任何控件以外的回调函数，就可以构成一个数据流驱动的信号发生器应用程序。



下面是用 DRVI 库控件直连实现的信号发生器 Python 样例程序代码。程序内容就是简单的由控件布局和控件绑定两部分组成，无需额外的流程控制代码。

```
import tkinter as tk
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
#==创建主窗口=====================
win= tk.Tk()
win.geometry('1100x640')
win.config(bg="#ddeeee")
win.wm_title('典型信号波形和频谱')
#==控件插入和布局=================
mSwitch1=dr.DRSwitch(win,20,80,80,36,'#004466','#aaddee')
mVButType=dr.DRRadioBut(win,120,20,120,140,txt='正弦波,方波,三角波,白噪声')
mKnobA=dr.DRKnob(win,280,20,130,130,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mKnobFre=dr.DRKnob(win,440,20,130,130,'#004466','#222222','#aaaaaa','1,1000',1,1000,50,11)
mSignal=dsp.DRGenerator(0,44100,4096,0.6,100,0)
mPlotWave=dr.DRPlot(win,20,180,900,270,ylow=-1,yhigh=1)
#==控件直连关系绑定================
mSwitch1.addCallBackSingle(mSignal.setValueSingle)
mSignal.addCallBack2D(mPlotWave.setValue2DFast)
mVButType.addCallBackSingle(mSignal.setVSType)
mKnobA.addCallBackSingle(mSignal.setVSAmp)
mKnobFre.addCallBackSingle(mSignal.setVSFre)
#==Main Loop======================
win.mainloop()
```

绑定 1：实现开关按钮与信号发生器模块定时器开关的绑定，打开开关则信号发生器连续产生信号并输出。

```
mSwitch1.addCallBackSingle(mSignal.setValueSingle)
```

绑定 2：实现信号发生器输出与曲线控件输入的绑定，曲线波形随信号发生器的运行不断刷新变化。

```
mSignal.addCallBack2D(mPlotWave.setValue2DFast)
```

绑定 3：实现单选按钮组与信号发生器的信号类型属性的绑定，信号发生器的信号类型随用户选择改变。

```
mVButType.addCallBackSingle(mSignal.setVSType)
```

绑定 4：实现旋钮 1 与信号发生器的信号幅值属性的绑定，信号发生器输出的信号幅值随旋钮 1 改变。

```
mKnobA.addCallBackSingle(mSignal.setVSAmp)
```

绑定 5：实现旋钮 2 与信号发生器的信号频率属性的绑定，信号发生器输出的信号频率随旋钮 2 改变。

```
mKnobFre.addCallBackSingle(mSignal.setVSFre)
```

控件的绑定关系图也就是程序的运行控制图。因此，基于 DRVI 直连控件库的图形化编程就只需要实现可视化的控件布局和控件间连线，我们称之为程序的装配；最后再将将布局中用到的控件和控件间的绑定关系导出为 Python 代码，就可以快速的为用户实现一个 Python 应用程序框架内，这实现起来也似乎不难。

HaHa，一个 ugly 的仿 Matlab AppDesigner 的图形化辅助编程工具实现了。

2.2 DRVI 库在 Python Class 结构程序设计中的应用

对复杂一些的应用程序，带 Python Class 结构的程序模块化功能会更好一些，同时也更便于实现设计者间的代码共享。毕竟将功能封装为一个类后，只要提供一个接口调用说明，一切就 OK，内部实现可以不用考虑。

Tkinter 支持 Python Class 结构，作为 Tkinter 基础上开发的虚拟仪器风格 GUI 控件包，DRVI 库也就顺势继承了这一特色。将 DRVI 库应用 Python Class 中，在原理实现上并没有区别，只要将程序的主体结构改为 Python Class 就可以了。通常包含如下 5 个设计步骤：

1、书写需要引用的 Tkinter 包、DRVI 包和 Numpy 包等程序用到的支持包；

2、书写 Tkinter 主窗体程序创建代码；

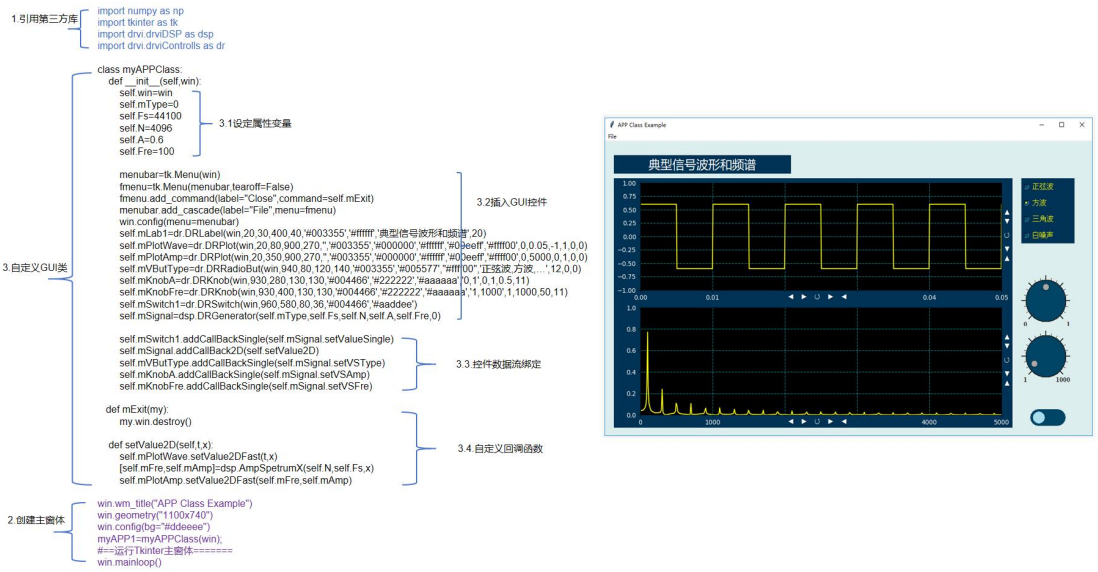
3、用 DRVI 控件布局和连线做为主体内容，创建一个 GUI Class；

4、在 GUI Class 中定义公共属性变量和完成用户所需功能的自定义回调函数。

5、将 GUI Class 加载到 Tkinter 中

通常，1、2、3、5 步，可以用图形化编程方式实现，也就是程序的框架结构可以用图形化编程方式实现；用户只要专注于完成特定功能的自定义回调函数编程即可。不想学 DRPython 图形化编程的话，也可以从安装后的 DRVI 库中找一个相近的样例代码，在其基础上修改实现。这样，用户编程主要集中在第 4 步，也就是自己的特色工作上。

下面是一个完整的由这五部分构成的 Python Class 结构形式的 DRVI GUI 应用程序，其功能就是一个典型信号的波形与频谱分析程序。



下面是 Class 形式书写的典型信号波形与频谱分析示例程序的 Python 代码：

```
import numpy as np
import tkinter as tk
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
#===自定义 GUI 类=========================================
class myAPPClass:
    def __init__(self,win):
        #==用 self 标记定义类中可引用的属性变量
        self.win=win
        self.mType=0
        self.Fs=44100
        self.N=4096
        self.A=0.6
```

```
        self.Fre=100
        self.Pha=0

        # 插入 GUI 控件==========
        menubar=tk.Menu(win)
        fmenu=tk.Menu(menubar,tearoff=False)
        fmenu.add_command(label="Close",command=self.mExit)
        menubar.add_cascade(label="File",menu=fmenu)
        win.config(menu=menubar)
        self.mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','典型信号波形和频谱',20)
        self.mPlotWave=dr.DRPlot(win,20,80,900,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.05,-1,1,0,0)
        self.mPlotAmp=dr.DRPlot(win,20,350,900,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,5000,0,1,0,0)
        self.mVButType=dr.DRRadioBut(win,940,80,120,140,'#003355','#005577',"#ffff00",'正弦波,方波,三角波,白噪声',12,0,0)
        self.mKnobA=dr.DRKnob(win,930,280,130,130,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
        self.mKnobFre=dr.DRKnob(win,930,400,130,130,'#004466','#222222','#aaaaaa','1,1000',1,1000,50,11)
        self.mSwitch1=dr.DRSwitch(win,960,580,80,36,'#004466','#aaddee')
        self.mSignal=dsp.DRGenerator(self.mType,self.Fs,self.N,self.A,self.Fre,self.Pha)
        #==控件数据流绑定======================
        self.mSwitch1.addCallBackSingle(self.mSignal.setValueSingle)
        self.mSignal.addCallBack2D(self.setValue2D)
        self.mVButType.addCallBackSingle(self.mSignal.setVSType)
        self.mKnobA.addCallBackSingle(self.mSignal.setVSAmp)
        self.mKnobFre.addCallBackSingle(self.mSignal.setVSFre)

    #==控件回调函数/自定义函数====================
    def mExit(my):
        my.win.destroy()

    def setValue2D(self,t,x):
        self.mPlotWave.setValue2DFast(t,x)
        [self.mFre,self.mAmp]=dsp.AmpSpetrumX(self.N,self.Fs,x)
        self.mPlotAmp.setValue2DFast(self.mFre,self.mAmp)

#==创建 Tkinter 主窗体==================
win= tk.Tk()
win.wm_title("APP Class Example")
win.geometry("1100x740")
win.config(bg="#ddeeee")
#==创建自定义 GUI 类
myAPP1=myAPPClass(win);
#==运行 Tkinter 主窗体====================
win.mainloop()
```

开发工作完成后，我还将 DRVI 移植到了 pyqt 上，用户可以选择用 Tkinter 做为基础 GUI 库或选择用 Pyqt5 做为基础 GUI 库来开发虚拟仪器程序。二者完全相同，只是引用的 GUI 库有所差异。下面是视频文件的音频信号波形和频谱分析程序代码。

```
from PyQt5.QtWidgets import *
from PyQt5.QtGui import *
from PyQt5.QtCore import *
import sys
import numpy as np
import drvi.drviDSP as dsp
import drvi.drviQTControlls as drqt

#===Start of MainClass==============
class drviMain(QMainWindow):
    def __init__(self):
        super(drviMain,self).__init__()
```

```
        self.initGUI()

    #===GUI 初始化=================================
    def initGUI(self):
        self.setWindowTitle("DRVI-MP4 文件波形和频谱 -- By DRPython")
        self.resize(1040,650)
        self.win=QWidget()
        self.win.setStyleSheet('background-color : #ddeeee; ')
        self.setCentralWidget(self.win)

        #=添加控件=============================
        self.mLab2=drqt.DRLabel(self.win,20,10,300,36,'#005588','#ffffff','MP4 文件波形和频谱',20)
        self.mFile=drqt.DREntry(self.win,600,10,310,36,'#000000','#ffff00','',12)
        self.mPlotWave=drqt.DRPlotFast(self.win,20,60,1000,260,0,0,0.09,-1,1,0,0)
        self.mPlotAmp=drqt.DRPlotFast(self.win,20,310,1000,260,0,0,5000,0,0.25,0,0)
        self.mPlayer=drqt.DRPlay(self.win,920,10,80,36,'#000000','#ffff00',11,'')
        self.mBut=drqt.DRHButtonGroup(self.win,400,10,190,36,'#006655','#ffff00','运行,停止,文件',12)

        #==事件流、数据流绑定=========================
        self.mBut.addCallBackSingle(self.mPlayer.setValueSingle)
        self.mPlayer.addCallBack2DX(self.setValue2DX)
        self.mPlayer.addCallBackStrFile(self.mFile.setValueString)
        #==初始运行状态================================
        self.TT=0

    #==事件流、数据流响应函数=======================
    def setFile(self,v):
        self.mPlayer.setValueSingle(2)

    def setValue2DX(self,dx,xArray):
        Fs=1/dx; N=len(xArray)
        CH=xArray.ndim
        if CH==2:
            xArray=xArray[:,0] #立体声, 取一通道数据
        mFre,mAmp=dsp.AmpSpetrum(N,Fs,xArray)
        self.mPlotWave.setValue2DX(dx,xArray)
        self.mPlotAmp.setValue2D(mFre,mAmp)

#==============================
app=QApplication(sys.argv)
win=drviMain()
win.show()
sys.exit(app.exec_())
```

程序运行效果如下：


DRVI-MP4文件波形和频谱 -- By DRPython
MP4文件波形和频谱
运行
停止
文件
D:/HLSDRPY/media/dream.mp4
718
1.00
0.50
0.00
-0.50
-1.00
0.000
0.022
0.045
0.250
0.188
0.125
0.063
0.000
0.0
1250.0
2500.0
3750.0
5000.0
视频播放器

三、DRVI 可重构虚拟仪器库控件列表

3.1 DRVI 基本 GUI 控件

DRVI 库中的基本 GUI 控件指的是在 Tkinter 库/pyqt 库的基本 GUI 控件基础上封装的一组虚拟仪器风格的 GUI 控件。这些 GUI 控件可以很好的满足信号分析类程序 GUI 界面的设计需求。各控件的功能和具体用法请点击链接，阅读对应的帮助文档。

DRButton：按钮控件，常用控件
DRLabel：标签控件，常用控件
DREntry：输入框控件，常用控件
DROptionMenu：弹出式菜单按钮控件，常用控件
DRFileBut：文件选择窗按钮
DRGButton：渐变色按钮控件
DRSwitch：开关按钮控件
DRIcon：图标按钮控件
DRHButtonGroup：水平按钮组控件
DRVButtonGroup：垂直按钮组控件
DRRadioBut：圆点型选择框控件
DRCheckButton：方块型选择框控件
DRListBox：列表选择框控件
DRComboBox：下拉列表选择框控件
DRSpinBox：数字调节钮控件
DRKnob：旋钮控件
DRGKnob：渐变色旋钮控件
DRHScale：水平滚动条控件
DRVScale：垂直滚动条控件
DRHSlider：水平滑块控件
DRVSlider：垂直滑块控件
DRProgress：进度条控件
DRRuler：棒图控件
DRHRuler：水平标尺控件
DRGauge：表盘控件
DRDigital：数字显示控件
DRLamp：警示灯控件
DRText：文本控件
DRList：表格控件
DRNoteBook：标签页控件
DRTab：紧缩式标签页控件

3.2 DRVI 曲线控件

DRVI 库中的曲线控件指的是在 Matplotlib 科学绘图包基础上封装的一组虚拟仪器风格的曲线显示控件。

DRPlot：Plot 型二维曲线控件，用于波形、频谱等显示，常用控件。
DRSubPlot：多曲线显示控件,,用于多幅波形、频谱等显示，常用控件
DRFigure：用户 Matplotlib 函数绘图容器控件，常用控件
DRMonitor：单幅多曲线显示控件，该控件通常用于多通道信号显示
DRStem：Stem 型二维曲线显示控件

DRSemilogX：X 坐标对数的二维曲线显示控件
DRSemilogY：Y 坐标对数的二维曲线显示控件
DRLoglog：X 坐标和 Y 坐标均为对数的二维曲线显示控件
DRScatter：散点图显示控件
DRPlotBar：直方图型显示控件
DRPlotPie：饼图显示控件
DRPlotPolar：极坐标图显示控件
DRPlot3D：三维坐标空间曲线显示控件
DRPlotSurface：三维曲面显示控件
DRPlotContour：伪彩色的等高线图显示控件
DRwaterFall：谱阵显示控件
DRColormesh：语谱图显示控件
DRMeshFast：实时语谱图显示控件，仅 pyqt 版
DRArrayX：实时谱阵图显示控件，仅 pyqt 版
DRImage：图形显示控件，如显示一幅 jpeg 图片
DRSubImages：多图片显示控件

3.3 DRVI 信号源控件

DRVI 库中的信号源控件指的是一组用数学公式产生虚拟仿真信号，或读取音频文件作为分析信号的控件，主要包括：

DRGenerator：虚拟信号发生器控件，常用控件
DRRecorder：麦克风/摄像头(pyqt)采集控件，常用控件
DRPLay：音频/视频(pyqt)文件播放器控件，常用控件
DRTimer：定时器控件，常用控件
DRPlayX：外挂 FFMPEG 音频/视频文件播放器
DRCameraCV：OpenCV 实现的摄像头视频信号采集控件
DRVideoCV：OpenCV 实现的视频文件播放和视频信号采集控件
DRTester：虚拟多自由度振动实验台和 H(s)实验台控件

3.4 DRVI 信号分析控件

DRVI 库中的信号分析控件指的是在 Numpy、Scipy 等第三方库基础上封装的信号波形分析、数字滤波、频谱分析、时频分析、概率密度和分布曲线，以及相关分析等常用信号分析功能。由于各库提供的信号分析函数已经很简捷，封装的必要性不大，只是出于直连编程的目的，封装了最常用的几种信号分析方法。

信号分析控件包括：

DRSpectrum：信号频谱分析控件，常用控件
DRWavePar：波形分析控件
DRSpectgram：谱阵分析控件
DRPdfCdf：信号概率密度和分布曲线分析控件
DRCorr：信号相关分析控件
DRFilter：信号数字滤波控件

3.5 DRVI 脚本程序控件

DRVI 库中的脚本控件用于在 GUI 界面上显示一段可以做为程序一部分参与运行的 Python 小程序代码。当该控件被数据流驱动时，控件中显示的小程序可以接收输入参数，然后被调用和执行。

DRScript：Python 小程序控件

3.6 DRVI 标准用户自定义函数

DRVI 库中的标准自定义函数指的是用 DRVI 标准参数作为输入参数的用户自定义函数，主要包括：

UserfunString(txt)：字符串 输入

UserfunSingle(v)：数字 输入

Userfun2DArray(x,y)：一维数组 + 一维数组 输入

Userfun2DXArray(dx,y)：数字 + 一维数组 输入

UserArrayTri(x,y,z)：一维数组 + 一维数组 + 二维数组 输入

UserImage(Image)：图象数据块 输入

四、DRVI 可重构虚拟仪器库控件编程指南

DRButton 按钮用法

按钮控件是最常用的 GUI 控件之一，可用于接收用户的鼠标点击输入，然后通过直连方式驱动另一控件工作，或通过回调函数形式定义其具体功能和操作。



DRButton 按钮控件显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRButton 按钮控件的命令和语法格式如下:

hl=DRButton(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

txt：标题，默认值='Button'

fsize：字符点数，默认值=12

val：按键值，默认值=101

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

下面是一个全参数的 DRButton 按钮控件创建实例：

```
hl=dr.DRButton(win,262,103,100,40,'#00aacc','#eeee00','Button1',12,101)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRButton(win,262,103,100,40,txt='Run')
```

创建后的 DRButton 按钮控件可以通过直连方式将用户鼠标点击操作传递给另一控件，或通过回调函数绑定的方式将用户的鼠标点击操作传递给被绑定的回调函数。

DRButton 按钮控件的直连或回调函数绑定格式为：

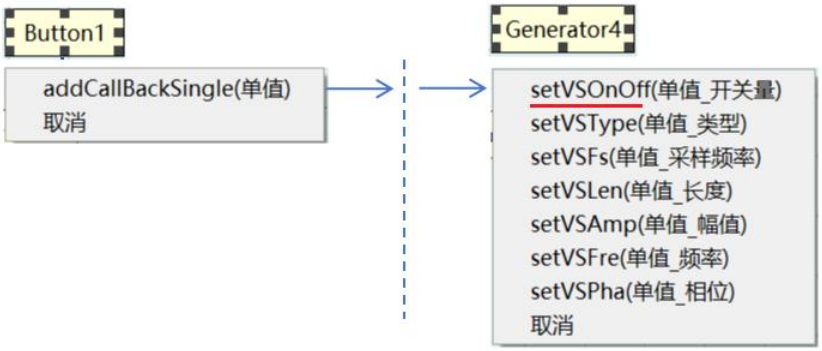
```
hl.addCallBackSingle(fun)
```

其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。DRButton 按钮控件绑定的 fun 函数是一个单数值函数，其收到的则是该按钮控件传递过来的按键值 val。下面是 fun 函数的形式：

```
def setValueSingle(v):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。按钮控件为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的单值(addCallBackSingle)为按钮的设定值。例如下图是将按钮控件的输出函数与信号发生器控件的开关设定函数相连。



下面是 DRButton 按钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到按钮控件的输出上。点击按钮，则在用户点击操作引发的数据流驱动下，文本框控件的输入值显示该按钮控件的按键值 val，而用户自定义的回调函数则 Print 出接收到的按键值 val。DRButton 按钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,50,42,100,40,txt='Run')
#==插入文本框控件
mEntry2=dr.DREntry(win,198,44,100,30)
#==直连方式绑定文本框控件
mButton1.addCallBackSingle(mEntry2.setValueSingle)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中， DRButton 按钮控件继承自 QPushButton，因此除了用 DRButton 的参数化外观定义外，还可以直接用 QPushButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

下面是用样式表来定义 DRButton 外观样例：

```
mButton1=dr.DRButton(win,78,38,100,40)
mButton1.setStyleSheet("background-color: red; color: white; font: bold 12px; border: 2px solid blue; border-radius: 10px;")
```

效果如下：



样式表的具体用法可参阅 pyqt 或 QT 中 QPushButton 样式表的设定部分。

下面 pyqt 版本中是 DRButton 按钮控件的用法示例：

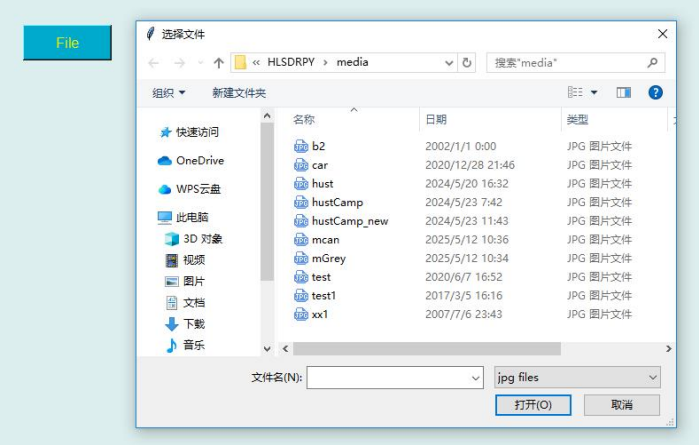
```
# 1. 引用的库
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import *
from PyQt5.QtCore import *
import drvi.drviQTControlls as dr

#2.回调函数
def mButClick(v):
    print('Value=',v)

#3.创建 GUI 窗体口象
app=QApplication(sys.argv)
app.setStyle("Fusion")
win=QWidget()
win.setWindowTitle('PyQt 程序')
win.resize(800,500)
win.setStyleSheet("background-color : #ddeeee; ")
#4.在 GUI 窗口插入控件和布局
mButton1=dr.DRButton(win,78,38,100,40)
mButton1.setStyleSheet("background-color: red; color: white; font: bold 12px; border: 2px solid blue; border-radius: 10px;")
#5.控件回调函数绑定==============
mButton1.addCallBackSingle(mButClick)
#6.显示 GUI 主窗口界面，等待用户操作
win.show()
sys.exit(app.exec_())
```

DRFileBut 文件选择窗按钮

文件选择窗按钮是按钮控件和文件选择窗的组合，可用于选择待处理的文件名，然后通过直连方式或回调函数形式将所选择的文件名传递给另一控件或回调函数，并驱动它们工作。



DRFileBut 文件选择窗按钮显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRFileBut 文件选择窗按钮的命令和语法格式如下:

hl=DRFileBut(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

txt：标题，默认值='File'

ext：标题，默认值='jpg'

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRFileBut 文件选择窗按钮有 1 个属性函数，其功能定义如下：

txt=hl.getValueString()：获取所选择的文件名

下面是一个全参数的 DRFileBut 文件选择窗按钮创建实例：

```
hl=dr.DRFileBut(win,196,91,100,40,'#00aacc','#eeee00','File','jpg',12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRFileBut(win,196,91,100,40,ext='wav')
```

创建后的 DRFileBut 文件选择窗按钮可以通过直连方式将用户选择的文件名传递给另一控件，或通过回调函数绑定的方式将传递给被绑定的回调函数。

DRFileBut 文件选择窗按钮的直连或回调函数绑定格式为：

```
hl.addCallBackString(fun)
```

其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。DRFileBut 文件选择窗按钮绑定的 fun 函数是一个字符串接收函数，其收到的则是该控件传递过来的文件名。下面是 fun 函数的形式：

```
def setValueString(filename):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。文件选择按钮控件为字符串输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的字符串(addCallBackString)为选择的文件名。



下面是 DRFileBut 文件选择窗按钮的创建和应用示例小程序，功能是在 GUI 界面上插入一个文件选择窗按钮控件、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到文件选择窗按钮控件的输出上。点击文件选择窗按钮进行文件选择，则文本框控件的输入值显示选择的文件，而用户自定义的回调函数则 Print 出接收到的文件名。DRFileBut 文件选择窗按钮应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserString3(txt):
    print(txt)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入文件选择窗按钮
mButton1=dr.DRFileBut(win,50,42,100,40,txt='File')
#==插入文本框控件
mEntry2=dr.DREntry(win,198,44,300,30,txt='')
#==直连绑定
mButton1.addCallBackString(mEntry2.setValueString)
mButton1.addCallBackString(muserString3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

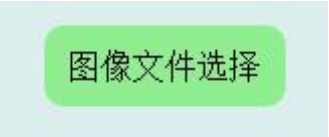
DRVI 有继承自 Tkinter 和继承自 pyqt 的两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中， DRFileBut 文件选择窗按钮继承自 QPushButton，因此除用 DRFileBut 的参数化外观定义外，还可以直接用 QPushButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

下面是用样式表来定义 DRFileBut 外观样例：

```
mFileBut1=dr.DRFileBut(win,172,87,120,40,'#00aacc','#eeee00','图像文件选择','jpg',12)
qss="""
QPushButton{color:black;
background-color:lightgreen;
border:2px;
border-radius:10px;
padding:2px 4px;}
QPushButton:hover{color:red}
"""
mFileBut1.setStyleSheet(qss)
```

效果如下：



样式表的具体用法可参阅 pyqt 或 QT 中 QPushButton 样式表的设定部分。

DRGButton 渐变色按钮用法

渐变色按钮控件的作用与按钮控件相同，可用于接收用户的鼠标点击输入，然后通过直连方式驱动另一控件工作，或通过回调函数形式定义其具体功能和操作。



DRGButton 渐变色按钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRGButton 渐变色按钮控件的命令和语法格式如下:

hl=DRGButton(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：起始渐变背景色，默认值='#00aacc'

cb2：终止渐变背景色，默认值='#eeee00'

cf：前景色，默认值='#000000'

R：按钮的圆角半径，默认值=10

txt：标题，默认值='GButton'

fsize：字符点数，默认值=12

val：按键值，默认值=102

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

下面是一个全参数的 DRGButton 渐变色按钮控件创建实例：

```
hl=dr.DRGButton(win,282,96,100,40,'#00aacc','#eeee00','#000000',12,'GButton1',12,101)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRGButton(win,97,150,100,40,R=6)
```

创建后的 DRGButton 渐变色按钮控件可以通过直连方式将用户鼠标点击操作传递给另一控件，或通过回调函数绑定的方式将用户的鼠标点击操作传递给被绑定的回调函数。

DRGButton 渐变色按钮控件的直连或回调函数绑定格式为：

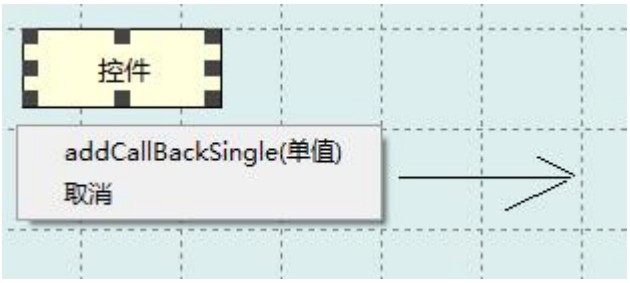
```
hl.addCallBackSingle(fun)
```

其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。渐变色按钮控件绑定的 fun 函数是一个单数值函数，收到的是渐变色按钮控件传递过来的按键值 val。下面是 fun 函数的形式：

```
def setValueSingle(v):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。渐变色按钮控件为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的单值(addCallBackSingle)为按钮的设定值。



下面是 DRGButton 渐变色按钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个渐变色按钮控件、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到渐变色按钮控件的输出上。点击按钮，则在用户点击操作引发的数据流驱动下，文本框控件的输入值显示该渐变色按钮控件的按键值 val，而用户自定义的回调函数则 Print 出接收到的按键值 val。DRGButton 渐变色按钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入渐变色按钮控件
mButton1=dr.DRGButton(win,50,42,100,40,txt='Run')
#==插入文本框控件
mEntry2=dr.DREntry(win,198,44,100,30)
#==直连方式绑定文本框控件
mButton1.addCallBackSingle(mEntry2.setValueSingle)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 的两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中， DRGButton 渐变色按钮继承自 QPushButton，因此除用 DRGButton 渐变色按钮的参数化外观定义外，还可以直接用 QPushButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

下面是用样式表来定义 DRFileBut 外观样例：

```
mBut1=dr.DRGButton(win,172,87,120,40)
qss="""
QPushButton{color: #ffffff;
background-color:qlineargradient(spread:pad, x1:0, y1:0, x2:1, y2:0, stop:0 #ff0000, stop:1 #0000ff);
border-radius: XXpx; border: 1px groove gray;
border-style: outset;}
"""
mBut1.setStyleSheet(qss)
```

效果如下：



样式表的具体用法可参阅 pyqt 或 QT 中 QPushButton 样式表的设定部分。

DRSwitch 开关按钮用法

DRSwitch 开关按钮控件的作用与一个开关相似，可用于接收用户的鼠标点击输入，其输出值在 0/1 两态间变换。可用于接收用户的鼠标点击输入，然后通过直连方式驱动另一控件工作，或通过回调函数形式定义其具体功能和操作。



DRSwitch 开关按钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRSwitch 开关按钮控件的命令和语法格式如下:

hl=DRSwitch(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#004466'

cf：前景色，默认值='#aaddee'

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

下面是一个全参数的 DRSwitch 开关按钮控件创建实例：

```
hl=dr.DRSwitch(win,122,67,80,36,'#004466','#aaddee')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRSwitch(win,122,67,80,36)
```

创建后的 DRSwitch 开关按钮控件可以通过直连方式将用户鼠标点击操作传递给另一控件，或通过回调函数绑定的方式将用户的鼠标点击操作传递给被绑定的回调函数。

DRSwitch 开关按钮控件的直连或回调函数绑定格式为：

```
hl.addCallBackSingle(fun)
```

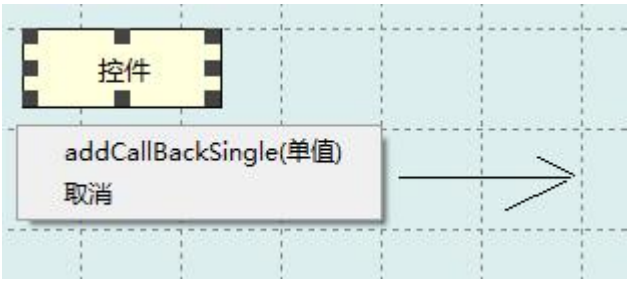
其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。DRSwitch 开关按钮控件绑定的 fun 函数是一个单数值函数，收到的是 DRSwitch 开关按钮控件传递过来的 0/1 按键值。下面是 fun 函数形式：

```
def setValueSingle(v):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操

作。开关按钮控件为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的单值(addCallBackSingle)为开关按钮的值(0/1 间切换)。



下面是 DRSwitch 开关按钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个开关按钮控件、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到开关按钮控件的输出上。点击开关按钮，则在用户点击操作引发的数据流驱动下，文本框控件的输入值显示该开关按钮控件的 0/1 按键值，而用户自定义的回调函数则 Print 出接收到的 0/1 按键值。

DRSwitch 开关按钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRSwitch(win,50,42,80,40)
#==插入文本框控件
mEntry2=dr.DREntry(win,198,44,100,30)
#==直连方式绑定文本框控件
mButton1.addCallBackSingle(mEntry2.setValueSingle)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 的两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中， DRGButton 继承自 QWidget 控件，不支持 setStyleSheet。

DRIcon 图标按钮用法

DRIcon 图标按钮控件用一幅图片作为背景，其作用与按钮控件相同，可用于接收用户的鼠标点击输入，然后通过直连方式驱动另一控件工作，或通过回调函数形式定义其具体功能和操作。



DRIcon 图标按钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRIcon 图标按钮控件的命令和语法格式如下:

hl=DRIcon(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

filename：采用的图标文件名和路径，默认值='imgs/a1.png'

val：按键值，默认值=103

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

下面是一个全参数的 DRIcon 图标按钮控件创建实例：

```
hl=dr.DRIcon(win,175,62,60,60,'imgs/a1.png',400)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRIcon(win,175,62,60,60,filename='imgs/myicon.png')
```

创建后的 DRIcon 图标按钮控件可以通过直连方式将用户鼠标点击操作传递给另一控件，或通过回调函数绑定的方式将用户的鼠标点击操作传递给被绑定的回调函数。

DRIcon 图标按钮控件的直连或回调函数绑定格式为：

```
hl.addCallBackSingle(fun)
```

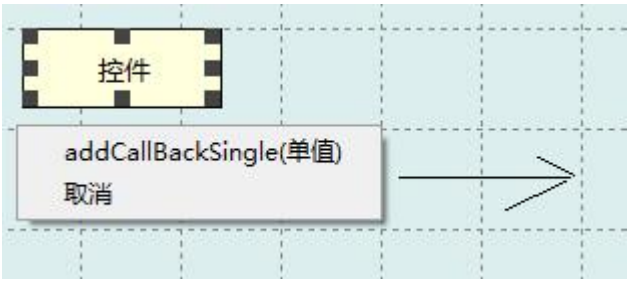
其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。DRIcon 图标按钮控件绑定的 fun 函数是一个单数值函数，收到的是 DRIcon 图标按钮控件传递过来的按键值 val。下面是 fun 函数或形式：

```
def setValueSingle(v):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。图标按钮控件为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，

点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的单值(addCallBackSingle)为按钮的设定值。



下面是 DRIcon 图标按钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个图标按钮控件、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到图标按钮控件的输出上。点击按钮，则在用户点击操作引发的数据流驱动下，文本框控件的输入值显示该图标按钮控件的按键值 val，而用户自定义的回调函数则 Print 出接收到的按键值 val。DRIcon 图标按钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviAppendPath as mPath

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入图标按钮控件
pFile=mPath.getDRVIPath()+'\\imgs/a1.png'
mButton1=dr.DRIcon(win,50,42,50,50,filename=pFile)
#==插入文本框控件
mEntry2=dr.DREntry(win,198,44,100,30)
#==直连方式绑定文本框控件
mButton1.addCallBackSingle(mEntry2.setValueSingle)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 的两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中， DRGButton 继承自 QPushButton 控件，支持用样式表进行外观定义，其样式表加载方法为：

```
setStyleSheet(styleString)
```

DRHButtonGroup 水平按钮组用法

DRHButtonGroup 水平按钮组控件就是一组水平排列在一起的按钮控件，可用于接收用户的鼠标点击输入，然后通过直连方式驱动另一控件工作，或通过回调函数形式定义其具体功能和操作，按钮组中相邻按钮的返回值差值为 1。



DRHButtonGroup 水平按钮组控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRHButtonGroup 水平按钮组控件控件的命令和语法格式如下:

hl=DRHButtonGroup(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

txt：标题，默认值='Start,Stop,File'，引号中单词数=按钮组中的按钮数

fsize：字符点数，默认值=12

val：按键值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

下面是一个全参数的 DRHButtonGroup 水平按钮组控件创建实例：

```
hl=dr.DRHButtonGroup(win,149,65,240,36,'#003355','#ffffff','start,stop,file',12,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRHButtonGroup(win,149,65,240,36,vla=10)
```

创建后的 DRHButtonGroup 水平按钮组控件可以通过直连方式将用户鼠标点击操作传递给另一控件，或通过回调函数绑定的方式将用户的鼠标点击操作传递给被绑定的回调函数。

DRHButtonGroup 水平按钮组控件的直连或回调函数绑定格式为：

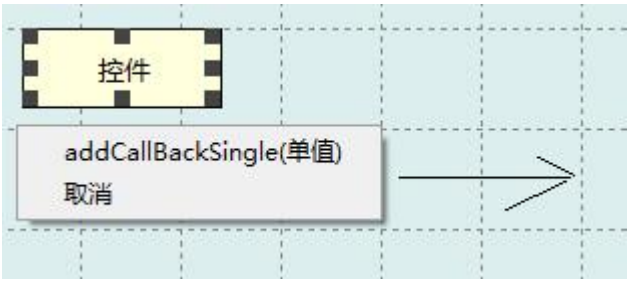
```
hl.addCallBackSingle(fun)
```

其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。DRHButtonGroup 水平按钮组控件绑定的 fun 函数是一个单数值函数，收到的该控件传递过来的按键值 val。下面是 fun 函数或形式：

```
def setValueSingle(v):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。水平按钮组控件为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的单值(addCallBackSingle)为按钮组的设定值+点击的按钮号。



下面是 DRHButtonGroup 水平按钮组控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个水平按钮组、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到水平按钮组的输出上。点击按钮，则在用户点击操作引发的数据流驱动下，文本框控件的输入值显示该水平按钮组的按键值 val+按键号，而用户自定义的回调函数则 Print 出接收到的按键值 val+按键号。DRHButtonGroup 水平按钮组控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入图标按钮控件
mButton1=dr.DRHButtonGroup(win,40,40,240,36,val=10)
#==插入文本框控件
mEntry2=dr.DREntry(win,350,44,100,30)
#==直连方式绑定文本框控件
mButton1.addCallBackSingle(mEntry2.setValueSingle)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

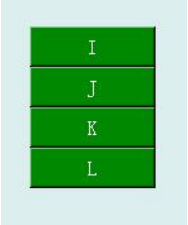
DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRHButtonGroup

水平按钮组控件继承自 QPushButton，因此除了用 DRHButtonGroup 的参数化外观定义外，还可以直接用 QPushButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRVButtonGroup 垂直按钮组用法

DRVButtonGroup 垂直按钮组控件就是一组垂直排列在一起的按钮控件，可用于接收用户的鼠标点击输入，然后通过直连方式驱动另一控件工作，或通过回调函数形式定义其具体功能和操作，按钮组中相邻按钮的返回值差值为 1。



DRVButtonGroup 垂直按钮组的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRVButtonGroup 垂直按钮组控件的命令和语法格式如下:

hl=DRVButtonGroup(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

txt：标题，默认值='Start,Stop,File'，引号中单词数=按钮组中的按钮数

fsize：字符点数，默认值=12

val：按键值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

下面是一个全参数的 DRVButtonGroup 垂直按钮组创建实例：

```
hl=dr.DRVButtonGroup(win,218,108,100,100,'#008800','#ffffff','start,stop,file',12,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRVButtonGroup(win,229,275,100,100,val=1)
```

创建后的 DRVButtonGroup 垂直按钮组可以通过直连方式将用户鼠标点击操作传递给另一控件，或通过回调函数绑定的方式将用户的鼠标点击操作传递给被绑定的回调函数。

DRVButtonGroup 垂直按钮组的直连或回调函数绑定格式为：

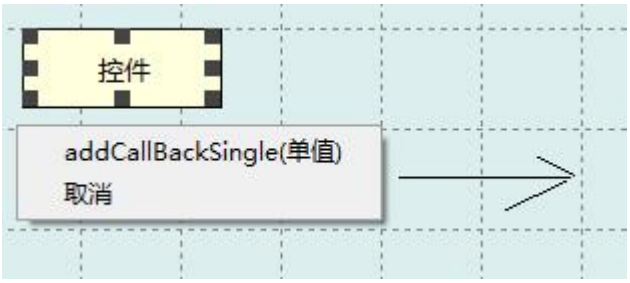
```
hl.addCallBackSingle(fun)
```

其中 fun 是被绑定的另一控件的 set 函数名，或用户自定义的回调函数名。DRVButtonGroup 垂直按钮组绑定的 fun 函数是一个单数值函数，收到的该控件传递过来的按键值 val。下面是 fun 函数或形式：

```
def setValueSingle(v):
    .....
```

通过回调函数绑定方式，就可以将用户的鼠标点击操作传递到回调函数中，或者传递到另一个控件的 set 方法中，驱动它们运行。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。垂直按钮组控件为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出的单值(addCallBackSingle)为按钮组的设定值+点击的按钮号。



下面是 DRVButtonGroup 垂直按钮组的创建和应用示例小程序，功能是在 GUI 界面上插入一个垂直按钮组、一个文本框控件和一个用户自定义函数，然后将它们分别绑定到垂直按钮组的输出上。点击按钮，则在用户点击操作引发的数据流驱动下，文本框控件的输入值显示该垂直按钮组的按键值 val+按键号，而用户自定义的回调函数则 Print 出接收到的按键值 val+按键号。DRVButtonGroup 垂直按钮组应用示例小程序代码如下：

```
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入控件
mButton1=dr.DRVButtonGroup(win,40,44,100,100,val=1010)
mEntry2=dr.DREntry(win,198,44,100,30)
#==绑定控件
mButton1.addCallBackSingle(mEntry2.setValueSingle)
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRHButtonGroup

水平按钮组控件继承自 QPushButton，因此除了用 DRHButtonGroup 的参数化外观定义外，还可以直接用 QPushButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRLabel 标签用法

DRLabel 标签控件是最常用的 GUI 控件之一，可用于显示一行文字或数字。



DRLabel 标签控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRLabel 标签控件的命令和语法格式如下:

hl=DRLabel(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#bbeeff'

cf：前景色，默认值='#000000'

txt：标题，默认值='Label'

fsize：字符点数，默认值=12

d：标签方向，默认值=0，0/1=水平/垂直

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRLabel 标签控件有 4 个属性函数，其功能定义如下：

hl.setValueSingle(v)：设定所显示的数字
hl.setValueString(txt)：设定所显示的文字
v=hl.getValueSingle()：获取所显示的数字
txt=hl.getValueString()：获取所显示的文字

下面是一个全参数的 DRLabel 标签控件创建实例：

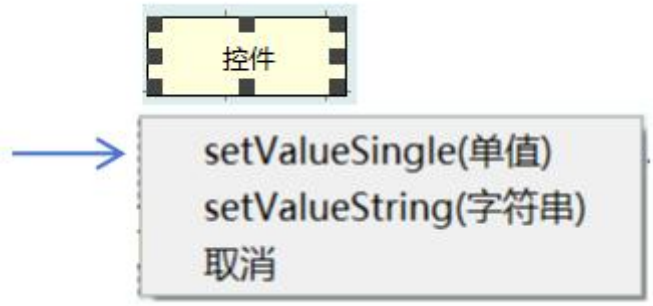
```
hl=dr.DRLabel(win,174,88,100,30,'#bbeeff','#000000','Label',12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRLabel(win,221,176,100,30,fsize=16)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。标签控件为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)或字符串端口(setValueString)收到数据流后，则做为标签控件的内容显示。



下面是 DRLabel 标签控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个标签控件、一个按钮和一个用户自定义函数。点击按钮，则调用自定义函数，对标签控件的显示内容进行 set 和 get 操作。DRLabel 标签控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    mLabe.setValueString('Button Clicked !')
    txt=mLabe.getValueString()
    print(txt)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,50,42,100,40,txt='Run')
#==插入标签控件
mLabe=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

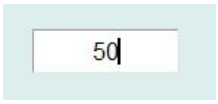
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRLabel 标签控件继承自 QLabel，因此除了用 DRLabel 的参数化外观定义外，还可以直接用 QLabel 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DREntry 输入框用法

DREntry 输入框控件是最常用的 GUI 控件之一，可用于输入文字或数字；当然，也可以象标签控件一样，用于显示文字或数字。



DREntry 输入框控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DREntry 输入框控件的命令和语法格式如下:

hl=DREntry(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ffffff'

cf：前景色，默认值='#000000'

txt：标题，默认值='0'

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DREntry 输入框控件有 4 个属性函数，其功能定义如下：

hl.setValueSingle(v)：设定所显示的数字
hl.setValueString(txt)：设定所显示的文字
v=hl.getValueSingle()：获取所显示的数字
txt=hl.getValueString()：获取所显示的文字

下面是一个全参数的 DREntry 输入框控件创建实例：

```
hl=dr.DREntry(win,413,168,100,30,'#ffffff','#000000','100',12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DREntry(win,413,268,100,30,txt='10')
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DREntry 输入框控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，数字型格式为：

```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是输入框中的数字。回调函数形式为：

```
def setValueSingle(v):
```

.....

文本型回调函数绑定格式为：

hl.addCallBackString(fun)

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的是输入框中的字符串输入信息。回调函数形式为：

def setValueString(t):

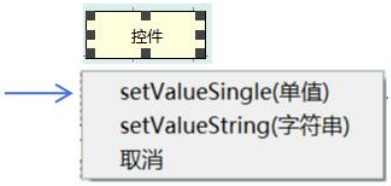
.....

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。输入框控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输入框内的输出内容可以通过单值端口(addCallBackSingle)或字符串端口(addCallBackString)输出。



输入框控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)或字符串端口(setValueString)收到数据流后，则做为输入框控件的内容显示。



下面是 DREntry 输入框控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个标签控件、一个用户自定义函数和一个输入框控件。再输入框控件中进行输入，则输入值通过直连方式传递到自定义函数和标签控件中。DREntry 输入框控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
```

```
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入输入框控件
mEntry=dr.DREntry(win,50,42,100,30,txt='10')
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mEntry.addCallBackSingle(muserSingle3)
mEntry.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

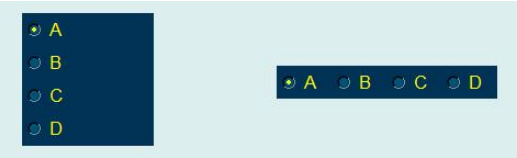
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DREntry 输入框控件继承自 QLineEdit，因此除了用 DREntry 的参数化外观定义外，还可以直接用 QLineEdit 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRRadioBut 圆点型选择框用法

DRRadioBut 圆点型选择框控件是一组纵向或水平排列在一起的选项，可用于接收用户的鼠标点击选择，然后通过回调函数形式编程定义其具体功能和操作，输出值可以是选项字符串，也可以是选项号。



DRRadioBut 圆点型选择框控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRRadioBut 圆点型选择框控件的命令和语法格式如下:

hl=DRRadioBut(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#003355'

cb1：次背景色，默认值='#007799'

cf：前景色，默认值='#ffff00'

txt：标题，默认值='A,B,C,D'，引号中单词数=选项数

fsize：字符点数，默认值=12

val：当前选项值，默认值=0

d：控件显示形式，默认值=0（0=垂直，1=水平）

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRRadioBut 圆点型选择框控件有 2 个 get 属性函数，功能如下：

txt=hl.getValueString()：获取当前选项菜单条文字

v=hl.getValueSingle()：获取当前选项菜单条号

下面是一个全参数的 DRRadioBut 圆点型选择框控件创建实例：

```
hl=dr.DRRadioBut(win,74,80,120,120,'#003355','#007799','#ffff00','A,B,C,D',12,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。例如，下面创建一个水平显示的 DRRadioBut 圆点型选择框控件：

```
hl=dr.DRRadioBut(win,55,424,260,30,d=1)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRRadioBut 圆点型选择框控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，数字型格式为：

```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是当前选项号+vla。回调函数形式为：

```
def setValueSingle(v):
    ......
```

文本型回调函数绑定格式为：

```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的是当前选项的菜单条文字。回调函数形式为：

```
def setValueString(t):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。圆点型选择框控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。选项号或条目可以通过单值端口(addCallBackSingle)或字符串端口(addCallBackString)输出。



下面是 DRRadioBut 圆点型选择框控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个圆点型选择框控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将圆点型选择框控件的选项菜单条文字传递给标签控件，将选项值传递给自定义函数。DRRadioBut 圆点型选择框控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入圆点型选择框控件控件
mRadio=dr.DRRadioBut(win,73,244,120,120)
#==插入标签控件
```

```
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mRadio.addCallBackSingle(muserSingle3)
mRadio.addCallBackString(mLab.setValueString)
#==主程序循环
win.mainloop()
```

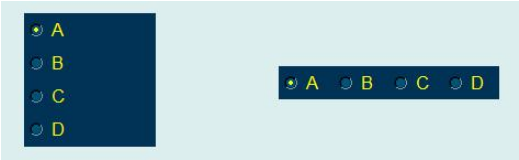
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRRadioBut 继承自 QRadioButton，因此除了用 DRRadioBut 的参数化外观定义外，还可以直接用 QRadioButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRCheckButton 方块型选择框用法

DRCheckButton 方块型选择框控件是一组纵向或水平排列在一起的选项，可用于接收用户的鼠标点击选择，然后通过回调函数形式编程定义其具体功能和操作，输出值可以是选项字符串，也可以是选项号+设定值。



DRCheckButton 方块型选择框控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRCheckButton 方块型选择框控件的命令和语法格式如下:

hl=DRCheckButton(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#003355'

cb1：次背景色，默认值='#007799'

cf：前景色，默认值='#ffff00'

txt：标题，默认值='A,B,C,D'，引号中单词数=选项数

fsize：字符点数，默认值=12

val：当前选项值，默认值=0

d：控件显示形式，默认值=0（0=垂直，1=水平）

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRCheckButton 方块型选择框控件有 2 个 get 属性函数，，功能如下：

txt=hl.getValueString()：获取当前选项菜单条文字

v=hl.getValueSingle()：获取当前选项菜单条号

下面是一个全参数的 DRCheckButton 方块型选择框控件创建实例：

```
hl=dr.DRCheckButton(win,74,80,120,120,'#003355','#007799','#ffff00','A,B,C,D',12,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。例如，下面创建一个水平显示的 DRCheckButton 方块型选择框控件：

```
hl=dr.DRCheckButton(win,55,424,260,30,d=1)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRCheckButton 方块型选择框控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，数字型格式为：

```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是当前选项号+vla。回调函数形式为：

```
def setValueSingle(v):
    ......
```

文本型回调函数绑定格式为：

```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的是当前选项的菜单条文字。回调函数形式为：

```
def setValueString(t):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。方块型选择框控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。选项号或条目可以通过单值端口(addCallBackSingle)或字符串端口(addCallBackString)输出。



下面是 DRCheckButton 方块型选择框控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个方块型选择框控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将方块型选择框控件的选项菜单条文字传递给标签控件，将选项值传递给自定义函数。DRCheckButton 方块型选择框控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入方块型选择框控件控件
mRadio=dr.DRCheckButton(win,73,244,120,120)
#==插入标签控件
```

```
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mRadio.addCallBackSingle(muserSingle3)
mRadio.addCallBackString(mLab.setValueString)
#==主程序循环
win.mainloop()
```

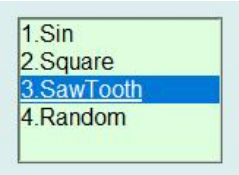
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRCheckButton 继承自 QCheckBox，因此除了用 DRCheckButton 的参数化外观定义外，还可以直接用 QCheckBox 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRListBox 列表选择框用法

DRListBox 列表选择框控件是将一组选项纵向排列在一起，供用户鼠标点击选择，然后通过回调函数形式编程定义其具体功能和操作，选第一项返回值为 0，选其余项的返回值依次递增 1。



DRListBox 列表选择框控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRListBox 列表选择框控件的命令和语法格式如下:

hl=DRListBox(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ddffdd'

cf：前景色，默认值='#000000'

txt：选项数组，默认值='1.Sin,2.Square,3.SawTooth,4.Random'，引号中单词数=选项数

fsize：字符点数，默认值=12

val：当前选项值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

列表控件有 4 个属性函数，功能如下：

hl.setValueSingle(v)：设定当前选项号

v=hl.getValueSingle()：获取当前选项号

txt=hl.getValueString()：获取当前选项菜单条

hl.updateItems(txt)：更新列表菜单，txt 为字符串，菜单项之间用逗号隔开，如：'A,B,C,D'

下面是一个全参数的 DRListBox 列表选择框控件创建实例：

```
hl=dr.DRListBox(win,111,88,140,100,'#ddffdd','#000000','1.Sin,2.Square,3.SawTooth,4.Random',12,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRListBox(win,111,88,140,100,val=2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRListBox 列表选择框控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，数字型格式为：

```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是当前选项号+vla。回调函数形式为：

```
def setValueSingle(v):
    ......
```

文本型回调函数绑定格式为：

```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的是当前选项的菜单条文字。回调函数形式为：

```
def setValueString(t):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。列表选择框控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。选项号或条目可以通过单值端口(addCallBackSingle)或字符串端口(addCallBackString)输出。



下面是 DRListBox 列表选择框控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个列表选择框控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRListBox 列表选择框控件的选项值传递给标签控件和自定义函数。DRListBox 列表选择框控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入列表选择框控件
mList=dr.DRListBox(win,111,88,140,100,val=2)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
```

```
#==绑定用户回调函数
mList.addCallBackSingle(muserSingle3)
mList.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

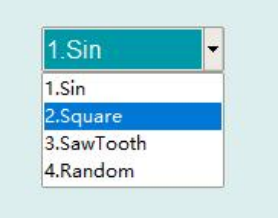
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRListBox 继承自 QListWidget，因此除了用 DRListBox 的参数化外观定义外，还可以直接用 QListWidget 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRComboBox 下拉列表选择框用法

DRComboBox 下拉列表选择框控件是将一组选项纵向排列在一起，供用户鼠标点击选择，然后通过回调函数形式编程定义其具体功能和操作，选第一项返回值为 0，选其余项的返回值依次递增 1。



DRComboBox 下拉列表选择框控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRComboBox 下拉列表选择框控件的命令和语法格式如下:

hl=DRComboBox(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#0099aa'

cf：前景色，默认值='#ffffff'

txt：选项数组，默认值='1.Sin,2.Square,3.SawTooth,4.Random'，引号中单词数=选项数

fsize：字符点数，默认值=12

val：当前选项值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRComboBox 下拉列表选择框控件有 4 个属性函数，功能如下：

hl.setValueSingle(v)：设定当前选项号

v=hl.getValueSingle()：获取当前选项号

txt=hl.getValueString()：获取当前选项菜单条

hl.updateItems(txt)：更新下拉菜单，txt 为字符串，菜单项之间用逗号隔开，如：'A,B,C,D'

下面是一个全参数的 DRComboBox 下拉列表选择框控件创建实例：

```
hl=dr.DRComboBox(win,77,54,120,30,'#0099aa','#ffffff','1.Sin,2.Square,3.SawTooth,4.Random',12,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRComboBox(win,106,166,120,30,val=1)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRComboBox 下拉列表选择框控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，数字型格式为：

hl.addCallBackSingle(fun)

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是当前选项号+vla。回调函数形式为：

```
def setValueSingle(v):
    ......
```

文本型回调函数绑定格式为：

```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的是当前选项的菜单条文字。回调函数形式为：

```
def setValueString(t):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。下拉列表框控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。选项号或条目可以通过单值端口(addCallBackSingle)或字符串端口(addCallBackString)输出。



下面是 DRComboBox 下拉列表选择框控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个下拉列表选择框控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRComboBox 下拉列表选择框控件的选项值传递给标签控件和自定义函数。DRComboBox 下拉列表选择框控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入下拉列表选择框控件
mComb=dr.DRComboBox(win,106,166,120,30,val=1)
#==插入标签控件
```

```
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mComb.addCallBackSingle(muserSingle3)
mComb.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

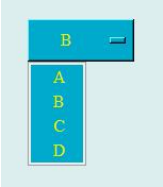
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRComboBox 继承自 QComboBox，因此除了用 DRComboBox 的参数化外观定义外，还可以直接用 QComboBox 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DROptionMenu 弹出式菜单按钮用法

DROptionMenu 弹出式菜单按钮控件是将一组选项纵向排列在一个按钮菜单内，供用户鼠标点击选择，然后通过回调函数形式编程定义其具体功能和操作，第一项返回值为 0，其余不同选项的返回值差值为 1。



DROptionMenu 弹出式菜单按钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DROptionMenu 弹出式菜单按钮控件的命令和语法格式如下:

hl=DROptionMenu(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

txt：选项数组，默认值='A,B,C,D'，引号中单词数=选项数

fsize：字符点数，默认值=12

val：当前选项值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DROptionMenu 弹出式菜单按钮控件有 3 个属性函数，功能如下：

v=hl.getValueSingle()：获取当前选项号

txt=hl.getValueString()：获取当前选项菜单条

hl.updateItems(txt)：更新下拉菜单，txt 为字符串，菜单项之间用逗号隔开，如：'A,B,C,D'

下面是一个全参数的 DROptionMenu 弹出式菜单按钮控件创建实例：

```
hl=dr.DROptionMenu(win,80,73,100,40,'#00aacc','#eeee00','A,B,C,D',12,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DROptionMenu(win,113,205,100,40,val=2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DROptionMenu 弹出式菜单按钮控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，数字型格式为：

```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是输入框中的数字。回调函数形式为：

```
def setValueSingle(v):
    ......
```

文本型回调函数绑定格式为：

```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的是输入框中的字符串输入信息。回调函数形式为：

```
def setValueString(txt):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。弹出式菜单按钮控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。选项号或条目可以通过单值端口(addCallBackSingle)或字符串端口(addCallBackString)输出。



下面是 DROptionMenu 弹出式菜单按钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个弹出式菜单按钮控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DROptionMenu 弹出式菜单按钮控件的选项值传递给标签控件和自定义函数。DROptionMenu 弹出式菜单按钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入弹出式菜单按钮控件
mOpt=dr.DROptionMenu(win,113,205,100,40,val=2)
#==插入标签控件
```

```
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mOpt.addCallBackSingle(muserSingle3)
mOpt.addCallBackString(mLab.setValueString)
#==主程序循环
win.mainloop()
```

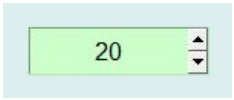
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DROptionMenu 继承自 QToolButton，因此除了用 DROptionMenu 的参数化外观定义外，还可以直接用 QToolButton 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRSpinBox 数字调节钮用法

DRSpinBox 数字调节钮控件数字调节钮控件是最常用的 GUI 控件之一，可用于快速调整输入的数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRSpinBox 数字调节钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRSpinBox 数字调节钮控件的命令和语法格式如下:

hl=DRSpinBox(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ccffcc'

cf：前景色，默认值='#000000'

val：初始值，默认值=2

step：增量步长，默认值=0.5

mlow：最小值，默认值=0

mhigh：最大值，默认值=10

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRSpinBox 数字调节钮控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值

v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRSpinBox 数字调节钮控件创建实例：

```
hl=dr.DRSpinBox(win,157,108,120,32,'#ccffcc','#000000',20,5,0,100,12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRSpinBox(win,183,206,120,32,val=5)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRSpinBox 数字调节钮控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

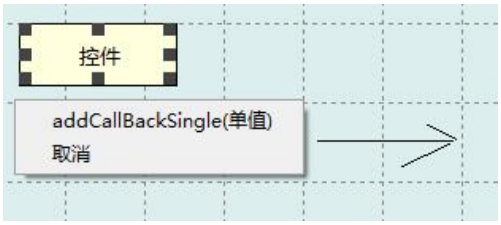
```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是输入框中的数字。回调函数形式为：

```
def setValueSingle(v):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。数字调节钮控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



数字调节钮控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRSpinBox 数字调节钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个数字调节钮控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRSpinBox 数字调节钮控件的值传递给标签控件和自定义函数。DRSpinBox 数字调节钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入数字调节钮控件
mDig=dr.DRSpinBox(win,50,40,120,32,val=5)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mDig.addCallBackSingle(muserSingle3)
mDig.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRSpinBox 继承自 QSpinBox，因此除了用 DRSpinBox 的参数化外观定义外，还可以直接用 QSpinBox 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRKnob 旋钮用法

DRKnob 旋钮控件提供一个可旋转调节数字量输入的控件，供用户鼠标拖动改变数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRKnob 旋钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRKnob 旋钮控件的命令和语法格式如下:

hl=DRKnob(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#004466'
cf：前景色，默认值='#222222'
cf1：前景色 1，默认值='#888888'
txt：表头量程显示文本，默认值='0,100'
mlow：最小值，默认值=0
mhigh：最大值，默认值=100
val：初始值，默认值=10
fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRKnob 旋钮控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值
v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRKnob 旋钮控件创建实例：

```
hl=dr.DRKnob(win,223,76,120,120,'#004466','#222222','#888888','0,100',0,100,20,12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRKnob(win,276,287,120,120,val=50)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRKnob 旋钮控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

```
hl.addCallBackSingle(fun)
```

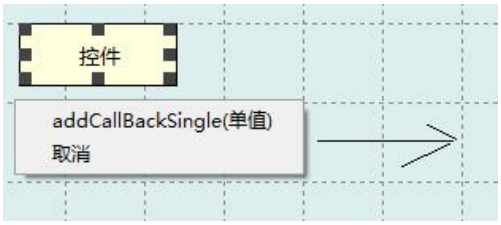
被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是输入框中的数字。回调函数形式为：

```
def setValueSingle(v):
```

.....

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。旋钮控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



旋钮控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRKnob 旋钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个旋钮控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRKnob 旋钮控件的值传递给标签控件和自定义函数。DRKnob 旋钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import DRKnob.DRKnobControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入旋钮控件
mKnob=dr.DRKnob(win,100,40,120,120,val=50)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mKnob.addCallBackSingle(muserSingle3)
mKnob.addCallBackSingle(mLab.setValueSingle)
```

```
#==主程序循环
win.mainloop()
```

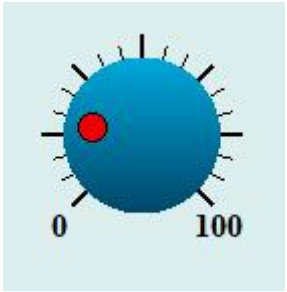
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRKnob 继承自 QDial，因此除了用 DRKnob 的参数化外观定义外，还可以直接用 QDial 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRGKnob 渐变色旋钮用法

DRGKnob 渐变色旋钮控件提供一个可旋转调节数字量输入的控件，供用户鼠标拖动改变数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRGKnob 渐变色旋钮控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRGKnob 渐变色旋钮控件的命令和语法格式如下:

hl=DRGKnob(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aadd'

cb1：背景色 1，默认值='#004466'

cf：前景色，默认值='#000000'

cf1：前景色 1，默认值='#ee0000'

txt：表头量程显示文本，默认值='0,100'

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=10

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRGKnob 渐变色旋钮控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值
v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRGKnob 渐变色旋钮控件创建实例：

```
hl=dr.DRGKnob(win,162,91,120,120,'#00aadd','#004466','#000000','#ee0000','0,100',0,100,20,12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRGKnob(win,209,297,120,120,val=50)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRGKnob 渐变色旋钮控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

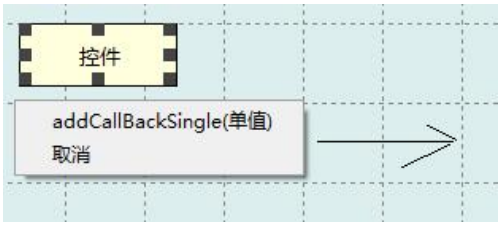
```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是输入框中的数字。回调函数形式为：

```
def setValueSingle(v):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。渐变色旋钮控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



渐变色旋钮控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRGKnob 渐变色旋钮控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个渐变色旋钮控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRGKnob 渐变色旋钮控件的值传递给标签控件和自定义函数。DRGKnob 渐变色旋钮控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入渐变色旋钮控件
mKnob=dr.DRGKnob(win,100,40,120,120,val=50)
```

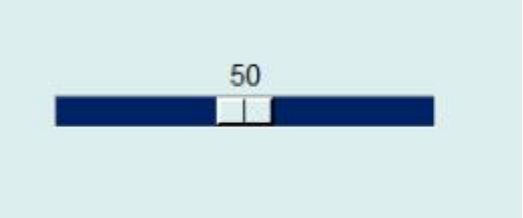
```
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mKnob.addCallBackSingle(muserSingle3)
mKnob.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRGKnob 继承自 QWidget，不支持样式表。

DRHScale 水平滚动条用法

DRHScale 水平滚动条控件提供一个可拖动调节数字量输入的控件，供用户鼠标拖动改变数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRHScale 水平滚动条控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRHScale 水平滚动条控件的命令和语法格式如下:

hl=DRHScale(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#002266'

cf：前景色，默认值='#000000'

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=10

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRHScale 水平滚动条控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值

v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRHScale 水平滚动条控件创建实例：

```
hl=dr.DRHScale(win,133,126,200,50,'#002266','#000000',0,100,50,11)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRHScale(win,167,262,200,50,val=10)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRHScale 水平滚动条控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

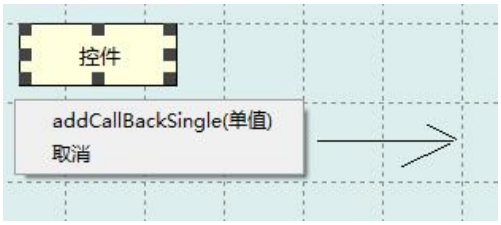
```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的滑块位置数字。回调函数形式为：

```
def setValueSingle(v):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。水平滚动条控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



水平滚动条控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRHScale 水平滚动条控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个水平滚动条控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRHScale 水平滚动条控件的值传递给标签控件和自定义函数。DRHScale 水平滚动条控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入水平滚动条控件
mS=dr.DRHScale(win,167,262,200,50,val=10)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mS.addCallBackSingle(muserSingle3)
mS.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRHScale 继承自 QScrollBar，因此除了用 DRHScale 的参数化外观定义外，还可以直接用 QScrollBar 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRVScale 垂直滚动条用法

DRVScale 垂直滚动条控件提供一个可拖动调节数字量输入的控件，供用户鼠标拖动改变数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRVScale 垂直滚动条控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRVScale 垂直滚动条控件的命令和语法格式如下:

hl=DRVScale(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#002266'

cf：前景色，默认值='#000000'

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=10

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRVScale 垂直滚动条控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值

v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRVScale 垂直滚动条控件创建实例：

```
hl=dr.DRVScale(win,100,100,50,200,'#002266','#000000',0,100,50,11)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRHScale(win,167,262,200,50,val=10)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRVScale 垂直滚动条控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

hl.addCallBackSingle(fun)

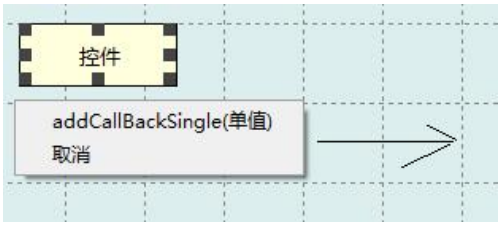
被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的滑块位置数字。回调函数形式为：

def setValueSingle(v):

.....

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。垂直滚动条控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



垂直滚动条控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRVScale 垂直滚动条控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个垂直滚动条控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRVScale 垂直滚动条控件的值传递给标签控件和自定义函数。DRVScale 垂直滚动条控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入垂直滚动条控件
```

```
mS=dr.DRVScale(win,100,100,50,200,val=10)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mS.addCallBackSingle(muserSingle3)
mS.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

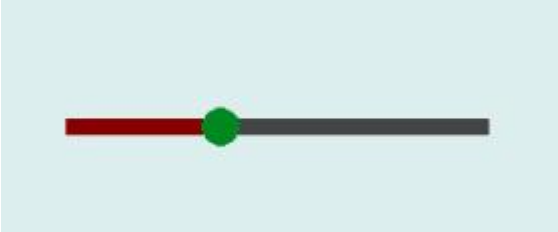
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRVScale 继承自 QScrollBar，因此除了用 DRVScale 的参数化外观定义外，还可以直接用 QScrollBar 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRHSlider 水平滑块用法

DRHSlider 水平滑块控件提供一个可拖动调节数字量输入的控件，供用户鼠标拖动改变数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRHSlider 水平滑块控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRHSlider 水平滑块控件的命令和语法格式如下:

hl=DRHSlider(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#444444'

cf：前景色，默认值='#880000'

cf1：前景色 1，默认值='#008822'

dh：滑轨宽度，默认值=8

dw：原型滑块直径，默认值=20

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=10

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRHSlider 水平滑块控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值
v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRHSlider 水平滑块控件创建实例：

```
hl=dr.DRHSlider(win,222,86,260,30,'#444444','#880000','#008822',8,20,0,100,20)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRHSlider(win,222,86,260,30,val=10)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRHSlider 水平滑块控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

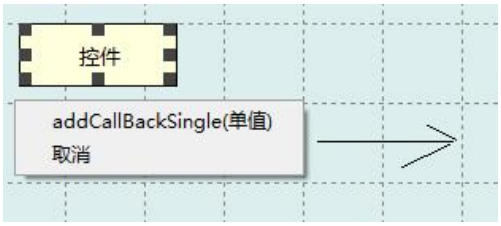
```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的滑块位置数字。回调函数形式为：

```
def setValueSingle(v):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。水平滑块控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



水平滑块控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRHSlider 水平滑块控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个水平滑块控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRHSlider 水平滑块控件的值传递给标签控件和自定义函数。DRHSlider 水平滑块控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入水平滑块控件
mS=dr.DRHSlider(win,222,86,260,30,val=10)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
```

```
mS.addCallBackSingle(muserSingle3)
mS.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

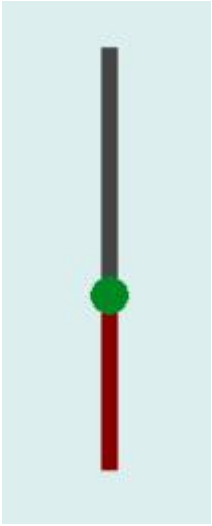
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRHSlider 继承自 QSlider，因此除了用 DRHSlider 的参数化外观定义外，还可以直接用 QSlider 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRVSlider 垂直滑块用法

DRVSlider 垂直滑块控件提供一个可拖动调节数字量输入的控件，供用户鼠标拖动改变数字值，然后将数值通过回调函数或直连形式传递给下一模块，驱动其运行。



DRVSlider 垂直滑块控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRVSlider 垂直滑块控件的命令和语法格式如下:

hl=DRVSlider(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#444444'

cf：前景色，默认值='#880000'

cf1：前景色 1，默认值='#008822'

dh：滑轨宽度，默认值=8

dw：原型滑块直径，默认值=20

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=10

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRVSlider 垂直滑块控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值

v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRVSlider 垂直滑块控件创建实例：

```
hl=dr.DRVSlider(win,357,278,30,260,'#444444','#880000','#008822',8,20,0,100,20)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRVSlider(win,357,278,30,260,val=10)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRVSlider 垂直滑块控件可以通过回调函数绑定的方式将用户的输入传递给绑定的回调函数，格式为：

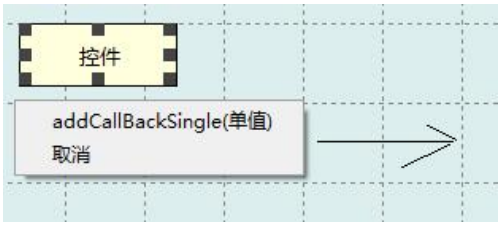
```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的滑块位置数字。回调函数形式为：

```
def setValueSingle(v):
    ......
```

通过回调函数绑定方式，就可以将用户的输入传递到自定义的回调函数中或传递给直连的另一控件。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。垂直滑块控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件值可以通过单值端口(addCallBackSingle)输出。



垂直滑块控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRVSlider 垂直滑块控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个垂直滑块控件，一个标签控件、一个用户自定义函数，然后通过直连绑定方式，将 DRVSlider 垂直滑块控件的值传递给标签控件和自定义函数。DRVSlider 垂直滑块控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
```

```
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入垂直滑块控件
mS=dr.DRVSlider(win,50,50,30,260,val=10)
#==插入标签控件
mLab=dr.DRLabel(win,221,42,200,30,fsize=16)
#==绑定用户回调函数
mS.addCallBackSingle(muserSingle3)
mS.addCallBackSingle(mLab.setValueSingle)
#==主程序循环
win.mainloop()
```

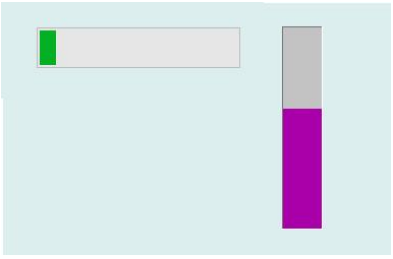
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRVSlider 继承自 QSlider，因此除了用 DRVSlider 的参数化外观定义外，还可以直接用 QSlider 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRProgress 进度条用法

DRProgress 进度条控件进度条控件主要用于指示某项任务的完成进程。同时，进度条控件也可按设定的时间间隔自动增长，指示程序正在运行，未死机。



DRProgress 进度条控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRProgress 进度条控件的命令和语法格式如下:

hl=DRProgress(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00dd00'

cf：前景色，默认值='#003355'

d：控件显示形式，默认值=0（0=垂直，1=水平）

dt：时间间隔，默认值=50 毫秒

val：初始值，默认值=0，0/100

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRProgress 进度条控件有 3 个属性函数，功能如下：

hl.setVSOnOff(v)：设定控件自动运行，0/1=运行/停止
hl.setVSPosition(v)：设定控件位置，0/100
v=hl.getVSPosition()：获取控件位置

下面是一个全参数的 DRProgress 水平显示的进度条控件创建实例：

```
hl=dr.DRProgress(win,373,80,200,40,'#00dd00','#003355',0,50,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个垂直显示的进度条创建实例：

```
hl=dr.DRProgress(win,446,197,40,200,d=1,val=50)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。进度条控件可以做为输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。单值端口(setVSOnOff)连接在控件开关属性上，收到数据流后，控件在运行/停止间切换；从单值端口(setVSPosition)收到数据流后，则做为控件的显示值。



下面是 DRProgress 进度条控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个进度条控件，一个滑块控件、一个用户自定义函数，然后通过直连绑定方式，用滑块控件来控制进度条的显示进度。DRProgress 进度条控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    x=mP1.getVSPosition()
    print(x)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入进度条控件
mP1=dr.DRProgress(win,422,63,200,40,d=0,val=0)
#==插入滑块控件
mS2=dr.DRHSlider(win,99,66,260,30,val=0)
#==绑定用户回调函数
mS2.addCallBackSingle(mP1.setVSPosition)
mS2.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

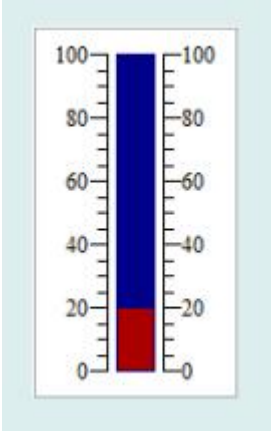
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRProgress 继承自 QProgressBar，因此除了用 DRProgress 的参数化外观定义外，还可以直接用 QProgressBar 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRRuler 棒图用法

DRRuler 棒图控件主要用于指示某个参数的值，如温度、压力等。



DRRuler 棒图控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRRuler 棒图控件的命令和语法格式如下:

hl=DRRuler(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ffffff'

cf：前景色，默认值='#000088'

cf1：前景色 1，默认值='#000000'

cf2：前景色 2，默认值='#aa0000'

cf3：前景色 3，默认值='#aaaaaa'

txt：刻度数组，默认值='100,80,60,40,20,0'

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=20

fsize：字符点数，默认值=10

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

棒图控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值

v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRRuler 棒图控件创建实例：

```
hl=dr.DRRuler(win,562,180,110,200,'#ffffff','#000088','#000000','#aa0000','#aaaaaa','100,80,60,40,20,0',0,100,20,10)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=mRuler1=dr.DRRuler(win,562,180,110,200,val=50)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。棒图控件为单数值输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRRuler 棒图控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个棒图控件，一个滑块控件、一个用户自定义函数，然后通过直连绑定方式，用滑块控件来控制棒图控件的显示值。DRRuler 棒图控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    x=mR1.getValueSingle()
    print(x)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入棒图控件
mR1=dr.DRRuler(win,562,180,110,200,val=0)
#==插入滑块控件
mS2=dr.DRHSlider(win,99,66,260,30,val=0)
#==绑定用户回调函数
mS2.addCallBackSingle(mR1.setValueSingle)
mS2.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRRuler 继承自 QWidget，不支持样式表。

DRHRuler 水平标尺用法

DRHRuler 水平标尺控件主要用于指示某个参数的值，如温度、压力等。



DRHRuler 水平标尺控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRHRuler 水平标尺控件的命令和语法格式如下:

hl=DRHRuler(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#eeeeee'

cf：前景色，默认值='#000088'

cf1：前景色 1，默认值='#000000'

cf2：前景色 2，默认值='#aa0000'

cf3：前景色 3，默认值='#aaaaaa'

txt：刻度数组，默认值='100,80,60,40,20,0'

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=20

fsize：字符点数，默认值=10

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

水平标尺控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值
v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRHRuler 水平标尺控件创建实例：

```
hl=dr.DRHRuler(win,533,57,200,80,'#eeeeee','#000088','#000000','#aa0000','#aaaaaa','100,80,60,40,20,0',0,100,20,10)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRHRuler(win,533,57,200,80,val=50)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。水平标尺控件为单数值输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则

弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRHRuler 水平标尺控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个水平标尺控件，一个滑块控件、一个用户自定义函数，然后通过直连绑定方式，用滑块控件来控制水平标尺控件的显示值。DRHRuler 水平标尺控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    x=mR1.getValueSingle()
    print(x)

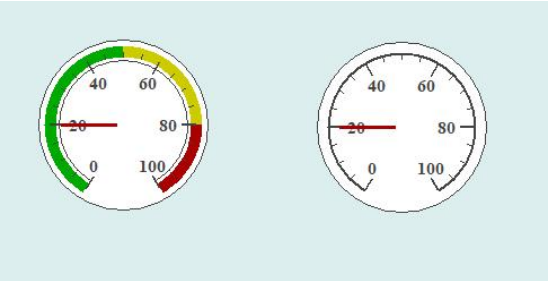
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入水平标尺控件
mR1=dr.DRHRuler(win,533,57,200,80,val=0)
#==插入滑块控件
mS2=dr.DRHSlider(win,99,66,260,30,val=0)
#==绑定用户回调函数
mS2.addCallBackSingle(mR1.setValueSingle)
mS2.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRHRuler 继承自 QWidget，不支持样式表。

DRGauge 表盘用法

DRGauge 表盘控件主要用于指示某个参数的值，如温度、压力等。



DRGauge 表盘控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRGauge 表盘控件的命令和语法格式如下:

hl=DRGauge(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ffffff'

cf：前景色，默认值='#444444'

cf1：前景色 1，默认值='#ffffff'

cf2：前景色 2，默认值='#aa0000'

txt：刻度数组，默认值='100,80,60,40,20,0'

mlow：最小值，默认值=0

mhigh：最大值，默认值=100

val：初始值，默认值=20

st：样式，默认值=1，0=简易型，1=标准型

fsize：字符点数，默认值=10

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

表盘控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值
v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRGauge 表盘控件创建实例：

```
hl=dr.DRGauge(win,494,179,160,160,'#ffffff','#444444','#ffffff','#aa0000','100,80,60,40,20,0',0,100,20,1,11)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRGauge(win,494,179,160,160,st=0,val=50)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。表盘控件为单数值输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹

出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRGauge 表盘控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个表盘控件，一个滑块控件、一个用户自定义函数，然后通过直连绑定方式，用滑块控件来控制表盘控件的显示值。DRGauge 表盘控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    x=mG1.getValueSingle()
    print(x)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入表盘控件
mG1=dr.DRGauge(win,494,179,160,160,val=0)
#==插入滑块控件
mS2=dr.DRHSlider(win,99,66,260,30,val=0)
#==绑定用户回调函数
mS2.addCallBackSingle(mG1.setValueSingle)
mS2.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRGauge 继承自 QWidget，不支持样式表。

DRDigital 数字显示控件用法

DRDigital 数字显示控件主要用于指示某个参数的值，如温度、压力等。



DRDigital 数字显示控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRDigital 数字显示控件的命令和语法格式如下:

hl=DRDigital(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#000000'

cf：前景色，默认值='#00ffaa'

val：初始值，默认值=0

dec：小数点后位数，默认值=2

fsize：字符点数，默认值=22

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

数字显示控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值

v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRDigital 数字显示控件创建实例：

```
hl=dr.DRDigital(win,230,173,100,36,'#000000','#00ffaa',0,2,22)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRDigital(win,230,173,100,36,val=10)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。数字显示控件为单数值输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示值。



下面是 DRDigital 数字显示控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个数字显示控件，一个滑块控件、一个用户自定义函数，然后通过直连绑定方式，用滑块控件来控制数字显示控件的显示值。DRDigital 数字显示控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    x=mD1.getValueSingle()
    print(x)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入数显控件
mD1=dr.DRDigital(win,230,173,100,36,val=0)
#==插入滑块控件
mS2=dr.DRHSlider(win,99,66,260,30,val=0)
#==绑定用户回调函数
mS2.addCallBackSingle(mD1.setValueSingle)
mS2.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，该控件的名称为 DRLCDNumber，继承自 QLCDNumber，因此除了用 DRDigital 的参数化外观定义外，还可以直接用 QLCDNumber 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

因继承自 QLCDNumbe 的缘故，DRLCDNumbe 数字显示控件与 TK 版的创建部分参数略有不同，的命令和语法格式如下:

hl=DRLCDNumbe(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#000000'

cf：前景色，默认值='#00ffaa'

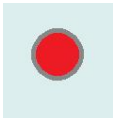
L：数字量字符长度

val：初始值，默认值=0

属性参数和连线方式等则与 TK 版的 DRDigital 相同。

DRLamp 警示灯用法

DRLamp 警示灯控件警示灯控件可以用绿、黄、红，或自定义色彩来指示工作状态。



DRLamp 警示灯控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRLamp 警示灯控件的命令和语法格式如下:

hl=DRLamp(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aa00'

cb1：背景色 1，默认值='#cccc00'

cb2：背景色 2，默认值='#cc0000'

cf：前景色，默认值='#888888'

lw：边框宽度，默认值=5

val：灯的色彩状态，默认值=0，取值范围：0，1，2

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

警示灯控件有 2 个属性函数，功能如下：

hl.setValueSingle(v)：设定控件值
v=hl.getValueSingle()：取控件值

下面是一个全参数的 DRLamp 警示灯控件创建实例：

```
hl=dr.DRLamp(win,448,114,60,60,'#00aa00','#cccc00','#cc0000','#888888',5,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRLamp(win,448,114,60,60,val=2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。警示灯控件为单数值输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从单值端口(setValueSingle)收到数据流后，则做为控件的显示状态切换值。



下面是 DRLamp 警示灯控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个警示灯控件，一个多联开关控件，然后通过直连绑定方式，用多联开关控件来控制警示灯的色彩。DRLamp 警示灯控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

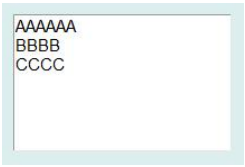
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入警示灯控件
mL1=dr.DRLamp(win,448,114,60,60,val=0)
#==插入开关组控件
mB2=dr.DRHButtonGroup(win,60,127,240,36,txt='绿,黄,红')
#=直连绑定
mB2.addCallBackSingle(mL1.setValueSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRLamp 继承自 QWidget，不支持样式表。

DRText 文本控件用法

DRText 文本控件文本框控件可用于显示或输入多行文字或数字信息。



DRText 文本控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRText 文本控件的命令和语法格式如下:

hl=DRText(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ffffff'

cf：前景色，默认值='#000000'

txt：显示文字，默认值=''

fsize：字符点数，默认值=11

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

文本控件有 2 个属性函数，功能如下：

hl.setValueString(v)：设定所显示的文字
v=hl.getValueString()：获取所显示的文字

下面是一个全参数的 DRText 文本控件创建实例：

```
hl=dr.DRText(win,477,82,120,100,'#ffffff','#000000','AA\nBB\nCC',11)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRText(win,477,82,120,100)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。文本控件为字符串输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。从字符串端口(setValueString)收到数据流后，则做为文本控件的内容显示。



下面是 DRText 文本控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个文本控件，一个开关控件，一个用户自定义函数。点击按钮，则通过用户自定义函数在文本控件中写入内容。DRText 文本控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    s="A\nB\nC\nD"
    mText.setValueString(s)
    print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,50,42,100,40,txt='Run')
#==插入文本控件
mText=dr.DRText(win,200,82,120,100)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

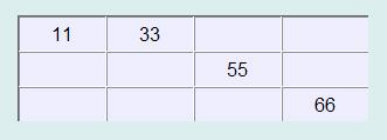
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRText 继承自 QPlainTextEdit，因此除了用 DRText 的参数化外观定义外，还可以直接用 QPlainTextEdit 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRList 表格控件用法

DRList 表格控件可以用列表形式显示和输入多个行文字或数字，实质就是一组文本输入框控件。



DRList 表格控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRList 表格控件的命令和语法格式如下:

hl=DRList(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#eeeeff'

cf：前景色，默认值='#000000'

N：行，默认值=3

M：列，默认值=4

fsize：字符点数，默认值=12

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRList 表格控件有 2 个属性函数，功能如下：

hl.setItems(i,j,txt)：设定各单元格所显示的文字

txt=hl.getItems(i,j)：获取各单元格所显示的文字

下面是一个全参数的 DRList 表格控件创建实例：

```
hl=dr.DRList(win,302,83,300,90,'#eeeeff','#000000',3,4,12)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRList(win,302,83,300,90,N=2,M=3)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

下面是 DRList 表格控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个表格控件，一个开关控件，一个用户自定义函数。点击按钮，则通过用户自定义函数在表格控件中写入内容。DRList 表格控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import DRList.DRListControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    mL.setItems(0,0,'A')
    print(v)

#==创建 Tkinter GUI 窗口
```

```
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,50,42,100,40,txt='Run')
#==插入列表控件
mL=dr.DRList(win,302,83,300,90,N=2,M=3)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

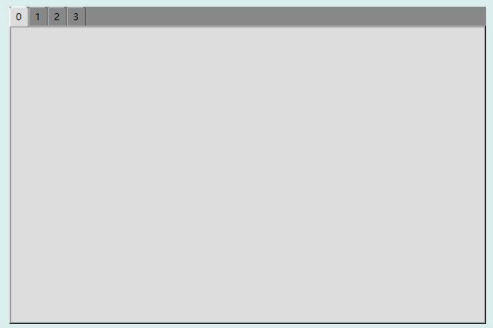
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRList 继承自 QTableWidget，因此除了用 DRList 的参数化外观定义外，还可以直接用 QTableWidget 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRNoteBook 标签页用法

DRNoteBook 标签页控件的作用是提供一个多页的页帧，可以在同一位置显示多个不同的页面。



DRNoteBook 标签页控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRNoteBook 标签页控件的命令和语法格式如下:

hl=DRNoteBook(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#dddddd'

cb1：背景色 1，默认值='#888888'

cf：前景色，默认值='#000000'

txt：页帧标题，默认值=’A,B,C,D’

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

一个标签页控件可以提供多个不同的页面{hl.win[0],...,hl.win[N-1]}来布置 GUI 和插入控件。例如，下面分别在第一页和第二页中各插入一个按钮：

```
mB0=dr.DRButton(hl.win[0],40,40,100,36)
mB1=dr.DRButton(hl.win[1],40,40,100,36)
```

DRNoteBook 标签页控件有 1 个属性函数，功能如下：

hl.setPage(v)：设定当前的显示页帧

下面是一个全参数的 DRNoteBook 标签页控件创建实例：

```
hl=dr.DRNoteBook(win,123,65,600,400,'#dddddd','#888888','#000000')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRNoteBook(win,123,65,600,400)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

下面是 DRNoteBook 标签页控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个 2 页的标签页控件，在第一页插入一个按钮组控件，在第二页插入一个文本框控件。点击按钮组中的按钮，则文本框中显示当前点击的按钮值。DRNoteBook 标签页控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import DRNoteBook.DRNoteBookControlls as dr
```

```
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mN=dr.DRNoteBook(win,123,65,600,400,N=2)
#==在第一页插入按钮组控件
mB1=dr.DRHButtonGroup(mN.win[0],40,40,240,36,txt='A,B,C,D')
#==在第二页插入文本框控件
mE2=dr.DREntry(mN.win[1],40,40,100,30)
#==控件绑定
mB1.addCallBackSingle(mE2.setValueSingle)
#==主程序循环
win.mainloop()
```

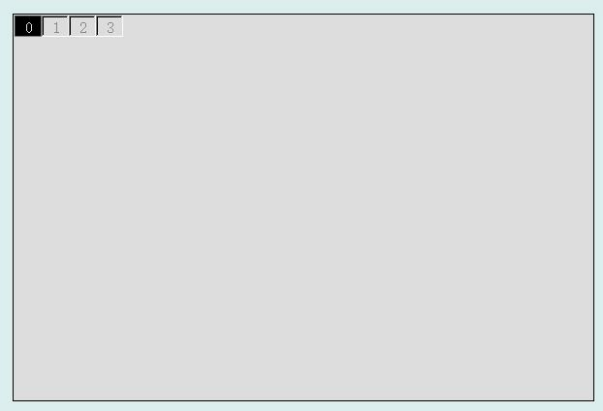
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRNoteBook 继承自 QTableWidget，因此除了用 DRNoteBook 的参数化外观定义外，还可以直接用 QTableWidget 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRTab 紧缩式标签页用法

DRTab 紧缩式标签页控件的作用是提供一个多页的页帧，可以在同一位置显示多个不同的页面。



DRTab 紧缩式标签页控件的图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRTab 紧缩式标签页控件的命令和语法格式如下:

hl=DRTab(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#dddddd'

cb1：背景色 1，默认值='#888888'

cf：前景色，默认值='#000000'

N：页帧数，默认值=4

p：标签位置，默认值=0。0=左上，1=右上，2=左下，3=右下。

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

一个紧缩式标签页控件可以提供多个不同的页面{hl.win[0],...,hl.win[N-1]}来布置 GUI 和插入控件。例如，下面分别在第一页和第二页中各插入一个按钮：

```
mB0=dr.DRButton(hl.win[0],40,40,100,36)
mB1=dr.DRButton(hl.win[1],40,40,100,36)
```

DRTab 紧缩式标签页控件有 1 个属性函数，功能如下：

hl.setPage(v)：设定当前的显示页帧

下面是一个全参数的 DRTab 紧缩式标签页控件创建实例：

```
hl=dr.DRTab(win,132,73,600,400,'#dddddd','#888888','#000000',4,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRTab(win,132,73,600,400,p=2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

下面是 DRTab 紧缩式标签页控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个 2 页的紧缩式标签页控件，在第一页插入一个按钮组控件，在第二页插入一个文本框控件。点击按钮组中的按钮，则文本框中显示当前点击的按钮值。DRTab 紧缩式标签页控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import DRTab.DRTabControlls as dr

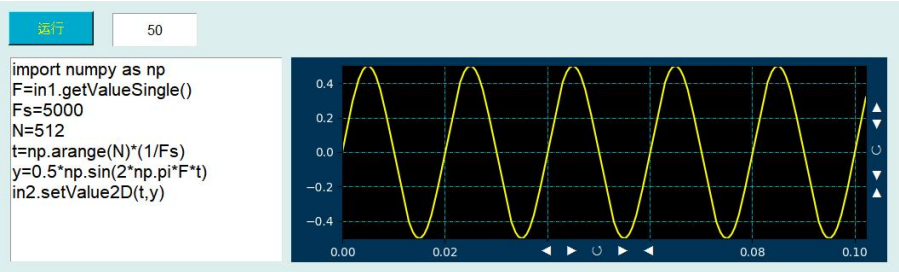
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入标签页控件
mN=dr.DRTab(win,123,65,600,400,p=3)
#==在第一页插入按钮组控件
mB1=dr.DRHButtonGroup(mN.win[0],40,40,240,36,txt='A,B,C,D')
#==在第二页插入文本框控件
mE2=dr.DREntry(mN.win[1],40,40,100,30)
#==控件绑定
mB1.addCallBackSingle(mE2.setValueSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

在 DRVI 的 pyqt 版本中无 DRTab 控件，可用 DRNoteBook 代替。

DRScript 脚本控件用法

DRScript 脚本控件用于在 GUI 界面上显示一段可以做为程序一部分参与运行的 Python 小程序代码。当该控件被数据流驱动时，控件中显示的小程序可以接收输入参数，然后被调用和执行。通常用于课堂教学时在线修改小程序，并实时查看运行效果。



DRScript 脚本控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRScript 控件的命令和语法格式如下:

hl=DRScript(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#ffffff'

cf：前景色，默认值='#000000'

txt：初始脚本，默认值='print(0)'

fsize：字符点数，默认值=11

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

文本控件有 2 个属性函数，功能如下：

hl.setValueString(v)：设定控件中的小程序

v=hl.getValueString()：获取控件中的小程序

out=doScript(in1=None,in2=None,in3=None,in4=None,in5=None,in6=None,in7=None,in8=None)：

通过参数 in1,....,in8 获取主程序中的变量，通过 out1,...,out8 将小程序运行结果返回。

下面是一个全参数的 DRScript 控件创建实例：

```
hl=dr.DRScript(win,170,165,600,200,'#ffffff','#000000',txt='print(1)',fsize=16)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。下面是一个创建实例：

```
hl=dr.DRScript(win,477,82,120,100)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

下面是 DRScript 控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个输入框控件，一个脚本控件，一个绘图控件和一个用户自定义函数。点击按钮，则通过用户自定义函数执行脚本控件。小程序代码的功能是从输入框控件获取信号频率值，生成正弦波，然后调用绘图控件绘图。

DRScript 控件应用示例小程序代码如下：

```
import tkinter as tk
```

```
import drvi.drviControlls as dr

def setRun(v):
    out=mScript3.doScript(in1=mEntry4,in2=mPlot2)

#=初始脚本====
S="import numpy as np\n"
S=S+"F=in1.getValueSingle()\n"
S=S+"Fs=5000\n"
S=S+"N=512\n"
S=S+"t=np.arange(N)*(1/Fs)\n"
S=S+"y=0.5*np.sin(2*np.pi*F*t)\n"
S=S+"in2.setValue2D(t,y)\n"
#==Main Window====================
win= tk.Tk()
win.wm_title('Main')
winWidthCur=1201
winHeightCur=716
win.geometry('1201x716')
win.config(bg="#ddeeee")
#==Layout of controls=================
mButton1=dr.DRButton(win,28,26,100,40,txt='运行')
mPlot2=dr.DRPlot(win,360,80,700,240)
mScript3=dr.DRScript(win,30,80,320,240,txt=S,fsize=16)
mEntry4=dr.DREntry(win,150,28,100,40,txt='50')
#=User defined data probe============
mButton1.addCallBackSingle(setRun)
#==Main Loop====================
win.mainloop()
```

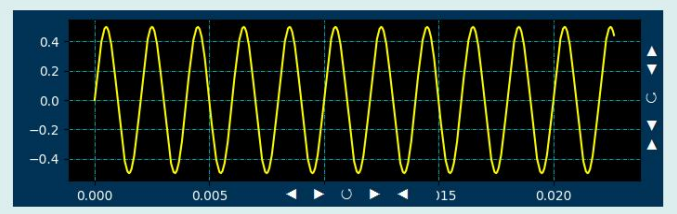
可参考实验案例：“204-TK 图象的 OpenCV 脚本程序.py”

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局、连线和运行完全相同的。

DRPlot 二维曲线控件用法

DRPlot 二维曲线控件主要用于显示信号波形、频谱等二维曲线。该函数做了速度优化，在曲线类显示时，尽量使用该函数。



DRPlot 二维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlot 二维曲线控件的命令和语法格式如下:

hl=DRPlot(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

xtype：曲线横坐标类型，默认值=0；0/1=线性/对数

ytype：曲线纵坐标类型，默认值=0；0/1=线性/对数

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。控件创建时，当 xlow 和 xhigh 均设为零时，横坐标轴采用自动量程；当 ylow 和 yhigh 均设为零时，纵坐标采用自动量程；当 xarray 和 yarray 设为零，则初始曲线为空。另外，在自动量程时，DRPlot 二维曲线控件中的曲线放大、缩小、滚动等操作符无效。

DRPlot 二维曲线控件有 6 个属性函数，其功能定义如下：

hl.setXlim(xmin,xmax)：设定 X 轴显示范围

hl.setYlim(ymin,ymax)：设定 Y 轴显示范围

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

hl.setValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以覆盖形式重新绘制曲线

hl.addValue2D(xarray,yarray,c=None,lw=None)：设定二维曲线数据，并以添加形式绘制新曲线，c 为色彩码，如 c='#ff0000'。lw 为线宽。

hl.addValue2DX(dx,yarray,c=None,lw=None)：设定等离散采样间隔二维曲线数据，并以添加形式绘制新曲线，c 为色彩码，如 c='#ff0000'。lw 为线宽。

下面是一个全参数的 DRPlot 二维曲线控件创建实例：

```
hl=dr.DRPlot(win,95,122,700,210,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlot(win,95,122,700,210,xlow=0,xhigh=0.05)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlot 二维曲线控件为输入控件，可接收 2 种输入形式，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)和端口(setValue2DX)收到的是控件的曲线显示数据。



下面是 DRPlot 二维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个自定义函数和一个 DRPlot 二维曲线控件，然后通过直连绑定方式，用按钮控件来驱动自定义函数生成信号，用 DRPlot 二维曲线控件来显示信号波形。DRPlot 二维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+10
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    mPlot.setValue2D(t,y)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
```

```
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 Plot 控件
mPlot=dr.DRPlot(win,95,122,700,210,xlow=0,xhigh=0.05)
#==控件直连绑定
mB1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，该控件的名称为 DRPlotFast，继承自 QChartView，其创建参数与继承自 Matplotlib 的 DRPlot 参数略有不同(pyqt 版中 DRPlot 仍然可用，但显示速度比 DRPlotFast 慢很多)，创建的命令和语法格式如下:

hl=DRPlotFast(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

st：曲线外观定义，默认值=0

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

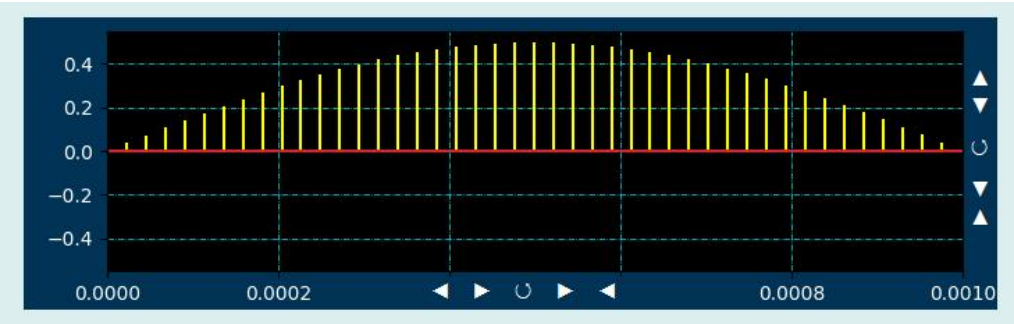
xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

DRPlotFast 控件的属性参数和连线方式等则与 TK 版的 DRPlot 相同。

DRStem 二维曲线控件用法

DRStem 二维曲线控件也可以用于显示信号波形、频谱等二维曲线，但曲线绘制形式不同，不是点间连线，而是点的高度线。



DRStem 二维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRStem 二维曲线控件的命令和语法格式如下:

hl=DRStem(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

xtype：曲线横坐标类型，默认值=0；0/1=线性/对数

xtype：曲线纵坐标类型，默认值=0；0/1=线性/对数

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

控件创建时，当 xlow 和 xhigh 均设为零时，横坐标轴采用自动量程；当 ylow 和 yhigh 均设为零时，纵坐标采用自动量程；当 xarray 和 yarray 设为零，则初始曲线为空。另外，在自动量程时，DRStem 二维曲线控件中的曲线放大、缩小、滚动等操作符无效。

DRStem 二维曲线控件有 6 个属性函数，其功能定义如下：

hl.setXlim(xmin,xmax)：设定 X 轴显示范围

hl.setYlim(ymin,ymax)：设定 Y 轴显示范围

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

hl.setValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以覆盖形式重新绘制曲线

hl.addValue2D(xarray,yarray)：设定二维曲线数据，并以添加形式绘制新曲线。

hl.addValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以添加形式绘制新曲线

下面是一个全参数的 DRStem 二维曲线控件创建实例：

```
hl=dr.DRStem(win,97,88,700,210,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.001,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRStem(win,97,88,700,210,cb1='#005533')
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRStem 二维曲线控件为输入控件，可接收 2 种输入形式，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)和端口(setValue2DX)收到的是控件的曲线显示数据。



下面是 DRStem 二维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个自定义函数和一个 DRStem 二维曲线控件，然后通过直连绑定方式，用按钮控件来驱动自定义函数生成信号，用 DRStem 二维曲线控件来显示信号波形。DRStem 二维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+10
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    mStem.setValue2D(t,y)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 Stem 控件
```

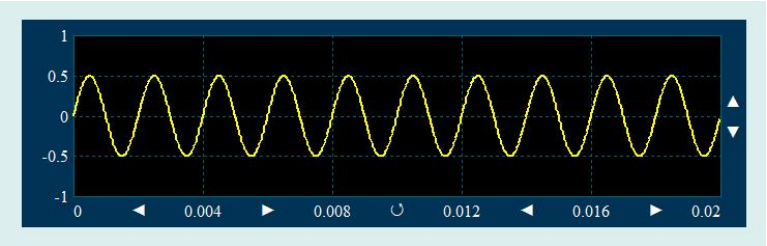
```
mStem=dr.DRStem(win,95,122,700,210,xlow=0,xhigh=0.05)
#==控件直连绑定
mB1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlotFast 二维曲线控件

DRPlotFast 二维曲线控件是一个简化版的 Plot 控件，主要用于信号采集等需要 实时滚动显示信号波形、频谱等场合。这时，若用构建在 Matplotlib 基础上的 Pl ot 控件，曲线刷新较慢，程序运行可能会卡顿。但解决的不好，曲线锯齿感、闪 烁感较重，不建议用。有速度需求的用户，建议用 DRVIqt 版。



PlotFast 二维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlotFast 二维曲线控件的命令和语法格式如下:

hl=DRPlotFast(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

fsize：字符点数，默认值=12

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0.05

ylow：纵坐标最小值，默认值=-1

yhigh：纵坐标最大值，默认值=1

dx：离散采样间隔，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。注意 PlotFast 不支持自动量程，xmin, xmax, ymin, ymax 必须设为实值。当 dx 和 yarray 设为零时，初始曲线为空。

DRPlotFast 控件有 2 个属性函数，其功能定义如下：

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

hl.setValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以覆盖形式重新绘制曲线

下面是一个全参数的 DRPlotFast 二维曲线控件创建实例：

```
hl=DRPlotFast(win,44,106,700,200,'#003355','#000000','#ffffff','#006677','#ffff00',12,0,0.02,-1,1,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlotFast(win,44,106,700,200,xlow=0,xhigh=0.05,ylow=-1.2,yhigh=1.2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

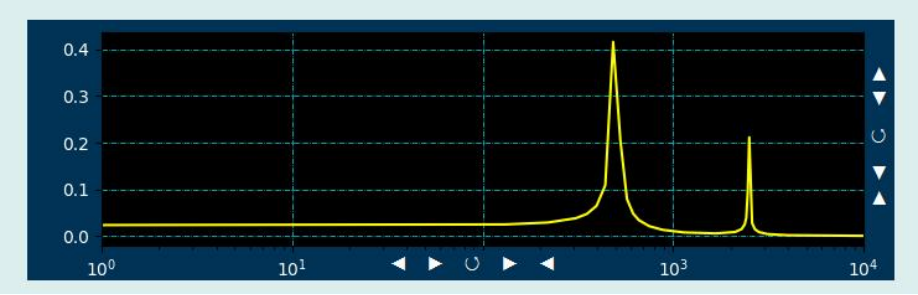
下面是 DRPlotFast 二维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个信号发生器控件和一个 DRPlotFast 二维曲线控件，然后通过直连绑定方式，用按钮控件来控制信号发生器控件的运行，用 DRPlotFast 二维曲线控件来显示从信号发生器接收到的信号波形。DRPlotFast 二维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入信号发生器控件
mG2=dsp.DRGenerator(0,44100,4096,0.6,100,0)
#==插入 PlotFast 控件
mP3=dr.DRPlotFast(win,95,122,700,210,xlow=0,xhigh=0.05)
#==控件直连绑定
mB1.addCallBackSingle(mG2.setVSOnOff)
mG2.addCallBack2D(mP3.setValue2D)
#==主程序循环
win.mainloop()
```

DRSemilogX 二维曲线控件用法

DRSemilogX 二维曲线控件的 X 坐标采用对数形式，也就是 log10(x)，主要用于显示频谱等二维曲线。当有曲线动态刷新速度要求时，建议用 DRPlot 控件，将 x 坐标类型设为对数，xtype=1，二者等价，但 DRPlot 控件做了动态显示优化，显示速度快一些。



DRSemilogX 二维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRSemilogX 二维曲线控件的命令和语法格式如下:

hl=DRSemilogX(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

控件创建时，当 xlow 和 xhigh 均设为零时，横坐标轴采用自动量程；当 ylow 和 yhigh 均设为零时，纵坐标采用自动量程；当 xarray 和 yarray 设为零，则初始曲线为空。另外，在自动量程时，DRSemilogX 二维曲线控件中的曲线放大、缩小、滚动等操作符无效。

DRSemilogX 二维曲线控件有 6 个属性函数，其功能定义如下：

hl.setXlim(xmin,xmax)：设定 X 轴显示范围

hl.setYlim(ymin,ymax)：设定 Y 轴显示范围

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

hl.setValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以覆盖形式重新绘制曲线

hl.addValue2D(xarray,yarray)：设定二维曲线数据，并以添加形式绘制新曲线。

hl.addValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以添加形式绘制新曲线

下面是一个全参数的 DRSemilogX 二维曲线控件创建实例：

```
hl=dr.DRSemilogX(win,34,98,700,210,'','#003355','#000000','#ffffff','#00eeff','#ffff00',1,10000,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRSemilogX(win,34,98,700,210,xlow=0.1,xhigh=1000)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRSemilogX 二维曲线控件为输入控件，可接收 2 种输入形式，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)和端口(setValue2DX)收到的是控件的曲线显示数据。



下面是 DRSemilogX 二维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个自定义函数和一个 DRSemilogX 二维曲线控件，然后通过直连绑定方式，用按钮控件来驱动自定义函数生成信号和信号频谱，然后用 DRSemilogX 二维曲线控件来显示信号频谱。DRSemilogX 二维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy.fft import rfft,irfft
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+100
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    N2=int(N/2)
    A=np.abs(np.fft.rfft(y))/(N2)
    df=Fs/N;
    mP.setValue2DX(df,A)

#==创建 Tkinter GUI 窗口
```

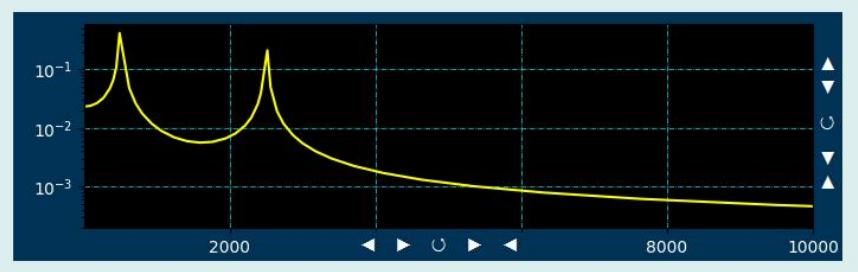
```
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 SemilogX 控件
mP=dr.DRSemilogX(win,95,122,700,210)
#==控件直连绑定
mB1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRSemilogY 二维曲线控件用法

DRSemilogY 二维曲线控件的 Y 坐标采用对数形式，也就是 log10(y)，主要用于显示对数谱等二维曲线。当有曲线动态刷新速度要求时，建议用 DRPlot 控件，将 y 坐标类型设为对数，ytype=1，二者等价，但 DRPlot 控件做了动态显示优化，显示速度快一些。



DRSemilogY 二维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRSemilogY 二维曲线控件的命令和语法格式如下:

hl=DRSemilogY(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

控件创建时，当 xlow 和 xhigh 均设为零时，横坐标轴采用自动量程；当 ylow 和 yhigh 均设为零时，纵坐标采用自动量程；当 xarray 和 yarray 设为零，则初始曲线为空。另外，在自动量程时，DRSemilogY 二维曲线控件中的曲线放大、缩小、滚动等操作符无效。

DRSemilogY 二维曲线控件有 6 个属性函数，其功能定义如下：

hl.setXlim(xmin,xmax)：设定 X 轴显示范围

hl.setYlim(ymin,ymax)：设定 Y 轴显示范围

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

hl.setValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以覆盖形式重新绘制曲线

hl.addValue2D(xarray,yarray)：设定二维曲线数据，并以添加形式绘制新曲线。

hl.addValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以添加形式绘制新曲线

下面是一个全参数的 DRSemilogY 二维曲线控件创建实例：

```
hl=dr.DRSemilogY(win,34,98,700,210,'','#003355','#000000','#ffffff','#00eeff','#ffff00',1,10000,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRSemilogY(win,34,98,700,210,xlow=0.1,xhigh=1000)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRSemilogY 二维曲线控件为输入控件，可接收 2 种输入形式，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)和端口(setValue2DX)收到的是控件的曲线显示数据。



下面是 DRSemilogY 二维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个自定义函数和一个 DRSemilogY 二维曲线控件，然后通过直连绑定方式，用按钮控件来驱动自定义函数生成信号和信号频谱，然后用 DRSemilogX 二维曲线控件来显示信号频谱。DRSemilogY 二维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy.fft import rfft,irfft
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+100
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    N2=int(N/2)
    A=np.abs(np.fft.rfft(y))/(N2)
    df=Fs/N;
    mP.setValue2DX(df,A)
```

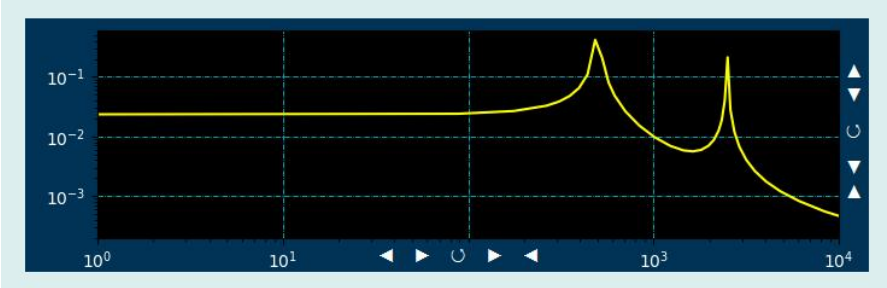
```
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 SemilogY 控件
mP=dr.DRSemilogY(win,95,122,700,210)
#==控件直连绑定
mB1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRLoglog 二维曲线控件用法

DRLoglog 二维曲线控件采用双对数坐标形式，也就是 log10(x)，log10(y)，主要用于以双对数形式显示频谱等二维曲线。当有曲线动态刷新速度要求时，建议用 DRPlot 控件，将 x 和 y 坐标类型均设为对数，xtype=1，ytype=1，二者等价，但 DRPlot 控件做了动态显示优化，显示速度快一些。



DRLoglog 二维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRLoglog 二维曲线控件的命令和语法格式如下：

hl=DRLoglog(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

控件创建时，当 xlow 和 xhigh 均设为零时，横坐标轴采用自动量程；当 ylow 和 yhigh 均设为零时，纵坐标采用自动量程；当 xarray 和 yarray 设为零，则初始曲线为空。另外，在自动量程时，DRLoglog 二维曲线控件中的曲线放大、缩小、滚动等操作符无效。

DRLoglog 二维曲线控件有 6 个属性函数，其功能定义如下：

hl.setXlim(xmin,xmax)：设定 X 轴显示范围

hl.setYlim(ymin,ymax)：设定 Y 轴显示范围

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

hl.setValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以覆盖形式重新绘制曲线

hl.addValue2D(xarray,yarray)：设定二维曲线数据，并以添加形式绘制新曲线。

hl.addValue2DX(dx,yarray)：设定等离散采样间隔二维曲线数据，并以添加形式绘制新曲线

下面是一个全参数的 DRLoglog 二维曲线控件创建实例：

```
hl=dr.DRLoglog(win,61,103,700,210,'','#003355','#000000','#ffffff','#00eeff','#ffff00',1,10000,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRLoglog(win,61,103,700,210)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRLogLog 二维曲线控件为输入控件，可接收 2 种输入形式，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)和端口(setValue2DX)收到的是控件的曲线显示数据。



下面是 DRLoglog 二维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个自定义函数和一个 DRLoglog 二维曲线控件，然后通过直连绑定方式，用按钮控件来驱动自定义函数生成信号和信号频谱，然后用 DRLoglog 二维曲线控件来显示信号频谱。DRLoglog 二维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy.fft import rfft,irfft
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+100
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    N2=int(N/2)
    A=np.abs(np.fft.rfft(y))/(N2)
    df=Fs/N;
    mP.setValue2DX(df,A)
```

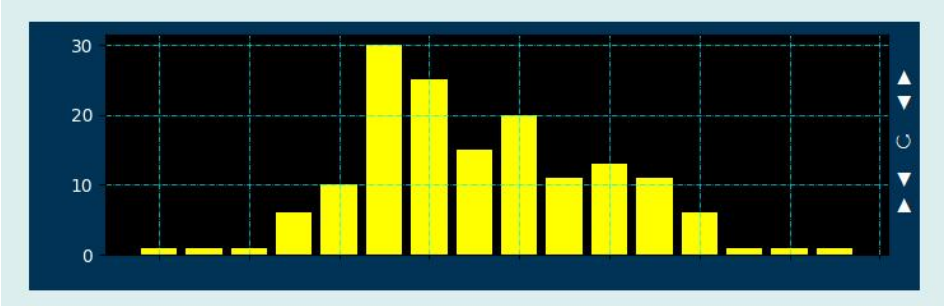
```
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 LogLog 控件
mP=dr.DRLoglog(win,95,122,700,210)
#==控件直连绑定
mB1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlotBar 直方图控件用法

DRPlotBar 直方图控件主要用于显示直方图曲线。



DRPlotBar 直方图控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlotBar 直方图控件的命令和语法格式如下:

hl=DRPlotBar(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

xarray：曲线横坐标数据数组，默认值=0

yarray：曲线纵坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

控件创建时，当 xlow 和 xhigh 均设为零时，横坐标轴采用自动量程；当 ylow 和 yhigh 均设为零时，纵坐标采用自动量程；当 xarray 和 yarray 设为零，则初始曲线为空。另外，在自动量程时，DRPlotBar 直方图控件中的曲线放大、缩小、滚动等操作符无效。

DRPlotBar 直方图控件有 3 个属性函数，其功能定义如下：

hl.setXlim(xmin,xmax)：设定 X 轴显示范围

hl.setYlim(ymin,ymax)：设定 Y 轴显示范围

hl.setValue2D(xarray,yarray)：设定二维曲线数据，并以覆盖形式重新绘制曲线

下面是一个全参数的 DRPlotBar 直方图控件创建实例：

```
hl=dr.DRPlotBar(win,70,105,700,210,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlotBar(win,70,105,700,210)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlotBar 直方图控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)收到的是控件的曲线显示数据。



下面是 DRPlotBar 直方图控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、和一个 DRPlotBar 直方图控件，然后通过直连绑定方式，用按钮控件来控制自定义函数生成仿真数据，然后用 DRPlotBar 直方图控件来显示。DRPlotBar 直方图控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle(v):
    x=[0,1,2,3,4,5,6,7,8]
    A=[0,2,4,6,8,4,3,2,1]
    mP.setValue2D(x,A)

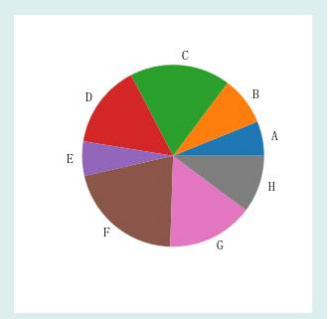
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 PlotBar 控件
mP=dr.DRPlotBar(win,70,105,700,210)
#==控件直连绑定
mB1.addCallBackSingle(muserSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlotPie 饼图控件用法

DRPlotPie 饼图控件主要用于主要用于显示饼图。



DRPlotPie 饼图控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlotPie 饼图控件的命令和语法格式如下:

hl=DRPlotPie(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

larray：标签，字符串数组，默认值=0

varray：饼值，浮点数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPlotPie 饼图控件有 1 个属性函数，其功能定义如下：

hl.setValue2DStringSingle(labelArray,pieArray)：设定饼图数据，并以覆盖形式重新绘制饼图

下面是一个全参数的 DRPlotPie 饼图控件创建实例：

```
hl=dr.DRPlotPie(win,152,63,300,300,'ABCD',['A','B','C','D','E','F'],[1,2,3,4,5,6])
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlotPie(win,70,105,700,210)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlotPie 控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)收到的是控件的曲线显示数据。



下面是 DRPlotPie 饼图控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、和一个 DRPlotPie 饼图控件，然后通过直连绑定方式，用按钮控件来控制自定义函数生成仿真数据，然后用 DRPlotPie 饼图控件来显示。DRPlotPie 饼图控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    sizes = [30, 40, 20, 10]
    labels = ['A', 'B', 'C', 'D']
    mPie.setValue2DStringSingle(labels,sizes)

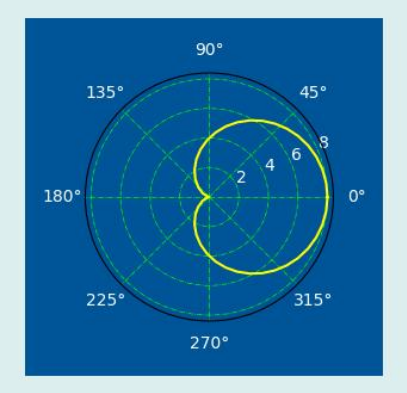
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRSwitch(win,50,42,80,40)
#==插入饼图控件
mPie=dr.DRPlotPie(win,152,63,300,300,'ABCD',['A','B','C','D','E','F'],[1,2,3,4,5,6])
#==直连方式绑定
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlotPolar 极坐标图控件用法

DRPlotPolar 极坐标图控件主要用于显示极坐标形式的二维曲线。



DRPlotPolar 极坐标图控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlotPolar 极坐标图控件的命令和语法格式如下:

hl=DRPlotPolar(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#005599'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00ff00'

cf3：前景色，默认值='#ffff00'

title：曲线标题，默认值=''

tarray：曲线角度坐标数据数组，默认值=0

rarray：曲线极径坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPlotPolar 极坐标图控件有 2 个属性函数，其功能定义如下：

hl.setValue2D(tarray,rarray)：设定极坐标图数据，并以覆盖形式重新绘制饼图

hl.addValue2D(tarray,rarray)：设定极坐标图数据，并以添加形式绘制新曲线。

下面是一个全参数的 DRPlotPolar 极坐标图控件创建实例：

```
hl=dr.DRPlotPolar(win,225,124,300,300,'#005599','#ffffff','#00ff00','#ffff00','',0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlotPolar(win,70,105,700,210)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlotPie 控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹

出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)收到的是控件的曲线显示数据。



下面是 DRPlotPolar 极坐标图控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、和一个 DRPlotPolar 极坐标图控件，然后通过直连绑定方式，用按钮控件来控制自定义函数生成仿真数据，然后用 DRPlotPolar 极坐标图控件来显示。DRPlotPolar 极坐标图控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import numpy as np

#用户自定义的回调函数
def muserSingle3(v):
    r = np.arange(0, 2, 0.01)
    theta = 2 * np.pi * r
    mPolar.setValue2D(theta,r)

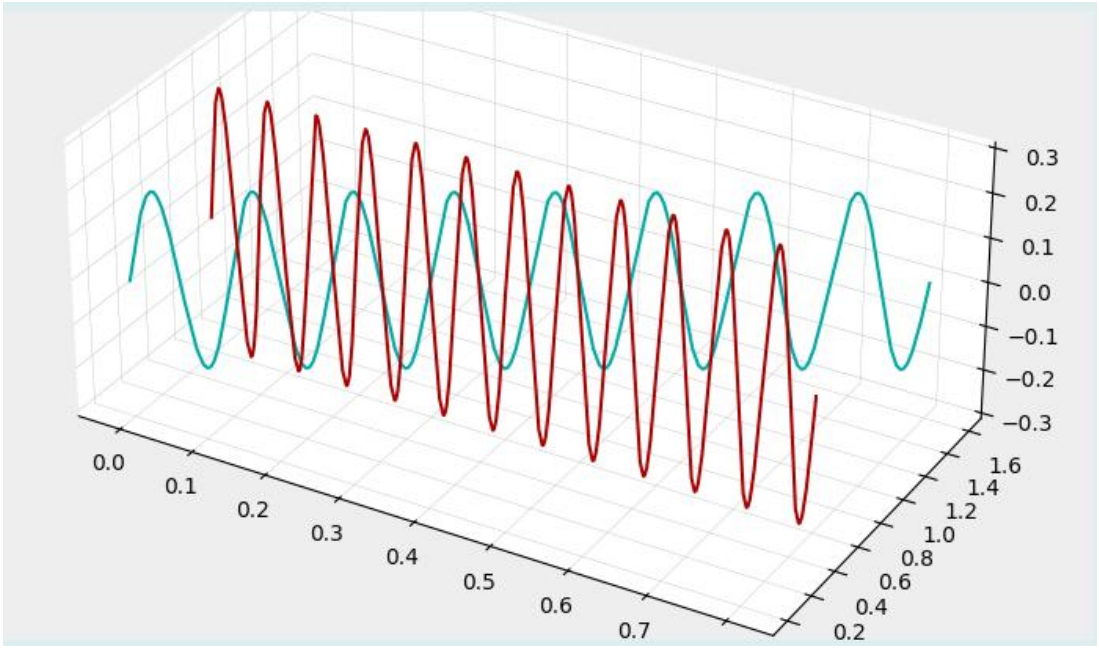
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRSwitch(win,50,42,80,40)
#==插入极坐标图控件
mPolar=dr.DRPlotPolar(win,225,124,300,300)
#==直连方式绑定
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlot3D 三维曲线控件用法

DRPlot3D 三维曲线控件主要用于在三维空间内显示信号曲线。



DRPlot3D 三维曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlot3D 三维曲线控件的命令和语法格式如下:

hl=DRPlot3D(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#ddeeee'

cb2：坐标平面背景色，默认值='#ffffff'

cf1：文字色，默认值='#000000'

cf2：网格线色，默认值='#bbbbbb'

xarray：曲线 x 坐标数据数组，默认值=0

yarray：曲线 y 坐标数据数组，默认值=0

zarray：曲线 z 坐标数据数组，默认值=0

xlow：X 轴最小值，默认值=0；

xhigh：X 轴最大值，默认值=0；若 xlow=xhigh，用自动量程

ylow：Y 轴最小值，默认值=0；

yhigh：Y 轴最大值，默认值=0；若 ylow=yhigh，用自动量程

zlow：Z 轴最小值，默认值=0；

zhigh：Z 轴最大值，默认值=0；若 zlow=zhigh，用自动量程

rate：x,y,z 轴绘图比例调整，默认值=1， 0/1=不调整/自动调整

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPlot3D 三维曲线控件有 2 个属性函数，其功能定义如下：

hl.setValue3D(xarray,yarray,zarray,c='#009999',s=0)：设定三维曲线数据，并以覆盖形式重新绘制饼图，s=0/1 对应 plot/stem。

hl.addValue3D(xarray,yarray,zarray,c='#aa0000',s=0)：设定三维曲线数据，并以添加形式绘制新曲线，s=0/1 对应 plot/stem

下面是一个全参数的 DRPlot3D 三维曲线控件创建实例：

```
hl=dr.DRPlot3D(win,213,40,420,380,'',0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlot3D(win,213,40,420,380)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlot3D 三维曲线控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue3D)收到的是控件的曲线显示数据。



下面是 DRPlot3D 三维曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个用户自定义函数、一个 DRPlot3D 三维曲线控件，然后调用用户自定义函数生成数据和显示。DRPlot3D 三维曲线控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import numpy as np

#用户自定义的回调函数==========
def muserSingle3(v):
    Fs=5120
    dt=1.0/Fs
    N=4096
    t=np.arange(N)*dt
    z1=0.2*np.sin(2*np.pi*10*t)
    z2=0.3*np.sin(2*np.pi*15*t)
    y=np.ones(N)*0.6
    mP3D.setValue3D(t,2*t,z1,'#00aaaa')
    mP3D.addValue3D(t,y,z2,'#aa0000')

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入控件
```

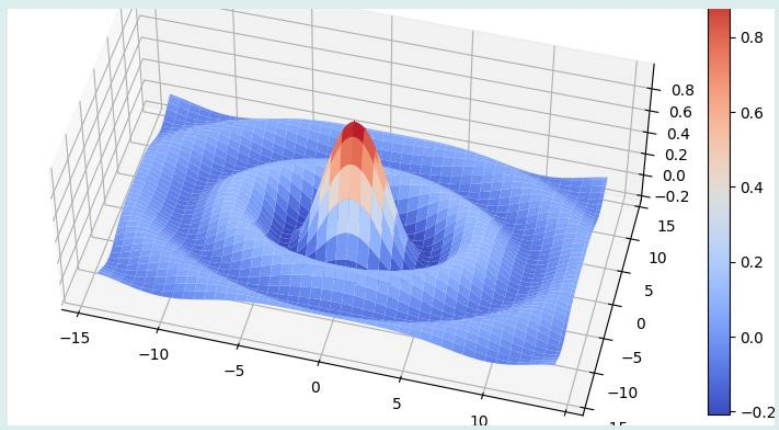
```
mP3D=dr.DRPlot3D(win,20,40,700,400)
#==直连方式绑定
muserSingle3(0)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlotSurface 三维曲面控件用法

DRPlotSurface 三维曲面控件主要用于在三维空间内显示三维谱阵等三维曲面形式的图形。



DRPlotSurface 三维曲面控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlotSurface 三维曲面控件的命令和语法格式如下:

hl=DRPlotSurface(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

xarray：曲线 x 坐标数据数组，默认值=0

yarray：曲线 y 坐标数据数组，默认值=0

zarray：曲线 z 坐标数据数组，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPlotSurface 三维曲面控件有 2 个属性函数，其功能定义如下：

hl.setValue3D(xarray,yarray,zarray)：设定三维曲面数据，并以覆盖形式重新绘制饼图

hl.addValue3D(xarray,yarray,zarray)：设定三维曲面数据，并以添加形式绘制新曲线。

下面是一个全参数的 DRPlotSurface 三维曲面控件创建实例：

```
hl=dr.DRPlotSurface(win,140,79,500,380,'',0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlotSurface(win,213,40,420,380)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlotSurface 三维曲面控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue3D)收到的是控件的曲线显示数据。



下面是 DRPlotSurface 三维曲面控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、一个 DRPlotSurface 三维曲面控件，然后通过直连绑定方式，用按钮控件来刷新三维曲面控件的显示。DRPlotSurface 三维曲面控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import numpy as np

#用户自定义的回调函数
def muserSingle3(v):
    x=np.linspace(-14,14,128)
    y=np.linspace(-14,14,128)
    [xSurf,ySurf]=np.meshgrid(x,y)
    r=np.sqrt(xSurf**2+ySurf**2)
    zSurf=np.sin(r).T/r
    mP3D.setValue3D(xSurf,ySurf,zSurf)

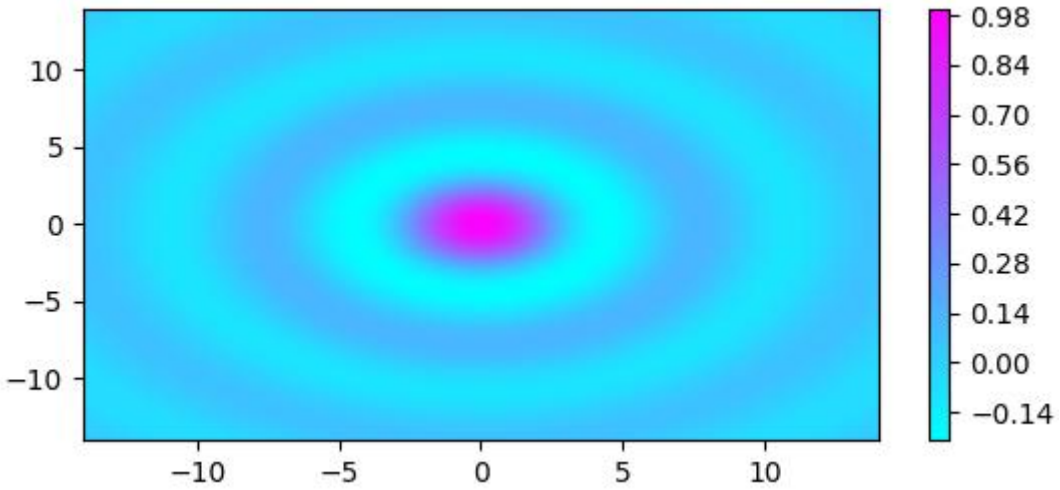
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRSwitch(win,50,42,80,40)
#==插入 3D 曲面控件
mP3D=dr.DRPlotSurface(win,140,79,500,380)
#==直连方式绑定
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRPlotContour 等高线图控件用法

DRPlotContour 等高线图控件主要用于以等高线图形式来显示三维曲线。



DRPlotContour 等高线图控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlotContour 等高线图控件的命令和语法格式如下:

hl=DRPlotContour(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

title：曲线标题，默认值=''

xarray：曲线 x 坐标数据数组，默认值=0

yarray：曲线 y 坐标数据数组，默认值=0

zarray：曲线 z 坐标数据数组，默认值=0

scale：色彩图级数，默认值=64

cmap：色彩图名称，默认值='cool'，见 Matplotlib 库 ColorMap 定义

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPlotContour 等高线图控件有 1 个属性函数，其功能定义如下：

hl.setValue3D(xarray,yarray,zarray)：设定等高线图数据，并以覆盖形式重新绘制

下面是一个全参数的 DRPlotContour 等高线图控件创建实例：

```
hl=dr.DRPlotContour(win,200,20,640,480,'',0,0,0,64,'cool')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRPlotContour(win,200,20,640,480)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPlotSurface 三维曲面控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点

击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue3D)收到的是控件的曲线显示数据。



下面是 DRPlotContour 等高线图控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、一个 DRPlotContour 等高线图控件，然后通过直连绑定方式，用按钮控件来刷新等高线图控件的显示。DRPlotContour 等高线图控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import numpy as np

#用户自定义的回调函数
def muserSingle3(v):
    x=np.linspace(-14,14,128)
    y=np.linspace(-14,14,128)
    [xSurf,ySurf]=np.meshgrid(x,y)
    r=np.sqrt(xSurf**2+ySurf**2)
    zSurf=np.sin(r).T/r
    mP3D.setValue3D(x,y,zSurf)

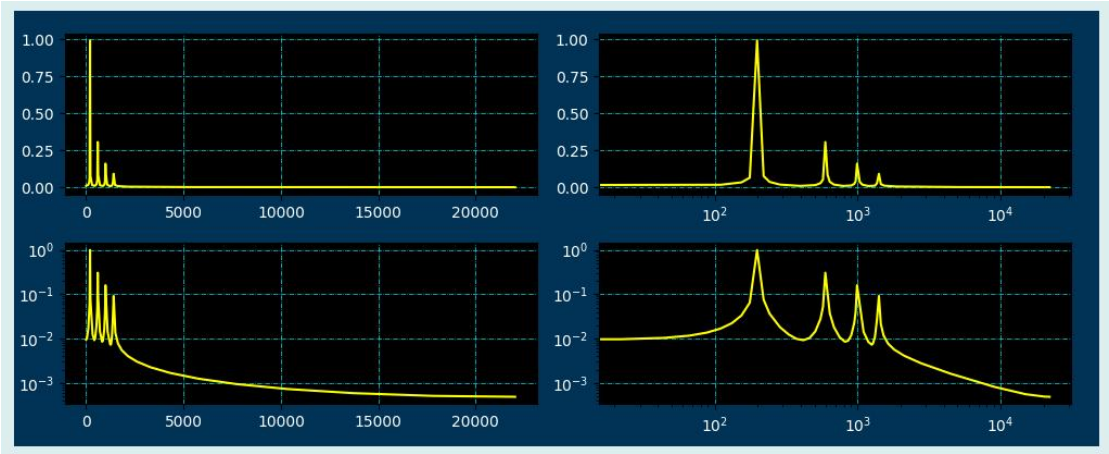
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRSwitch(win,50,42,80,40)
#==插入等高线图控件
mP3D=dr.DRPlotContour(win,200,20,640,480,'',0,0,0,64,'cool')
#==直连方式绑定
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRSubPlot 曲线控件用法

DRSubPlot 曲线控件主要用于显示多条信号波形、频谱等二维曲线。因做了显示优化，与几个独立的 Plot 控件相比，在显示不断动态刷新的多条曲线时，效果要好一些；但因曲线安排紧凑，无法安放幅值调整等控件。



DRSubPlot 曲线控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRSubPlot 曲线控件的命令和语法格式如下:

hl=DRSubPlot(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#003355'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

cf3：前景色，默认值='#ffff00'

N：曲线图的行数，默认值=2

M：曲线图的列数，默认值=2

Style：各曲线图的曲线格式，默认值=[0,0,0,0]；其中 0=plot，1=stem，2=semix，3=semiy,4=loglog

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRSubPlot 曲线控件有 7 个属性函数，其功能定义如下：

hl.setXlim(k,xlow,xhigh)：设定子曲线图 k 的 X 坐标显示范围，k 为图号，取值 1 到 8

hl.setYlim(k,ylow,yhigh)：设定子曲线图 k 的 Y 坐标显示范围，k 为图号，取值 1 到 8

hl.setValue2DX(k,xarray,yarray)：设定子曲线图 k 的数据，并绘制曲线图。k 为图号，取值 1 到 8

hl.setValue2D1(xarray,yarray)：设定子曲线图 1 的数据；

hl.setValue2D2(xarray,yarray)：设定子曲线图 2 的数据；

hl.setValue2D3(xarray,yarray)：设定子曲线图 3 的数据；

hl.setValue2D4(xarray,yarray)：设定子曲线图 4 的数据；

下面是一个全参数的 DRSubPlot 曲线控件创建实例：

```
hl=dr.DRSubPlot(win,61,112,700,400,'#003355','#000000','#ffffff','#00eeff','#ffff00',2,1,[0,0])
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRSubPlot(win,62,80,1000,400,N=2,M=2,style=[0,2,3,4])
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRSubPlot 曲线控件为输入控件。在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。4 个输入项(setValue2D1、setValue2D2、setValue2D3、setValue2D4)分别对应于控件中显示的 4 条曲线。



下面是 DRSubPlot 曲线控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、一个 DRSubPlot 曲线控件，然后通过直连绑定方式，用按钮控件来调用用户自定义函数，生成测试信号和信号的幅值谱，然后在 subplot 上用 plot、semix、semiy 和 loglog 四种不同形式来显示信号的幅值谱。DRSubPlot 曲线控件应用示例小程序代码如下：

```
#==引用的包
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import numpy as np
from scipy.fftpack import fft,ifft

#用户自定义的回调函数
def muserSingle3(v):
    #==生成多频测试信号
    Fs=44100;
    N=2000;
    dt=1.0/Fs
    t=np.arange(N)*dt
    x1=np.sin(2*np.pi*200*t)
    x1=x1+0.33*np.sin(2*np.pi*600*t)
    x1=x1+0.2*np.sin(2*np.pi*1000*t)
    x1=x1+0.14*np.sin(2*np.pi*1400*t)
    #==计算信号的幅值谱
    y2=np.abs(fft(x1))
```

```
    f=np.arange(int(N/2))*Fs/N;
    a2=2*y2[:int(N/2)]/N
    #==用 Subplot 显示幅值谱
    mSub.setValue2DX(1,f,a2)
    mSub.setValue2DX(2,f,a2)
    mSub.setValue2DX(3,f,a2)
    mSub.setValue2DX(4,f,a2)

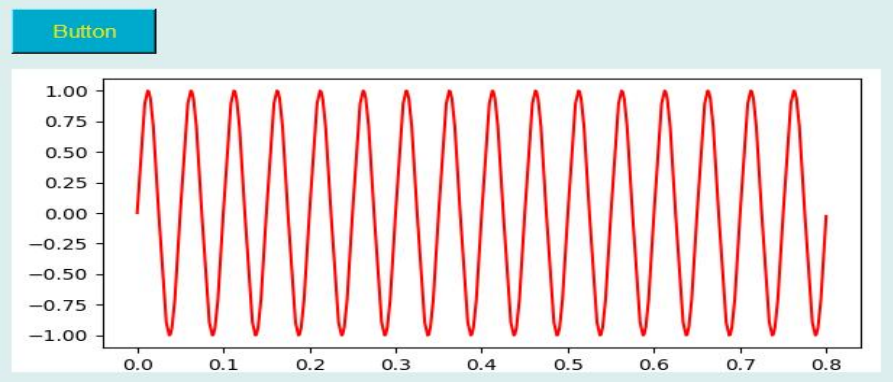
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('1000x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRSwitch(win,50,22,80,40)
#==插入 Subplot,2x2， plot,semix,semiy,loglog
mSub=dr.DRSubPlot(win,62,80,1000,400,N=2,M=2,style=[0,2,3,4])
#==直连方式绑定
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRFigure 控件用法

DRFigure 控件主要用于在 GUI 界面上提供一个显示容器，用于显示一用户用 Matplotlib 绘图语句绘制的曲线。利用该控件，用户可以按习惯的 Matplotlib 方式绘制曲线，然后导入到 DRFigure 容器控件进行显示。



DRFigure 控件效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRFigure 控件的命令和语法格式如下:

hl=DRFigure(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：容器背景色，默认值=None，None=不做更改

cb1：曲线取背景色，默认值=None

cf：坐标文字色，默认值=None

cf1：边框和网格线色，默认值=None

cf2：曲线颜色，默认值=None

tight：容器布局，默认值=False；True/False=紧缩/非紧缩

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRFigure 控件有 1 个属性函数，其功能定义如下：

hl.setValueFig(fig)：设定 Matplotlib 库 Figure 格式的图形对象，用于在容器中现数

下面是一个全参数的 DRFigure 控件创建实例：

```
hl=dr.DRFigure(win,21,78,700,200,None,None,None,None,None,True)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRFigure(win,21,78,700,200)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

下面是 DRFigure 控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个滑块控件，一个自定义函数和一个 DRFigure 控件，然后通过直连绑定方式，用滑块控件驱动自定义函数产生不同频率的正弦

波和用 Matplotlib 绘图，最后用 DRFigure 控件在 GUI 界面上进行显示。DRFigure 控件的应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy.fft import rfft,irfft
import matplotlib.pyplot as plt
import drvi.drviControlls as dr
#=====================
def muserSingle(v):
    global F
    F=F+100
    Fs=5120
    N=512
    t=np.arange(N)*(1/Fs)
    y=np.sin(2*np.pi*F*t)
    N2=int(N/2)
    A=np.abs(np.fft.rfft(y))/(N2)
    f=np.arange(N2+1)*(Fs/N);
    plt.close('all')
    fig,ax=plt.subplots(2,1)
    ax[0].plot(t,y)
    ax[1].plot(f,A)
    mFigure.setValueFig(fig)

#==Main Window=====================
win= tk.Tk()
win.geometry('1204x716')
win.config(bg="#ddeeee")
#==Layout of controls=================
mB1=dr.DRButton(win,20,20,100,30,'#00aacc','#eeee00','Button',12,101)
mFigure=dr.DRFigure(win,21,78,700,360,cb='#003355',cb1='#000000',cf='#ffffff',cf1='#009999',cf2='#ffff00',tight=
False)
#=User defined data probe============
mB1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==Main Loop====================
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRImage 图象控件用法

DRImage 图象显示控件主要用于在 GUI 界面上显示一幅图片。



DRImage 图象显示控件效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRImage 图象显示控件的命令和语法格式如下:

hl=DRImage(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#dddddd'

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRImage 图象显示有 4 个属性函数，其功能定义如下：

hl.setValueString(name)：设定显示图象的文件名和路径

hl.setValueImage(im)：设定 pillow 格式(Image)的图象对象

hl.setValueImArray(data)：设定 RGB 格式的三维数组图象数据(np.unit8)。

hl.getValueImage()：返回 pillow 格式(Image)的图象对象

下面是一个全参数的 DRImage 图象显示控件创建实例：

```
hl=dr.DRImage(win,194,31,420,260,'#dddddd')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRImage(win,194,31,420,260)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

下面是 DRImage 图象显示控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个文件选择按钮控件，一个 DRImage 图象显示控件，然后通过直连绑定方式，用文件选择按钮控件来选择图象文件，用 DRImage 图象显示控件来进行图象显示。DRImage 图象显示控件的应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserString(txt):
    print(txt)
```

```
    mImg.setValueString(txt)


#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入文件选择窗按钮
mButton1=dr.DRFileBut(win,50,42,100,40,txt='File')
#==插入图象显示控件
mImg=dr.DRImage(win,194,31,420,260)
#==直连绑定
mButton1.addCallBackString(muserString)
#==主程序循环
win.mainloop()
```

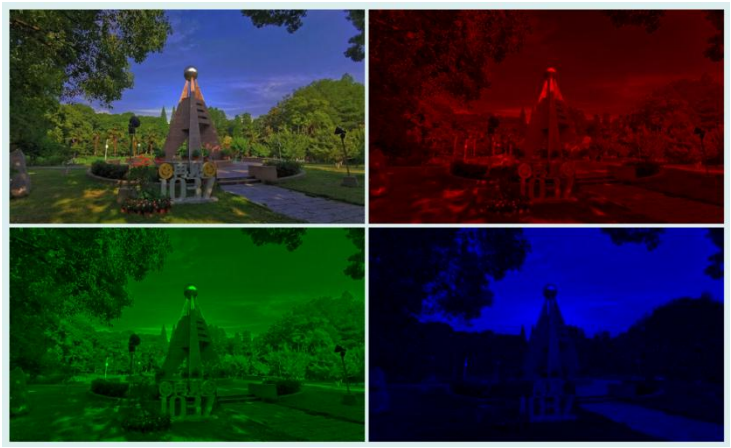
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRImage 继承自 QLabel，因此除了用 DRImage 的参数化外观定义外，还可以直接用 QLabel 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
    setStyleSheet(styleString)
```

DRSubImages 图象控件用法

DRSubImages 曲线控件主要用于集中显示多幅图片。



DRSubImages 显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRSubImages 控件的命令和语法格式如下:

hl=DRSubImages(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#dddddd'

N：曲线图的行数，默认值=2

M：曲线图的列数，默认值=2

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRSubImages 控件有 7 个属性函数，其功能定义如下：

hl.setValueImageK(k,im)：设定第 k 幅图象的 pillow 格式(Image)的图象对象

hl.setValueImgArrayK(k,data)：设定第 k 幅图象的 RGB 格式的三维数组图象数据(np.unit8)

hl.setValueImage(im)：设定第 0 幅图象的 pillow 格式(Image)的图象对象

hl.setValueImage1(im)：设定第 1 幅图象的 pillow 格式(Image)的图象对象

hl.setValueImage2(im)：设定第 2 幅图象的 pillow 格式(Image)的图象对象

hl.setValueImage3(im)：设定第 3 幅图象的 pillow 格式(Image)的图象对象

hl.getValueImage(k)：返回第 k 幅图象的 pillow 格式(Image)的图象对象

下面是一个全参数的 DRSubImages 控件创建实例：

```
hl=dr.DRSubImages(win,146,119,600,400,'#dddddd',2,2)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRSubPlot(win,62,80,1000,400,N=2,M=3)
```

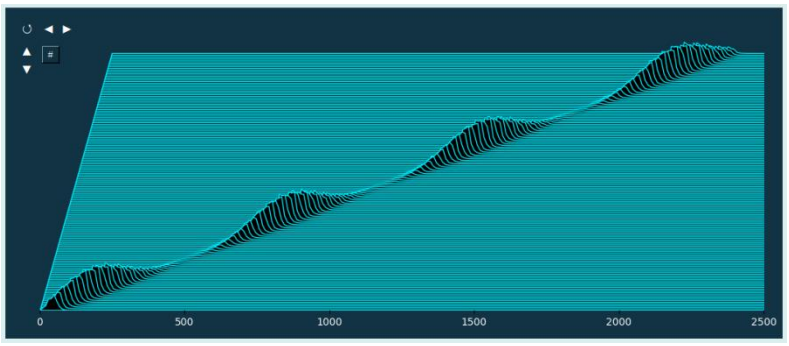
创建后可以通过返回的控件句柄对其属性函数进行操作。

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRwaterFall 谱阵显示控件用法

DRwaterFall 谱阵显示控件主要用于显示时变信号的谱阵图。一般来说，幅值谱阵的显示效果较好；另外，谱阵的阶数通常控制在 100 阶以下，过高显示效果也不好。在阵图显示方面，其作用与 DRPlotSurface 三维曲面控件相似，但因采用二维图象模式，显示效率更高；另外 DRPlotSurface 所依托的 matplotlib 库中 plot_surface 函数对谱阵形成的伪三维曲面的支持不好，显示效果欠佳。



DRwaterFall 谱阵显示控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRwaterFall 谱阵显示控件的命令和语法格式如下：

hl= DRwaterFall(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#113344'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

xDist：阵图横移系数，默认值=0.1

yDist：阵图顶边距系数，默认值=0.1

zDist：阵图幅值比例系数，默认值=0.1

flow：横坐标最小值，默认值=0

fhigh：横坐标最大值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。控件创建时，当 flow 和 fhigh 均设为零时，横坐标轴采用自动量程。

DRwaterFall 谱阵显示控件有 2 个属性函数，其功能定义如下：

hl.setValue3D(t,f,a2D)：设定谱阵曲线数据，t 和 f 为一维数组，a2D 为二维数组，对应 f,t,a2D=signal.spectrogram(...)

hl.addFrame(M,f,A,d)：添加一帧谱阵曲线，M 为谱阵曲线的总阶数，f 推入频谱的频率点数据，A 为频谱数据，对应：f,A=dsp.AmplitudeSpetrum(...)；d 为推入方向，取 0/1。

下面是一个全参数的 DRwaterFall 谱阵显示控件创建实例：

```
hl=dr. DRwaterFall(win,54,104,1000,400,'#113344','#000000','#ffffff','#00eeff',0.1,0.1,0.1,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr. DRwaterFall(win,54,104,1000,400,xDist=0.05)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRwaterFall 谱阵显示控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue3D)收到的是控件的曲线显示数据。



下面是 DRwaterFall 谱阵显示控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件、一个 DRwaterFall 谱阵显示控件和一个用户自定义函数，然后通过直连绑定方式，用按钮控件来驱动用户自定义回调函数产生一段 chirp 信号，然后计算 STFT 谱阵，最后用 DRwaterFall 谱阵显示控件来显示。 DRwaterFall 谱阵显示控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy import signal
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    Fs=5120;
    N=50000;
    t=np.arange(N)*(1.0/Fs)
    y=np.sin(2*np.pi*(100+5*t*t)*t)
    nft=1024; ovl=int(nft/2)
    ff,tt,Sxx=signal.spectrogram(y,Fs,nperseg=nft,noverlap=ovl)
    mArray.setValue3D(tt,ff,Sxx)

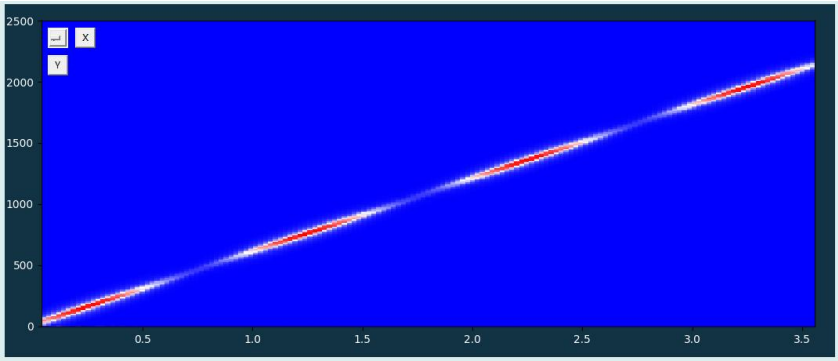
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,20,20,100,30,txt='Run')
#==插入谱阵显示控件
mArray=dr. DRwaterFall(win,20,60,800,400)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRColormesh 语谱图控件用法

DRColormesh 语谱图显示控件主要用于显示时变信号的语谱图。一般来说，对数功率谱阵的显示效果较好。其作用与 DRPlotContour 伪彩色图控件相似，但依托的是 matplotlib 库中的 pcolormesh 伪彩色图函数，而非 plotcontour 三维曲面图函数，效率更高一些；另外直接采用 STFT 变换结果数据格式，也更方便。该控件主要用于教学目的，专业的语谱图分析和显示建议用 Librosa 库的 ibrosa.display.specshow+DRFigure 函数。



DRColormesh 语谱图显示控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRColormesh 语谱图显示控件的命令和语法格式如下：

hl= DRColormesh(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#113344'

cb2：背景色 1，默认值='#000000'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

ylog：Y 轴显示模式，默认值 0，0/1=线性、对数

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRColormesh 语谱图显示控件有 4 个属性函数，其功能定义如下：

hl.setValue3D(t,f,a2D)：设定语谱图曲线数据，t 和 f 为一维数组，a2D 为二维数组，对应 f,t,a2D=signal.spectrogram(...) ；但通常将 spectrogram 函数返回的功率谱阵数据转化为对数功率谱阵后再进行显示，即 20*np.log10(a2D)。

hl.setXlim(x1,x2)：设定 X 轴(通常代表时间)的显示量程范围。

hl.setYlim(y1,y2)：设定 Y 轴(通常代表频率)的显示量程范围。

hl.setCmap(name)：设定伪彩色图谱名称，参略 matplotlib 的 cmap 参数介绍，常用图谱名称有：bwr、rainbow、cool、viridis、Blues、Reds、Greys。

下面是一个全参数的 DRColormesh 语谱图显示控件创建实例：

```
hl=dr.DRColormesh(win,50,92,1000,400,'#113344','#000000','#ffffff','#00eeff')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRColormesh(win,50,92,1000,400)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRColormesh 语谱图控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue3D)收到的是控件的曲线显示数据。



下面是 DRColormesh 语谱图显示控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件、一个 DRColormesh 语谱图显示控件和一个用户自定义函数，然后通过直连绑定方式，用按钮控件来驱动用户自定义回调函数产生一段 chirp 信号，然后计算 STFT 谱阵，最后用 DRColormesh 语谱图显示控件来显示。 DRColormesh 语谱图显示控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy import signal
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    Fs=5120;
    N=50000;
    t=np.arange(N)*(1.0/Fs)
    y=np.sin(2*np.pi*(100+5*t*t)*t)
    nft=1024; ovl=int(nft/2)
    ff,tt,Sxx=signal.spectrogram(y,Fs,nperseg=nft,noverlap=ovl)
    mArray.setValue3D(tt,ff,20*np.log10(Sxx))

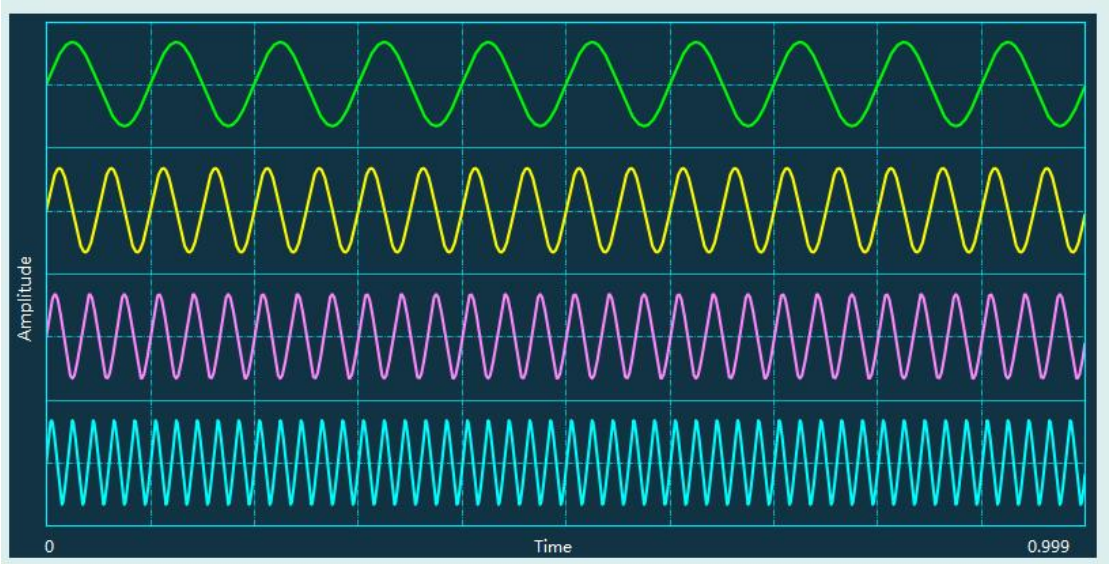
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x497')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,20,20,100,30,txt='Run')
#==插入语谱图显示控件
mArray=dr.DRColormesh(win,20,60,800,400)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRMonitor 多曲线显示控件用法

DRMonitor 多曲线显示控件主要用于显示多条信号波形曲线和信号的小波多尺度分解曲线等。因曲线显示在同一个示波控件上，画面利用率更高，约束是曲线的类型必须一致。



DRMonitor 多曲线显示控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRMonitor 多曲线显示控件的命令和语法格式如下:

hl=DRMonitor(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#113344'

cf1：前景色，默认值='#ffffff'

cf2：前景色，默认值='#00eeff'

xlable：X 坐标轴标题，默认值='Time'

ylable：Y 坐标轴标题，默认值='Amplitude'

T：X 坐标轴显示窗口宽度，默认值=0，0=取信号窗口宽度

wn：曲线区数量或曲线条数，默认值=4

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRMonitor 多曲线显示控件有 7 个属性函数，其功能定义如下：

hl.setValue2DM(M,tarray,yarray,c='#ffff00',x0=0,ylow=0,yhigh=0,cl=0)：设定子曲线图 M 的曲线数据和绘图参数。cl=0/1，代表覆盖和添加曲线绘制模式，默认为覆盖模式。

hl.setValue2DXM(M,dt,yarray,c='#ffff00',x0=0,ylow=0,yhigh=0,cl=0)：设定子曲线图 M 的曲线数据和绘图参数。

hl.setValue2D1(xarray,yarray)：设定子曲线图 1 的数据，用于快速直连；

hl.setValue2D2(xarray,yarray)：设定子曲线图 2 的数据，用于快速直连；

hl.setValue2D3(xarray,yarray)：设定子曲线图 3 的数据，用于快速直连；

hl.setValue2D4(xarray,yarray)：设定子曲线图 4 的数据，用于快速直连；

hl.setValue2DX1(dt,yarray)：设定子曲线图 1 的数据，用于快速直连；

hl.setValue2DX2(dt,yarray)：设定子曲线图 2 的数据，用于快速直连；

hl.setValue2DX3(dt,yarray)：设定子曲线图 3 的数据，用于快速直连；

hl.setValue2DX4(dt,yarray)：设定子曲线图 4 的数据，用于快速直连；

hl.setXWidth(T)：用于更改 X 坐标的显示窗口宽度

hl.clearFig(wn=None)：清空整个曲线绘图区。通常当使用添加模式绘制曲线后，下一次开始重新绘制整个图形时，可使用该函数清空画面；若未使用添加模式，则新曲线会自动覆盖旧曲线，此时不用该函数。wn 为曲线区数量或曲线条数，默认为继承控件创建时的曲线数。

下面是一个全参数的 DRMonitor 多曲线显示控件创建实例：

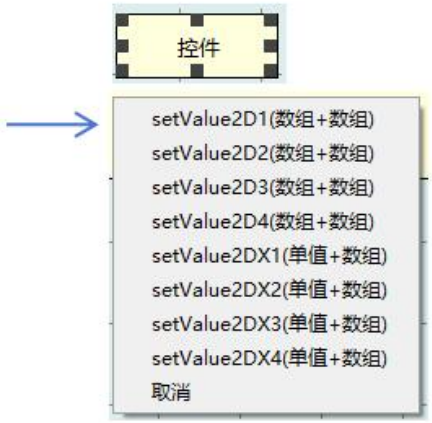
```
hl=dr.DRMonitor(win,136,62,700,300,'#113344','#ffffff','#00eeff','Time','Amplitude',0,4)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRMonitor(win,136,62,700,300,wn=2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRMonitor 多曲线显示控件为输入控件。在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。4 个输入项(setValue2D1、setValue2D2、setValue2D3、setValue2D4)和另 4 个输入项(setValue2DX1、setValue2DX2、setValue2DX3、setValue2DX4)分别对应于控件中显示的 4 条曲线。



下面是 DRMonitor 多曲线显示控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数、一个 DRMonitor 多曲线显示控件，然后通过直连绑定方式，用按钮控件来调

用用户自定义函数，生成测试信号，然后在 DRMonitor 显示。DRMonitor 多曲线显示控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle(v):
    global ff;
    Fs=1000
    N=1000
    ff=ff+10
    t=np.arange(N)*(1/Fs)
    y1=np.sin(2*np.pi*ff*t)
    y2=np.sin(2*np.pi*2*ff*t)
    y3=np.sin(2*np.pi*3*ff*t)
    y4=np.sin(2*np.pi*5*ff*t)
    mPlot.setValue2DM(0,t,y1,'#00ff00',0,-1.5,1.5)
    mPlot.setValue2DM(1,t,y2,'#ffff00',0,-1.5,1.5)
    mPlot.setValue2DM(2,t,y3,'#ff88ff',0,-1.5,1.5)
    mPlot.setValue2DM(3,t,y4,'#00ffff',0,-1.5,1.5)

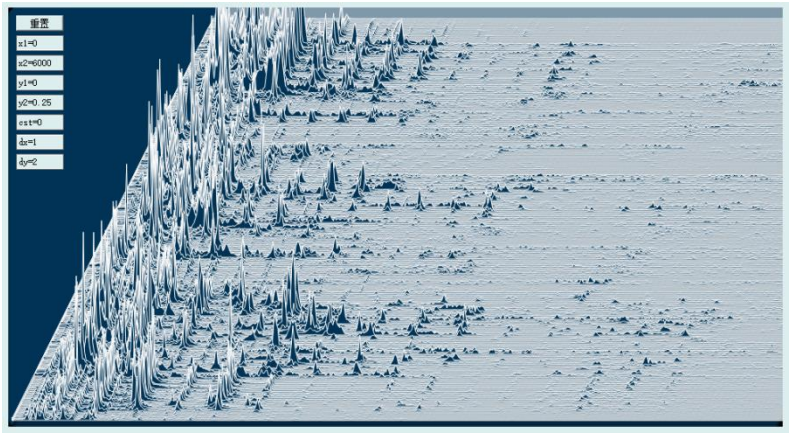
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x497')
win.config(bg="#ddeeee")
ff=10
#==插入控件
mButton1=dr.DRButton(win,20,20,100,30,txt='Run')
mPlot=dr.DRMonitor(win,20,80,800,400,wn=4)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle)
#==global var===
ff=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRArrayX 实时谱阵显示控件用法

DRArrayX 实时谱阵显示控件主要用于显示随信号实时变化的谱阵图。因需要耗费较多的显示资源，该控件只支持 pyqt 版本。



DRArrayX 实时谱阵显示控件显示效果图

在 Python 程序 pyqt GUI 界面中创建一个 DRArrayX 实时谱阵显示控件的命令和语法格式如下:

hl= DRArrayX(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 pyqt 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#003355'

cb：背景色，默认值='#ffffff'

cst：渐变色开关，默认值=0；0/1(单色/渐变色)

dx：谱图间 x 方向平移点数，默认值=1

dy：谱图间 y 方向平移点数，默认值=3

xlow：X 轴(频率轴)最小值，默认值=0Hz

xhigh：X 轴(频率轴)最大值，默认值=10000Hz

ylow：Y 轴(幅值轴)最小值，默认值=0

yhigh：Y 轴(幅值轴)最大值，默认值=0.5

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。控件创建时，当 flow 和 fhigh 均设为零时，横坐标轴采用自动量程。

DRArrayX 实时谱阵显示控件有 1 个属性函数，其功能定义如下：

hl.setValue2D(t,Amp)：添加当前频谱到谱阵中，t 和 Amp 为一维数组，对应频谱曲线的频率和幅值数据。

下面是一个 DRArrayX 实时谱阵显示控件创建实例：

```
hl=dr.DRArrayX(self.win,20,240,1070,430,cst=0,dx=1,dy=4,xlow=0,xhigh=1500,ylow=0,yhigh=1)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操

作。DRArrayX 实时谱阵显示控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)收到的是频谱图数据。



下面是 DRArrayX 实时谱阵显示控件的创建和应用示例小程序。功能是用一个信号发生器连续生成信号，然后显示和计算信号的波形与幅值谱，并将当前幅值谱添加到 DRArrayX 实时谱阵显示控件中，实现谱阵的实时显示。示例小程序代码如下：

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import *
from PyQt5.QtCore import *
import numpy as np
import drvi.drviDSP as dsp
import drvi.drviQTControlls as dr

#===Start of MainClass=====================
class drviMain(QMainWindow):
    def __init__(self):
        super(drviMain,self).__init__()
        self.initGUI()

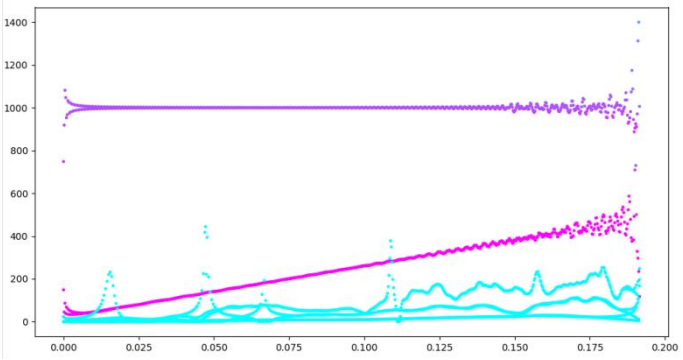
    #===Init GUI================================
    def initGUI(self):
        self.setWindowTitle("APP Frame by DRPython")
        self.resize(1100,720)
        self.win=QWidget()
        self.win.setStyleSheet("background-color : #ddeeee; ")
        self.setCentralWidget(self.win)
        #=Add controls
        self.mButton1=dr.DRButton(self.win,20,20,100,40,txt='运行/停止')
        self.mHSlider2=dr.DRHSlider(self.win,150,20,900,30,'#444444','#880000','#008822',8,20,0,1000,20)
        self.mGenerator3=dsp.DRGenerator(0,5000,4096,0.6,100,0,250)
        self.mPlot4=dr.DRPlotFast(self.win,20,70,530,160,xlow=0,xhigh=0.05)
        self.mPlot5=dr.DRPlotFast(self.win,560,70,530,160)
        self.mSpectrum6=dsp.DRSpectrum()
        self.mPlot7=dr.DRArrayX(self.win,20,240,1070,430,cst=0,dx=1,dy=4,xlow=0,xhigh=1500,ylow=0,yhigh=1)
        self.mButton1.addCallBackSingle(self.mGenerator3.setVSOnOff)
        self.mHSlider2.addCallBackSingle(self.mGenerator3.setVSFre)
        self.mGenerator3.addCallBack2D(self.mPlot4.setValue2D)
        self.mGenerator3.addCallBack2DX(self.mSpectrum6.setValue2DX)
        self.mSpectrum6.addCallBack2DAmp(self.mPlot5.setValue2D)
        self.mSpectrum6.addCallBack2DAmp(self.mPlot7.setValue2D)

#==Main Loop=====================
app=QApplication(sys.argv)
app.setStyle("Fusion")
win=drviMain()
win.show()
sys.exit(app.exec())
```

可参考实验“003_QT 媒体文件信号谱阵分析.py”和“005_QT 麦克风谱阵分析.py”。

DRScatter 散点图控件用法

DRScatter 散点图控件主要用于显示二维数组定义的散点图；也可以用于用于显示三维数组定义的散点图，此时 z 坐标对应的幅值变化映射为不同的色彩。



DRScatter 散点图控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRScatter 散点图控件的命令和语法格式如下：

hl=DRScatter(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb1：背景色，默认值='#ffffff'

cf1：前景色，默认值='#000000'，座标色

cf2：前景色，默认值='#00eeff'，曲线色

cmap：色彩映射集，默认值='cool'，参略 matplotlib 库 cmap 参数

dot：散点直径，默认值=3；

xlow：横坐标最小值，默认值=0

xhigh：横坐标最大值，默认值=0

ylow：纵坐标最小值，默认值=0

yhigh：纵坐标最大值，默认值=0

zlow：色彩坐标最小值，默认值=0

zhigh：色彩坐标最大值，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。控件创建时，当相应轴 low 和 xhigh 均设为零时，采用自动量程。

DRScatter 散点图控件有 4 个属性函数，其功能定义如下：

hl.setValue2D(xarray,yarray,c=None)：设定二维散点数据，并以覆盖形式重新绘制，c 为 16 进制色彩码，如'#ff0000'

hl.addValue2D(xarray,yarray,c=None)：设定二维散点数据，并以添加形式绘制，c 为 16 进制色彩码，如'#ff0000'

hl.setValue3DC(xarray,yarray,carray)：用三个数组设定散点图数据，第三个数组映射为色彩变换，以覆盖形式重新绘制。

hl.addValue3DC(xarray,yarray,carray)：用三个数组设定散点图数据，第三个数组映射为色彩变换，以添加形式绘制。

下面是一个全参数的 DRScatter 散点图控件创建实例：

```
hl=dr.DRScatter(win,143,119,700,300,'#ffffff','#000000','#00eeff','cool',5,0,0,0,0,0,0)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRScatter(win,143,119,700,300,dot=3)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRScatter 散点图控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)收到的是控件的曲线显示数据。



下面是 DRScatter 散点图控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个用户自定义函数和一个 DRScatter 散点图控件，然后通过直连绑定方式，用按钮控件来控制自定义函数生成仿真数据，然后用 DRScatter 散点图控件来显示，同时将信号幅值本身作为色彩，从而绘制出渐变色的曲线。DRScatter 散点图控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr

def userFun(v):
    global ff;
    Fs=1000
    N=1000
    ff=ff+2
    t=np.arange(N)*(1/Fs)
    y=np.sin(2*np.pi*ff*t)
    mP3.setValue3DC(t,y,y)

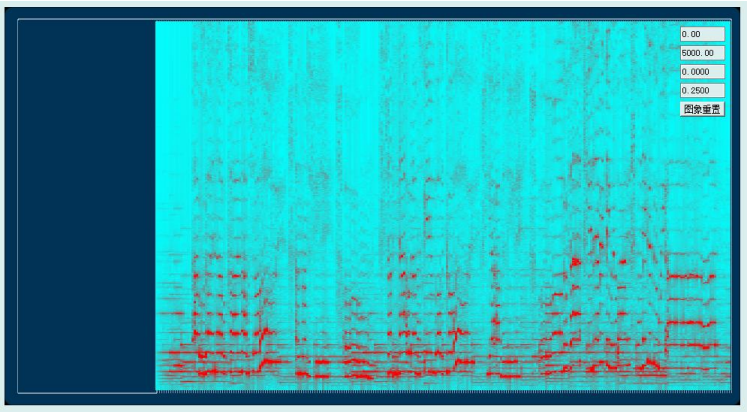
#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,78,39,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入 Scatter 控件
mP3=dr.DRScatter(win,95,122,700,210)
#==控件直连绑定
mB1.addCallBackSingle(userFun)
#==global var===
ff=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Matplotlib，该函数在两个版本中用法无区别。

DRMeshFast 实时语谱图显示控件用法

DRMeshFast 实时语谱图显示控件主要用于显示随信号实时变化的语谱图。因需要耗费较多的显示资源，该控件只支持 pyqt 版本；另外，该控件是为教学中示例自定义控件设计方法而做，并不完善，仅用于展示自定义控件的设计概念。



DRMeshFast 实时语谱图显示控件显示效果图

在 Python 程序 pyqt GUI 界面中创建一个 DRMeshFast 实时语谱图显示控件的命令和语法格式如下：

hl= DRMeshFast(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 pyqt 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#003355'

cb：背景色，默认值='#ffffff'

M：语谱图中的谱图数量，默认值=400

xlow：Y 轴(频率轴)最小值，默认值=0Hz

xhigh：Y 轴(频率轴)最大值，默认值=10000Hz

ylow：Z 轴(幅值，色彩表示)最小值，默认值=0

yhigh：Z 轴(幅值，色彩表示)最大值，默认值=1

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。控件创建时。

DRMeshFast 实时谱阵显示控件有 1 个属性函数，其功能定义如下：

hl.setValue2D(t,Amp)：添加当前频谱到谱阵中，t 和 Amp 为一维数组，对应频谱曲线的频率和幅值数据。

下面是一个 DRMeshFast 实时谱阵显示控件创建实例：

```
hl=dr.DRMeshFast(self.win,77,118,1000,400,M=400,xlow=0,xhigh=5000,ylow=0,yhigh=0.25)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建后可以通过返回的控件句柄对其属性函数进行操作。

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRMeshFast 控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹

出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入端口(setValue2D)收到的是频谱图数据。



下面是 DRMeshFast 实时语谱图控件的创建和应用示例小程序。功能是用一个信号发生器连续生成信号，然后显示和计算信号的波形与幅值谱，并将当前幅值谱添加到 DRMeshFast 实时语谱图控件中，实现语谱图的实时显示。示例小程序代码如下：

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import *
from PyQt5.QtCore import *
import numpy as np
import drvi.drviDSP as dsp
import drvi.drviQTControlls as dr

#===Start of MainClass=====================
class drviMain(QMainWindow):
    def __init__(self):
        super(drviMain,self).__init__()
        self.initGUI()

    #===Init GUI================================
    def initGUI(self):
        self.setWindowTitle("APP Frame by DRPython")
        self.resize(1100,720)
        self.win=QWidget()
        self.win.setStyleSheet("background-color : #ddeeee; ")
        self.setCentralWidget(self.win)
        #=Add controls
        self.mButton1=dr.DRButton(self.win,20,20,100,40,txt='运行/停止')
        self.mHSlider2=dr.DRHSlider(self.win,150,20,900,30,'#444444','#880000','#008822',8,20,0,1000,20)
        self.mGenerator3=dsp.DRGenerator(2,5000,4096,0.6,100,0,250)
        self.mPlot4=dr.DRPlotFast(self.win,20,70,530,160,xlow=0,xhigh=0.05)
        self.mPlot5=dr.DRPlotFast(self.win,560,70,530,160)
        self.mSpectrum6=dsp.DRSpectrum()
        self.mPlot7=dr.DRMeshFast(self.win,20,240,1070,430,cb='#003355',cf='#ffffff',M=400,xlow=0,xhigh=1500,ylow=0,yhigh=0.5)
        self.mButton1.addCallBackSingle(self.mGenerator3.setVSOnOff)
        self.mHSlider2.addCallBackSingle(self.mGenerator3.setVSFre)
        self.mGenerator3.addCallBack2D(self.mPlot4.setValue2D)
        self.mGenerator3.addCallBack2DX(self.mSpectrum6.setValue2DX)
        self.mSpectrum6.addCallBack2DAmp(self.mPlot5.setValue2D)
        self.mSpectrum6.addCallBack2DAmp(self.muserArrayTwo)

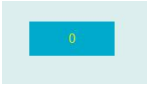
    def muserArrayTwo(self,tArray,xArray):
        self.mPlot7.setValue2D(tArray,np.sqrt(np.sqrt(xArray)))


#==Main Loop=====================
app=QApplication(sys.argv)
app.setStyle("Fusion")
win=drviMain()
win.show()
sys.exit(app.exec())
```

可参考实验“003_QT 媒体文件信号谱阵分析.py”和“005_QT 麦克风谱阵分析.py”。

DRTimer 定时器用法

DRTimer 定时器控件提供一个按毫秒计时的模块，可以定时发出时钟事件，驱动其他控件或回调函数工作。



DRTimer 定时器控件图形显示效果图

在 Python 程序 Tkinter GUI 界面中创建一个 DRTimer 定时器控件的命令和语法格式如下:

hl=DRTimer(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

dt：定时器时间间隔，默认值=125。单位毫秒

fsize：字符点数，默认值=11

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRTimer 定时器控件有 2 个属性函数，其功能定义如下：

hl.setValueSingle(v)：定时器启停开关， 收到数据流后，定时器在启停之间切换。

T=hl.getValueSingle()：取当前定时值

下面是一个全参数的 DRTimer 定时器控件创建实例：

```
hl=dr.DRTimer(win,128,97,100,40,'#00aacc','#eeee00',100,11)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRTimer(win,211,35,100,40,dt=200)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRTimer 定时器控件可以通过绑定的方式将定时事件入传递给绑定的回调函数或其他控件，格式为：

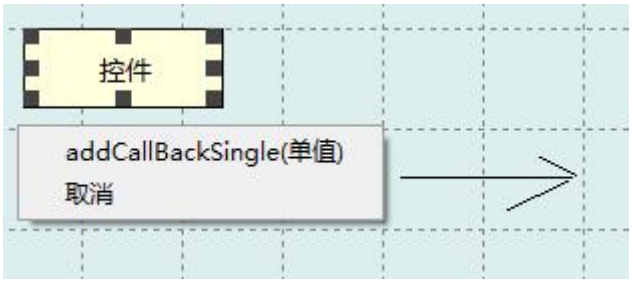
```
hl.addCallBackSingle(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单数值函数，收到的是定时器控件的当前值。回调函数形式为：

```
def setValueSingle(v):
    ......
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRTimer 定时器控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；

选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。控件的定时值可以通过单值端口(addCallBackSingle)输出。



DRTimer 定时器控件也可以做为单数值输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数(setValueSingle)连接在控件开关上，收到单值数据流后在功能在“开/关上切换”。



下面是 DRTimer 定时器控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件，一个输入框控件、一个用户自定义函数和一个 DRTimer 定时器控件，然后通过直连绑定方式，将定时值在输入框控件中显示和传递到用户自定义回调函数中去。DRTimer 定时器控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#用户自定义的回调函数
def muserSingle3(v):
    print(v)
    mEntry3.setValueSingle(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入开关按钮控件
mButton1=dr.DRButton(win,84,34,100,40,'#00aacc','#eeee00','Button',12,101)
#==插入定时器控件
mTimer2=dr.DRTimer(win,211,35,100,40,dt=200)
#==插入输入框控件
mEntry3=dr.DREntry(win,356,36,100,30)
#=直连绑定
mButton1.addCallBackSingle(mTimer2.setValueSingle)
mTimer2.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

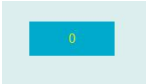
pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRTimer 标签控件继承自 QLabel，因此除了用 DRTimer 的参数化外观定义外，还可以直接用 QLabel 的样式表来对其进行外观描述，并继承其样式表加载方法：

```
setStyleSheet(styleString)
```

DRRecorder 音频视频信号采集用法

DRRecorder 音频视频信号采集控件提供一个计算机麦克风音频信号和摄像头视频信号(仅 pyqt 版本)采集模块，可以实时获取外部的音频/视频信号，并时通过数据流和回调函数形式返回采集的声音数据。



DRRecorder 控件图形显示效果图

Tkinter 版 DRRecorder 控件在使用前需引入 drviAudio 支持库：

```
import drvi.drviAudio as drAudio
```

在 Python 程序 Tkinter GUI 界面中创建一个 DRRecorder 控件的命令和语法格式如下:

```
hl=DRRecorder(win,x,y,w,h,option=value, ...)
```

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

Fs：音频信号采样频率，默认值=44100，单位：Hz

N：信号采样长度，默认值=8192，单位：点

T：采样进程定时间隔，默认值=200，单位：毫秒。 T>1000*N/Fs

fsize：字符点数，默认值=11

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRRecorder 控件有 2 个属性函数，其功能定义如下：

hl.setVSOnOff(v)：音频信号采集开关， 收到数据流后，在启停之间切换。

dx,yarray=hl.getArrayTwoX()：获取当前音频信号采样数据

下面是一个全参数的 DRRecorder 控件创建实例：

```
hl=drAudio.DRRecorder(win,211,27,100,40,'#00aacc','#eeee00',44100,8192,200,11)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=dr.DRRecorder(win,211,35,100,40,T=200)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRRecorder 控件可以通过绑定的方式将采集的麦克风音频信号传递给绑定的回调函数或其他控件，格式为：

```
hl.addCallBack2D(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是音频信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
```

...

或：

hl.addCallBack2DX(fun)

被绑定的回调函数或另一控件的 set 函数是单值+一维数组函数，收到的是音频信号曲线的采样间隔和音频信号信号数据，其数据传输效率更高一些。回调函数形式为：

def setArrayTwoX(dx,yArray):

.....

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRRecorder 麦克风录音控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数包括：当前的录音数据(addCallBack2D 或 addCallBack2DX)。



DRRecorder 控件也可以做为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为控件运行开关(setVSOnOff)，收到单值数据流后在功能在“开/关上切换”。



下面是 DRRecorder 控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，一个 DRRecorder 控件、2 个 Plot 曲线控件和 1 个用户自定义回调函数，然后分别通过直连绑定的方式，用一个按钮控件控制麦克风信号采集的启/停;用 2 个 Plot 曲线控件显示录音信号波形和频谱。DRRecorder 控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy.fft import rfft,irfft
import drvi.drviControlls as dr
import drvi.drviAudio as drAudio
```

```
#用户自定义的回调函数
def muser2d(t,y):
    tArray=t
    yArray=y
    N=len(y)
    N2=int(N/2)
    Fs=1/t[1]
    df=Fs/N;
    A=np.abs(np.fft.rfft(y))/N2
    mPlot3.setValue2DX(tArray,yArray)
    mPlot4.setValue2DX(df,A)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('1000x497')
win.config(bg="#ddeeee")
#==插入 1 个开关按钮控件
mButton1=dr.DRButton(win,32,40,100,40,txt='Run/Stop')
#==插入麦克风控件
mRecorder2=drAudio.DRRecorder(win,29,112,100,40)
#==插入 2 个曲线控件
mPlot3=dr.DRPlot(win,159,34,800,200)
mPlot4=dr.DRPlot(win,159,250,800,200)
#=直连绑定
mButton1.addCallBackSingle(mRecorder2.setVSOnOff)
mRecorder2.addCallBack2D(muser2d)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRRecorder 继承自 QtMultimedia，支持音频与视频采样，其参数与 TK 版的创建参数略有不同，的命令和语法格式如下:

hl=DRRecorder(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

fsize：字符点数，默认值=11

Fs：音频信号采样频率，默认值=44100，单位：Hz

N：信号采样长度，默认值=8192，单位：点

nChunk：采集的音频数据块数，默认值=4

nFrame：采样进程定时间隔(数据块数)，默认值=6

当视频采样时，DRRecorder 控件在采集音频信号的同时，还可以通过绑定的方式将同步采集的图象信号传递给绑定的回调函数或其他控件，格式为：

hl.addCallBackImage(fun)

被绑定的回调函数或另一控件的 set 函数是一个图象函数，收到的是采集的图像帧。回调函数形式为：

```
def setImage(mImage):

    ...
```

其余属性参数和连线方式等则与 TK 版相同。

下面是 pyqt 版 DRRecorder 控件图象采集程序样例：

```
# 引用的库
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import *
from PyQt5.QtCore import *
import drvi.drviQTControlls as dr

#创建 GUI 窗体口象
app=QApplication(sys.argv)
app.setStyle("Fusion")
win=QWidget()
win.setWindowTitle('PyQt 程序')
win.resize(800,500)
win.setStyleSheet("background-color : #ddeeee; ")
#在 GUI 窗口插入控件和布局
mButton1=dr.DRButton(win,60,45,100,40,'#00aacc','#eeee00','运行/停止')
mRecorder2=dr.DRRecorder(win,228,46,100,40,'#00aacc','#eeee00')
mImage3=dr.DRImage(win,72,139,420,260,'#dddddd')
#连线
mButton1.addCallBackSingle(mRecorder2.setVSOnOff)
mRecorder2.addCallBackImage(mImage3.setValueImage)
#显示 GUI 主窗口界面，等待用户操作
win.show()
sys.exit(app.exec_())
```

DRPlay 音频文件播放器用法

DRPlay 音频文件播放器控件是利用 Python 和 Librosa 的音频文件读取和播放功能实现的控件，可以播放 WAV/MP3 格式的音频文件，并将所播放的音频信号通过直连绑定的形式传递给其它控件或用户自定义函数。



DRPlay 音频文件播放器控件图形显示效果图

DRPlay 音频文件播放器控件在使用前需引入 drviSimpleAudio 支持库，此外，Python 中需安装了 librosa 库和 simpleaudio 库：

```
import drvi.drviSimpleAudio as ply
import simpleaudio as sa
```

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlay 音频文件播放器控件的命令和语法格式如下：

hl=DRPlay(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

fsize：字符点数，默认值=11

T：采样进程定时间隔，默认值=200，单位：毫秒。

mfile：播放的音频文件名，默认值=''

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPLay 音频文件播放器控件有 5 个属性函数，其功能定义如下：

hl.setVSOnOff(v)：音频播放器状态开关，0/1/2=运行/停止/选择播放文件，>2 则在启停之间切换。

hl.setValueSingle(v)：设定音频播放器状态，收到数据流后，在启停之间切换。

hl.setSelectfile(v)：弹出音频文件选择窗，选择播放文件

hl.setValueString(txt)：设定播放的音频文件

dx,yarray=hl.getArrayTwoX()：获取当前信号采样数据

下面是一个全参数的 DRPLay 音频文件播放器控件创建实例：

```
hl=ply.DRPlay(win,304,13,100,40,'#00aacc','#eeee00',11,200,'')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=ply.DRPlay(win,350,20,100,40)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRPLay 音频文件播放器控件可以通过绑定的方式将采集的信号数据传递给绑定的回调函数或其他控件。整个音频文件数据的传递格式为：

```
hl.addCallBack2DXAll(fun)
```

回调函数形式为：

```
def setArrayTwoX(dx,yArray):
    ......
```

DRPLay 音频文件播放器控件当前播放数据的传输格式为：

```
hl.addCallBack2D(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

或：

```
hl.addCallBack2DX(fun)
```

被绑定的回调函数或另一控件的 set 函数是单值+一维数组函数，收到的是信号曲线的采样间隔和信号数据，其数据传输效率更高一些。回调函数形式为：

```
def setArrayTwoX(dx,yArray):
    ......
```

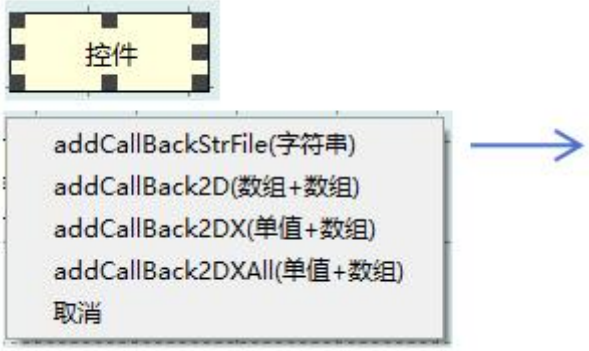
另外，在进行播放文件选择时，也可以通过文本型回调函数绑定方式，传递所选的文件名，格式为：

```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的就是所选的音频文件名。回调函数形式为：

```
def setValueString(t):
    ......
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPLay 音频文件播放器控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数包括：选择播放的音频文件名(addCallBackStrFile)，当前播放的音频数据(addCallBack2D 或 addCallBack2DX)和整个音频文件的数据(addCallBack2DXAll)。



DRPLay 音频文件播放器控件也可以做为单数值输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数包括控件运行开关(setVSOnOff)，收到单值数据流后在功能在“开/关上切换”；音频文件选择(setSelectfile)，收到单值数据流后弹出音频文件选择窗，选择播放的音频文件；音频文件名设定(setValueString)，收到字符串数据流后加载对应的音频文件。



下面是 DRPLay 音频文件播放器控件的创建和应用示例小程序，功能是在 GUI 界面上插入 3 个按钮控件，一个 DRPLay 音频文件播放器控件、2 个 Plot 曲线控件、1 个文本框控件和 1 个用户自定义回调函数，然后分别通过直连绑定的方式，用一个按钮控件选择音频文件，一个按钮控制音频播放器的启/停；再用一个按钮获取当前信号采集数据。若计算机老旧，DRPlot 曲线显示较慢，可将代码中 ccc=2 调大。DRPLay 音频文件播放器控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp
import drvi.drviSimpleAudio as ply
import warnings
warnings.filterwarnings('ignore')

#用户自定义的回调函数
def muser2D(t,y):
    global cc
    cc=cc+1
    tArray=t
    yArray=y
    if cc==2:
        cc=0
        mPlot1.setValue2D(t,y)
        mPlot1.setYlim(-1,1)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入 3 个按钮控件
mBut1=dr.DRButton(win,130,20,100,40,txt='Run/Stop')
mBut2=dr.DRButton(win,20,20,100,40,txt='SelectFile')
#==插入文本框控件
mEntry1=dr.DREntry(win,470,20,250,40)
#==插入 WAV 播放器
mPlayer=ply.DRPlay(win,350,20,100,40,'#00aacc','#eeee00',11)
```

```
#==插入 2 个曲线控件
mPlot1=dr.DRPlot(win,20,80,700,210)

#=直连绑定
mBut1.addCallBackSingle(mPlayer.setVSOnOff)
mBut2.addCallBackSingle(mPlayer.setSelectfile)
mPlayer.addCallBack2D(muser2D)
mPlayer.addCallBackStrFile(mEntry1.setValueString)
#=global=========
cc=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。在 DRVI 的 pyqt 版本中，DRPlay 继承自 QtMultimedia，支持播放 wav/mp3/mp4 音频和视频文件，其参数与 TK 版的创建参数略有不同，的命令和语法格式如下:

hl=DRPlay(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

fsize：字符点数，默认值=11

mfile：播放的音频文件名，默认值=''

mShow：视频播放窗口开关，默认值=0(关)

nChunk：采集的音频数据块数，默认值=4

nFrame：采样进程定时间隔(数据块数)，默认值=6

当视频采样时，DRRecorder 控件在采集音频信号的同时，还可以通过绑定的方式将同步采集的图象信号传递给绑定的回调函数或其他控件，格式为：

hl.addCallBackImage(fun)

被绑定的回调函数或另一控件的 set 函数是一个图象函数，收到的是采集的图像帧。回调函数形式为：

```
def setImage(mImage):
    ...
```

其余属性参数和连线方式等则与 TK 版相同。

用法可参考软件包中的“媒体文件信号波形和频谱分析.py”案例。

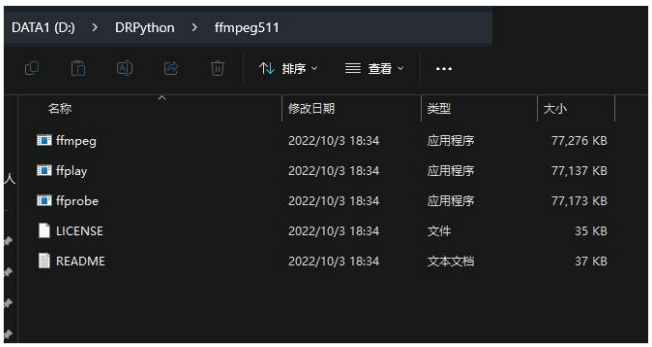
DRPlayX 音视频文件播放器用法

DRPlayX 音视频文件播放器是利用外挂的 FFMPEG 软件包，以前后台的方式进行多媒体文件的播放，优点是支持 WAV、MP3 和 MP4 等多种文件格式；缺点是需下载和安装命令行形式运行的 FFMPEG 软件包。



DRPlayX 音视频文件播放器控件图形显示效果图

关于可执行文件型的 FFMPEG 软件包的下载和安装，请用“FFMPEG”关键词自行查阅网络文献。注意，安装时建议使用简单路径，如：d:/ffmpeg511，同时路径中避免带汉字和空格等特殊符号。本控件在后台调用了 ffmpeg.exe 和 ffplay.exe 两个可执行文件，分别用于音频信号数据的获取和多媒体文件的播放。



FFMPEG 软件包内容图

DRPlayX 音频文件播放器控件在使用前还需引入 drviSimpleAudio 支持库：

```
import drvi.drviSimpleAudio as ply
```

在 Python 程序 Tkinter GUI 界面中创建一个 DRPlayX 音频文件播放器控件的命令和语法格式如下：

hl=DRPlayX(win,x,y,w,h,option=value, ...)

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

cb：背景色，默认值='#00aacc'

cf：前景色，默认值='#eeee00'

vPlayer：FFMPEG 的安装路径，默认值='ffmpeg511'

fsize：字符点数，默认值=11

T：采样进程定时间隔，默认值=200，单位：毫秒。

mfile：播放的音频文件名，默认值=''

注：ffmpeg511=当前 Paython 文件路径\ffmpeg511。在课件提供的免安装便携版 DRVI 中，drvi 库、FFMPEG 和 Python 环境均以子目录的形式嵌入在用户 Python 文件路径下，优点是用 U 盘就可以将整个系统拷下(大约 500M)；上课时，考到教室计算机中即可运行。

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRPlayX 音频文件播放器控件有 5 个属性函数，其功能定义如下：

hl.setVSOnOff(v)：音频播放器状态开关，0/1/2=运行/停止/选择播放文件，>2 则在启停之间切换。

hl.setValueSingle(v)：设定音频播放器状态，收到数据流后，在启停之间切换。

hl.setSelectfile(v)：弹出音频文件选择窗，选择播放文件

hl.setValueString(txt)：设定播放的音频文件

dx,yarray=hl.getArrayTwoX()：获取当前信号采样数据

下面是一个全参数的 DRPlayX 音频文件播放器控件创建实例：

```
hl=ply.DRPlayX(win,274,38,100,40,'#00aacc','#eeee00','ffmpeg511',11,200,'')
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=ply.DRPlayX(win,274,38,100,40)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRPlayX 音频文件播放器控件可以通过绑定的方式将采集的信号数据传递给绑定的回调函数或其他控件，格式为：

```
hl.addCallBack2D(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

或：

```
hl.addCallBack2DX(fun)
```

被绑定的回调函数或另一控件的 set 函数是单值+一维数组函数，收到的是信号曲线的采样间隔和信号数据，其数据传输效率更高一些。回调函数形式为：

```
def setArrayTwoX(dx,yArray):
    ......
```

另外，在进行播放文件选择时，也可以通过文本型回调函数绑定方式，传递所选的文件名，格式为：

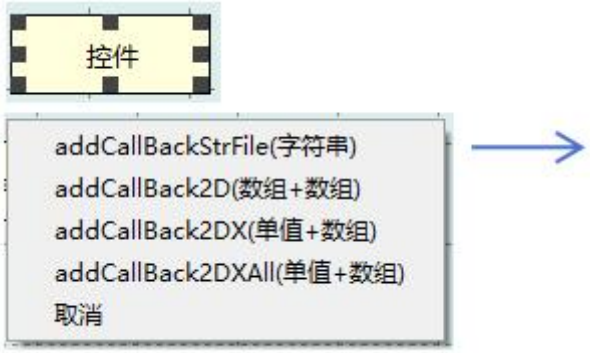
```
hl.addCallBackString(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个字符串函数，收到的就是所选的音频文件名。回调函数形式为：

```
def setValueString(t):
    ......
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPLayX 音频文件播放器控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出

端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数包括：选择播放的音频文件名(addCallBackStrFile)，当前播放的音频数据(addCallBack2D 或 addCallBack2DX)和整个音频文件的数据(addCallBack2DXAll)。



DRPLayX 音频文件播放器控件也可以做为单数值输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数包括控件运行开关(setVSOnOff)，收到单值数据流后在功能在“开/关上切换”；音频文件选择(setSelectfile)，收到单值数据流后弹出音频文件选择窗，选择播放的音频文件；音频文件名设定(setValueString)，收到字符串数据流后加载对应的音频文件。



下面是 DRPLayX 音频文件播放器控件的创建和应用示例小程序，功能是在 GUI 界面上插入 3 个按钮控件，一个 DRPLay 音频文件播放器控件、2 个 Plot 曲线控件、1 个文本框控件和 1 个用户自定义回调函数，然后分别通过直连绑定的方式，用一个按钮控件选择音频文件，一个按钮控制音频播放器的启/停；再用一个按钮获取当前信号采集数据。若计算机老旧，DRPlot 曲线显示较慢，可将代码中 ccc=2 调大。DRPLayX 音频文件播放器控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp
import drvi.drviSimpleAudio as ply

#用户自定义的回调函数
def muser2D(t,y):
    global cc
    cc=cc+1
    tArray=t
    yArray=y
    if cc==2:
```

```
        cc=0
        mPlot1.setValue2D(t,y)
        mPlot1.setYlim(-1,1)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入 3 个按钮控件
mBut1=dr.DRButton(win,130,20,100,40,txt='Run/Stop')
mBut2=dr.DRButton(win,20,20,100,40,txt='SelectFile')
#==插入文本框控件
mEntry1=dr.DREntry(win,470,20,250,40)
#==插入 FFMPEG 播放器
mPlayer=ply.DRPlayX(win,350,20,100,40)
#==插入 2 个曲线控件
mPlot1=dr.DRPlot(win,20,80,700,210)
#=直连绑定
mBut1.addCallBackSingle(mPlayer.setVSOnOff)
mBut2.addCallBackSingle(mPlayer.setSelectfile)
mPlayer.addCallBack2D(muser2D)
mPlayer.addCallBackStrFile(mEntry1.setValueString)
#=global=========
cc=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

无该控件，请用 DRPlay 控件。

DRGenerator 信号发生器用法

DRGenerator 信号发生器控件提供一个标准的软件信号源，可以生成正弦波、方波、三角波和白噪声信号，并将所产生的信号通过直连绑定的形式传递给其它控件或用户自定义函数。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRGenerator 信号发生器控件的命令和语法格式如下:

hl=DRGenerator(option=value, ...)

控件创建时有一组可选的输入参数，包括：

stype：信号类型，默认值=0；0/1/2/3=正弦波/方波/三角波/白噪声

Fs：采样频率，默认值=44100

N：采样长度，默认值=8192

A：信号幅值，默认值=0.6

F：信号频率，默认值=100

P：信号相位，默认值=0

T：数据刷新定时间隔，默认值=200，单位：毫秒。

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRGenerator 信号发生器控件有 8 个属性函数，其功能定义如下：

hl.setVSOnOff(v)：音频信号采集开关， 收到数据流后，在启停之间切换。

hl setVSType(v)：设定信号类型，0/1/2/3=正弦波/方波/三角波/白噪声

hl.setVSFs(v)：设定采样频率

hl.setVSLen(self,v)：设定采样长度

hl.setVSAmp(self,v)：设定信号幅值

hl.setVSFre(self,v)：设定信号频率

hl.setVSPha(self,v)：设定信号相位

dx,yarray=hl.getArrayTwoX()：获取当前信号采样数据

下面是一个全参数的 DRGenerator 信号发生器控件创建实例：

```
hl=dsp.DRGenerator(0,44100,4096,0.6,100,0,250)
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRGenerator(F=200)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRGenerator 信号发生器控件可以通过绑定的方式将采集的信号数据传递给绑定的回调函数或其他控件，格式为：

```
hl.addCallBack2D(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

或：

hl.addCallBack2DX(fun)

被绑定的回调函数或另一控件的 set 函数是单值+一维数组函数，收到的是信号曲线的采样间隔和信号数据，其数据传输效率更高一些。回调函数形式为：

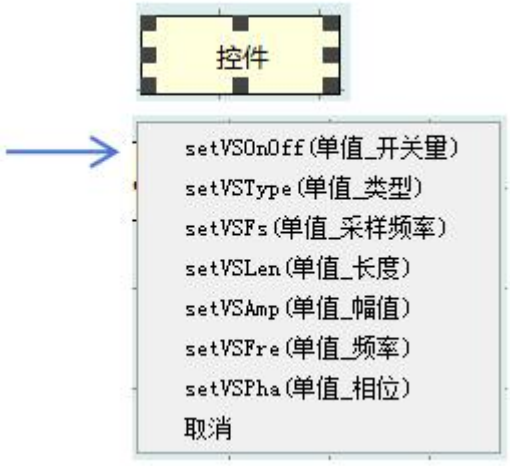
def setArrayTwoX(dx,yArray):

.....

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRGenerator 信号发生器控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数包括：当前的产生的信号数据(addCallBack2D 或 addCallBack2DX)。



DRGenerator 信号发生器控件也可以做为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数包括单数值的控件运行开关(setVSOnOff)，收到单值数据流后在功能在“开/关上切换”；单数值的信号类型(setVSType)，单数值的信号采样频率(setVSFs)，单数值的信号采样长度(setVSLen)，单数值的信号幅值(setVSAmp)，单数值的信号频率(setVSFre)，单数值的信号相位(setVSPha)。



下面是 DRGenerator 信号发生器控件的创建和应用示例小程序，功能是在 GUI 界面上插入 2 个按钮控件，一个 DRGenerator 信号发生器控件、2 个 Plot 曲线控件和 1 个用户自定义回调函数，然后分别通过直连绑定的方式，用一个按钮控件控制信号发生器的启/停；用另一个控件获取当前信号采集数据。DRGenerator 信号发生器控件应用示例小程序代码如下：

```
#==引用的包
```

```
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle6(v):
    dx,yarray=mGen2.getArrayTwoX()
    mPlot4.setValue2DX(dx,yarray)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x497')
win.config(bg="#ddeeee")
#==插入 2 个开关按钮控件
mButton1=dr.DRButton(win,32,40,100,40,txt='Run/Stop')
mButton5=dr.DRButton(win,27,257,100,40,txt='GetData')
#==插入信号发生器控件
mGen2=dsp.DRGenerator(F=200)
#==插入 2 个曲线控件
mPlot3=dr.DRPlot(win,159,34,700,159)
mPlot4=dr.DRPlot(win,151,217,700,180)
#=直连绑定
mButton1.addCallBackSingle(mGen2.setVSOnOff)
mButton5.addCallBackSingle(muserSingle6)
mGen2.addCallBack2D(mPlot3.setValue2D)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DRCameraCV 摄像头控件用法

DRCameraCV 摄像头视频信号采集控件是利用 OPenCV 库实现；因此，使用前选需安装 Python 的 OpenCV 支持库。注意，该控件只采集视频画面，不采集音频信号；另外，该控件为后台工作控件，GUI 界面上无显示。

DRCameraCV 控件封装在 drvi.drOpenCV 中：

```
import drvi.drOpenCV as drcv
```

在 Python 程序 Tkinter GUI 界面中创建一个 DRCameraCV 控件的命令和语法格式如下:

```
hl=DRCameraCV(win,x,y,w,h,option=value, ...)
```

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

mid：系统默认的摄像头 ID 号，默认值=700

imgType：输出图像格式，默认值=0，0（RGB　Array）,1(PIL Image)

sleep：图像采样间隔，默认值=100 毫秒

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRCameraCV 控件有 1 个属性函数，其功能定义如下：

hl.setValueSingle(v)：视频信号采集开关，0/1=运行/停止。

下面是一个全参数的 DRCameraCV 控件创建实例：

```
hl=drcv.DRCameraCV(win,277,23,80,40,mid=700,imgType=0,sleep=100)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=drcv.DRCameraCV(win,277,23,80,40)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRCameraCV 控件可以通过绑定的方式将采集的视频图像传递给绑定的回调函数或其他控件，格式为：

```
hl.addCallBackImage(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个图像函数，收到的是采集的图像数据。回调函数形式为：

```
def setImage(im):
    ...
```

下面是 DRCameraCV 控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，一个 DRCameraCV 控件和 1 个 DRImage 图像显示控，示例小程序代码如下：

```
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drOpenCV as drcv

#==Main Window====================
win= tk.Tk()
win.wm_title('Main')
winWidthCur=816
winHeightCur=497
win.geometry('816x497')
```

```
win.config(bg="#ddeeee")
#==Layout of controls=================
mButton1=dr.DRButton(win,86,22,100,40)
mCamera2=drcv.DRCameraCV(win,277,23,80,40)
mImage3=dr.DRImage(win,135,134,420,260,'#dddddd')
#=User defined data probe============
mButton1.addCallBackSingle(mCamera2.setValueSingle)
mCamera2.addCallBackImage(mImage3.setValueImage)
#==Main Loop=====================
win.mainloop()
```

pyqt 版本中的用法说明：

均继承自 OpenCV，使用无区别，但 pyqt 版建议将该控件的输出图像格式设为 0（BGR Array）而不是 1(PIL Image)，否则需要将 PIL 的 Image 格式转换为 QImage 才方便显示。

另外，pyqt 版的 DRRecorder 控件也支持视频采集，视频采集更直接和高效。

DRTester 仿真测试实验台用法

DRTester 仿真测试实验台控件提供一个由用户定义的传递函数 H(s)定义的单输入、单输出测量系统；或多自由度振动模型定义的单点激励、多输出系统。给仿真实验台输入一个激励信号，就可以得到系统的响应信号。可以仿真生成一阶系统、二阶系统的脉冲响应、阶跃响应和振动系统的扫频波响应等信号。因此，可以作为一个可控的测量实验信号源，在信号分析学习中使用。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRTester 仿真测试实验台控件的命令和语法格式如下：

hl=DRSystem(option=value, ...)

控件创建时有一组可选的输入参数，包括：

msys：预定义的仿真系统类型系统传递函数，默认值=2

0 代表传递函数型，Hs=1/(s**2+10*s+10000)

1 代表 2 自由度振动系统，

m0=1,m1=1,c0=5,c1=5,c2=5,k0=1e3,k1=1e3,k2=1e3

M = np.array([[m0, 0],[0, m1]])

C = np.array([[c0+c1, -c2],[-c1, c2+c2]])

K = np.array([[k0+k1, -k2],[-k1, k2+k2]])

2 代表 3 自由度振动系统

m0=1,m1=1,m2=1,c0=15,c1=15,c2=15,k0=1600,k1=1600,k2=1600

M = np.array([[m0, 0, 0],[0, m1, 0],[0, 0, m2]])

K = np.array([[k0+k1, -k1, 0],[-k1, k1+k2, -k2],[0, -k2,k2]])

C = np.array([[c0+c1, -c1, 0],[-c1, c1+c2, -c2],[0, -c2,c2]])

N：控件内部激励和响应信号长度，默认值=4096

mt：控件内含的激励信号类型，默认值=2；0/1/2/3=脉冲/阶跃/扫频/噪声

P：激振力施加点位置，默认值 0，0/1/2/degN-1，此参数仅振动模型有效。

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRTester 仿真测试实验台控件有 13 个属性函数，其功能定义如下：

hl.setHS(self,txt)：用字符串设定控制系统模型。如 txt='1/(s**2+10*s+10000)'

hl.setMCK(degN,M,C,K)：用 M，C，K 设定 N 自由度振动系统模型

hl.setVIB2(m0,m1,c0,c1,c2,k0,k1,k2)：用各节点质量、阻尼、刚度参数设定 2 自由度振动模型。

hl.setVIB3(m0,m1,m2,c0,c1,c2,k0,k1,k2)：：用各节点质量、阻尼、刚度参数设定 3 自由度振动模型。

hl.setVSType(v)：设定内部激励信号类型，0/1/2=脉冲/阶跃/扫频

hl.setValue2D(self,t,F,P=0)：设定外部激励信号，这时内部激励信号源自动关闭。P 为激振力施加点位置，0/1/2/degN-1，此参数仅振动模型有效。

hl.setValue2DX(dt,F,P=0)：设定外部激励信号。

fMax=hl.getMaxF()：取系统幅频特性中的最大峰值频率，若无峰值，则幅值 1。

fMin=hl.getMinF()：取系统幅频特性中的最小峰值频率，若无峰值，则幅值 0。

Fs=hl.getFs()：取当前的信号采样频率，当使用内部激励信号时=10*fMax；当使用外部激励信号源时，该值等于外部激励信号的采样频率。

dt,farray=hl.getValue2DXF()：获取当前激励信号数据

tarray,farray=hl.getValue2DF()：获取当前激励信号数据

dt,yarray=hl.getValue2DXR(ch=1)：获取当前响应信号数据，ch 为响应点位置，0/1/2/degN-1，此参数仅振动模型有效。

tarray,yarray=hl.getValue2DR(ch=1)：获取当前响应信号数据。

下面是一个 DRTester 仿真测试实验台控件创建实例：

```
hl=dsp.DRSystem()
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRSystem(N=2048)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRTester 仿真测试实验台控件可以通过绑定的方式将系统的激励信号和响应信号数据传递给绑定的回调函数或其他控件。获取激励信号数据的绑定方法为：

```
hl.addCallBack2DXF(fun)
```

获取响应信号数据(振动模型为测点 1，其它点位需用 getValue2DR 获取)的绑定方法为：

```
hl.addCallBack2DXR(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单值+数组函数，收到的是信号曲线的采样间隔和数组值。回调函数形式为：

```
def setArrayTwoX(dt,yArray):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRTester 仿真测试实验台控件可以做为单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数包括：当前的产生的实验台响应数据(addCallBack2DXR)，和实验台激励力数据(addCallBack2DXF)。



DRGenerator 信号发生器控件也可以做为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数包括单数值内部激励信号类型(setVSType)，单数值+一维数组的激励力信号(setValue2DX)。



下面是 DRTester 仿真测试实验台控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮组控件，一个 DRTester 仿真测试实验台控件、2 个 Plot 曲线控件，然后分别通过直连绑定的方式，用按钮组控件驱动 DRTester 仿真测试实验台控件产生不同激励下的响应信号，然后用 plot 控件显示。DRTester 仿真测试实验台控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle(v):
    print(v)
    mSys.setVSType(v)
    dt,F=mSys.getValue2DXF()
    dt,y1=mSys.getValue2DXR(1)
    mPlot3.setValue2DX(dt,F)
    mPlot4.setValue2DX(dt,y1)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('1100x600')
win.config(bg="#ddeeee")
#==插入控件
mHBGr1=dr.DRHButtonGroup(win,30,19,280,36,txt='脉冲,阶跃,扫频,噪声')
mSys=dsp.DRTester(N=2048)
mPlot3=dr.DRPlot(win,27,76,700,210)
mPlot4=dr.DRPlot(win,26,307,700,210)
#==信号的直连绑定
mHBGr1.addCallBackSingle(muserSingle)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 control 库，该函数在两个版本中用法无区别。

DRVideoCV 视频文件采集控件用法

DRVideoCV 视频文件采集控件是利用 OPenCV 库实现；因此，使用前选需安装 Python 的 OpenCV 支持库。注意，该控件只采集视频画面，不采集音频信号；另外，该控件为后台工作控件，GUI 界面上无显示。

DRVideoCV 视频文件采集控件封装在 drvi.drOpenCV 中：

```
import drvi.drOpenCV as drcv
```

在 Python 程序 Tkinter GUI 界面中创建一个 DRVideoCV 控件的命令和语法格式如下:

```
hl=DRVideoCV(win,x,y,w,h,option=value, ...)
```

其中输入参数 win 为 Tkinter 库创建的 GUI 窗口句柄，[x,y,w,h]四个参数为该控件插入 GUI 窗口 win 时的左上角坐标[x,y]和该控件的宽、高[w,h]。另外是一组可选的输入参数，包括：

mfile：视频文件名，默认值=”

imgType：输出图像格式，默认值=0，0（RGB　Array）,1(PIL Image)

rate：图像采样选抽率，默认值=15

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRVideoCV 控件有 2 个属性函数，其功能定义如下：

hl.setValueSingle(v)：视频信号采集开关，0/1=运行/停止。

hl.setValueString(txt)：设定播放的视频文件名。

下面是一个全参数的 DRVideoCV 控件创建实例：

```
hl=drcv.DRVideoCV(self.win,583,114,80,40,mfile='',imgType=0,rate=10)
```

通常，可以选择设定部分选择参数的形式进行设定，除基本参数 win、[x,y,w,h]和用变量名设定的可选参数外，其余可选参数采用默认值。创建实例如下：

```
hl=drcv.DRVideoCV(win,277,23,80,40)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRVideoCV 控件可以通过绑定的方式将采集的视频图像传递给绑定的回调函数或其他控件，格式为：

```
hl.addCallBackImage(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个图像函数，收到的是采集的图像数据。回调函数形式为：

```
def setImage(im):
    ...
```

下面是 DRVideoCV 控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，一个文件选择控件，一个 DRVideoCV 控件和 1 个 DRImage 图像显示控，示例小程序代码如下：

```
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drOpenCV as drcv

#==Main Window====================
win= tk.Tk()
win.wm_title('Main')
winWidthCur=816
winHeightCur=497
```

```
win.geometry('816x497')
win.config(bg="#ddeeee")
#==Layout of controls=================
mButton1=dr.DRButton(win,55,33,100,40,'#00aacc','#eeee00','运行/停止',12,101)
mFileBut2=dr.DRFileBut(win,181,31,110,40,'#00aacc','#eeee00','文件选择','mp4',12)
mImage3=dr.DRImage(win,63,106,420,260,'#dddddd')
mVideo4=drcv.DRVideoCV(win,486,65,80,40,mfile='',mid=700,imgType=0,rate=10)
#=User defined data probe============
mButton1.addCallBackSingle(mVideo4.setValueSingle)
mFileBut2.addCallBackString(mVideo4.setValueString)
mVideo4.addCallBackImage(mImage3.setValueImage)
#==Main Loop====================
win.mainloop()
```

pyqt 版本中的用法说明：

均继承自 OpenCV，使用无区别，但 pyqt 版建议将该控件的输出图像格式设为 0（BGR Array）而不是 1(PIL Image)，否则需要将 PIL 的 Image 格式转换为 QImage 才方便显示。

另外，pyqt 版的 DRPlayr 控件也支持视频采集，视频采集更直接和高效。

DRSpectrum 信号频谱分析用法

DRSpectrum 信号频谱分析控件提供一个 FFT 信号分析模块，可以对收到的采样信号进行频谱分析。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRSpectrum 信号频谱分析控件的命令和语法格式如下:

hl=DRSpectrum(option=value, ...)

该控件创建时只有一个可选的输入参数：

N：FFT 变换长度，默认值=0，即按收到的信号长度

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRSpectrum 信号频谱分析控件有 4 个属性函数，其功能定义如下：

hl.setValue2DX(dx,yarray)：设定信号的采样间隔和采样数组

xarray,yarray=hl.getArrayTwoAmp()：获取当前幅值谱计算的频率和幅值数组数据

xarray,yarray=hl.getArrayTwoPower()：获取当前功率谱计算的频率和幅值数组数据

xarray,yarray=hl.getArrayTwoAmpA()：获取当前插值修正幅值谱计算的频率和幅值数组数据

下面是一个全参数的 DRSpectrum 信号频谱分析控件创建实例：

```
hl=dsp.DRSpectrum(0)
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRSpectrum()
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRSpectrum 信号频谱分析控件可以通过绑定的方式信号的频谱分析数据传递给绑定的回调函数或其他控件。获取幅值谱的计算数据绑定方法为：

```
hl.addCallBack2DAmp(fun)
```

获取功率谱的计算数据绑定方法为：

```
hl.addCallBack2DPower(fun)
```

获取插值修正幅值谱的计算数据绑定方法为：

```
hl.addCallBack2DAmpA(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRSpectrum 信号频谱分析控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成

连线。输出参数为 2 个一维数组组成的幅值谱(addCallBack2DAmp)，2 个一维数组组成的功率谱(addCallBack2DPower)，2 个一维数组组成的修正幅值谱(addCallBack2DAmpA)。



DRSpectrum 信号频谱分析控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为单值+一维数组的采样信号(addValue2DX)。



下面是 DRSpectrum 信号频谱分析控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，1 个信号发生器控件、1 个 DRSpectrum 信号频谱和 2 个 Plot 曲线控件，然后分别通过直连绑定的方式，用一个按钮控件控制信号发生器的启/停和信号波形曲线与幅值谱曲线的显示。DRSpectrum 信号频谱分析控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+10
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    mPlot4.setValue2D(t,y)
    mSpectrum3.setValue2DX(1/Fs,y)
    ff,AA=mSpectrum3.getArrayTwoAmpA()
    mPlot5.setValue2D(ff,AA)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('816x600')
```

```
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,53,29,100,40,txt='Run/Stop')
#==插入频谱分析控件
mSpectrum3=dsp.DRSpectrum()
#==插入曲线显示控件
mPlot4=dr.DRPlot(win,50,91,700,210)
mPlot5=dr.DRPlot(win,49,319,700,210)
#==信号的直连绑定
mButton1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DROctave 倍频程谱控件

DROctave 倍频程谱控件提供一个用 FFT 实现的信号倍频程谱分析模块，可以对收到的采样信号进行倍频程谱分析。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DROctave 倍频程谱控件的命令和语法格式如下:

hl=DROctave(option=value, ...)

该控件创建时只有一个可选的输入参数：

N：FFT 变换长度，默认值=0，即按收到的信号长度

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DROctave 倍频程谱控件有 1 个属性函数，其功能定义如下：

hl.setValue2DX(dx,yarray)：设定信号的采样间隔和采样数组

下面是一个全参数的 DROctave 倍频程谱控件创建实例：

```
hl=dsp.DROctave(0)
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DROctave()
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DROctave 倍频程谱控件可以通过绑定的方式信号的频谱分析数据传递给绑定的回调函数或其他控件。获取 1/3 倍频程谱计算数据的绑定方法为：

```
hl.addCallBack2D31f(fun)
```

或：

```
hl.addCallBack2D31(fun)
```

前者返回的 xArray 数组值是中心频率，后者仅是简单的序号值。

获取 1/1 倍频程谱计算数据的绑定方法为：

```
hl.addCallBack2D11f(fun)
```

或：

```
hl.addCallBack2D11(fun)
```

前者返回的 xArray 数组值是中心频率，后者仅是简单的序号值。

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

下面是 DROctave 倍频程谱控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，1 个信号发生器控件、1 个 DROctave 倍频程谱控件和 1 个 Stem 曲线控件，然后分别通过直连绑定的方式，

用按钮控件控制信号发生器的启/停和信号 1/3 倍频程谱曲线的显示。DROctave 倍频程谱控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mB1=dr.DRButton(win,70,20,100,40)
#==插入信号发生器控件
mG2=dsp.DRGenerator(0,44100,40960,0.6,1000,0)
#==插入倍频程谱控件
mS3=dsp.DROctave()
#==插入 Stem 控件
mP5=dr.DRStem(win,70,80,700,210,xscale=1)
#==控件直连绑定
mB1.addCallBackSingle(mG2.setVSOnOff)
mG2.addCallBack2DX(mS3.setValue2DX)
mS3.addCallBack2D31f(mP5.setValue2D)
#==主程序循环
win.mainloop()
```

DRWavePar 波形分析用法

DRWavePar 波形分析控件提供一个计算信号波形参数的模块，可以对收到的采样信号进行波形参数计算。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRWavePar 波形分析控件的命令和语法格式如下:

hl=DRWavePar(option=value, ...)

该控件创建时只有一个可选的输入参数：

N：分析信号长度，默认值=0，即按收到的信号长度

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRWavePar 波形分析控件有 2 个属性函数，其功能定义如下：

hl.setValue2DX(dx,yarray)：设定信号的采样间隔和采样数组

par=hl.getArrayWavePar()：取返回的波形参数数组，排放顺序为：频率、周期、双峰值、有效值、方差、均值、翘度。

下面是一个全参数的 DRWavePar 波形分析控件创建实例：

```
hl=dsp.DRWavePar(0)
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRWavePar()
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRWavePar 波形分析控件可以通过绑定的方式将信号的波形参数传递给回调函数或其他控件。获取识别的信号频率参数的绑定方法为：

```
hl.addCallBackVSFre(fun)
```

获取识别的信号周期参数的绑定方法为：

```
hl.addCallBackVST(fun)
```

获取识别的信号双峰值参数的绑定方法为：

```
hl.addCallBackVSPP(fun)
```

获取识别的信号有效值参数的绑定方法为：

```
hl.addCallBackVRMS(fun)
```

获取识别的信号方差参数的绑定方法为：

```
hl.addCallBackVSStd(fun)
```

获取识别的信号均值参数的绑定方法为：

```
hl.addCallBackVSMean(fun)
```

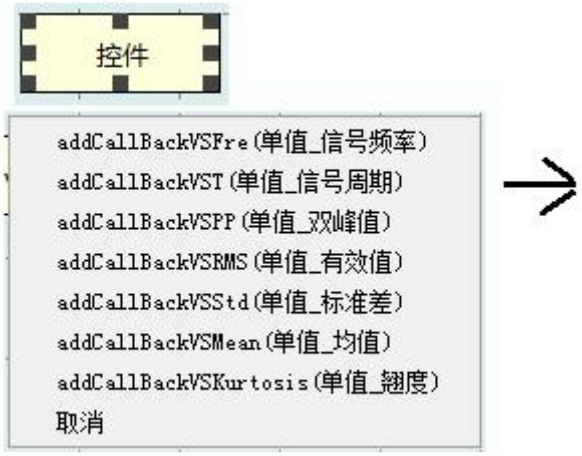
获取识别的信号翘度参数的绑定方法为：

```
hl.addCallBackVSKurtosis(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单值函数，收到的是指定的信号波形参数。回调函数形式为：

```
def setValueSingle(v):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRWavePar 波形分析控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数为波形分析结果，包括单值的识别出的信号频率(addCallBackVSFreAmp)，单值的识别出的信号周期(addCallBackVST)，单值的识别出的信号双峰值(addCallBackVSPP)，单值的识别出的信号有效值(addCallBackVSRMS)，单值的识别出的信号标准差(addCallBackVSFreStd)，单值的识别出的信号均值(addCallBackVSMean)，单值的识别出的信号翘度(addCallBackVSKurtosis)。



DRWavePar 波形分析控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为单值+一维数组的采样信号(addValue2DX)。



下面是 DRWavePar 波形分析控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，1 个信号发生器控件、1 个 DRWavePar 波形分析控件、一个用户自定义回调函数和 1 个 Plot 曲线控件，然后分别通过直连绑定的方式，用按钮控件控制信号发生器的启/停、显示信号波形曲线和 print 识别的波形参数：

```
#==引用的包
import tkinter as tk
```

```
import numpy as np
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+10
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    mPlot.setValue2D(t,y)
    mWavePar3.setValue2DX(1/Fs,y)
    par=mWavePar3.getArrayWavePar()
    print(par)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,56,14,100,40)
#==插入波形分析控件
mWavePar3=dsp.DRWavePar()
#==插入曲线控件
mPlot=dr.DRPlot(win,58,91,700,210)
#=直连绑定
mButton1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DRSpectgram 谱阵分析用法

DRSpectgram 谱阵分析控件提供一个 STFT 信号分析模块，可以对收到的采样信号进行短时傅里叶变换谱阵分析。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRSpectgram 谱阵分析控件的命令和语法格式如下:

hl=DRSpectrum(option=value, ...)

该控件创建时只有一组可选的输入参数，包括：

nfft：每帧数据长度和 FFT 变换长度，默认值=1024

overlap：帧间重叠长度，默认值=128，即 nfft/8

mt：返回谱阵数据类型，默认值=0；0/1/2=功率谱/幅值谱/对数功率谱

mclip：对数功率谱时的数据动态范围，默认值：140db，过大语谱图显示效果不好

Pre：信号预加重开关，默认值=0，0/1=关/开；预加重即信号谱阵分析前加了一个差分处理(高通滤波)，以加重信号中的高频成分。

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRSpectgram 谱阵分析控件有 4 个属性函数，其功能定义如下：

hl.setValue2DX(dx,yarray)：设定谱阵分析信号的采样间隔和采样数组

tarray,farray,data2D=hl.getArray3D()：获取当前谱阵计算数据

hl.setPar(nfft,overlap)：设定 STFT 计算参数

hl.setVSType(mt)：设定谱阵计算类型，0/1/2=功率谱/幅值谱/对数功率谱

hl.setVSPre(pre)：设定预加重开关，0/1=关/开

下面是一个全参数的 DRSpectgram 谱阵分析控件创建实例：

```
hl=dsp.DRSpectgram()
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRSpectgram(mt=2)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRSpectgram 谱阵分析控件可以通过绑定的方式信号的频谱分析数据传递给绑定的回调函数或其他控件。获取幅值谱的计算数据绑定方法为：

```
hl.addCallBack3D(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个由 3 个数组参数组成的函数，收到分别是谱阵的时间点，频率点和谱阵数据。回调函数形式为：

```
def setValue3D(tArray,fArray,stft2DArray):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRSpectgram 谱阵分析控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数为 2 个一维数组和一个 2 维数组构成的谱阵数据(addCallBack3D)。



DRSpectgram 谱阵分析控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为单值+一维数组的采样信号(setValue2DX)，单值的谱阵类型(setVSType)，单值的数据预加重开关(setVSPre)。



下面是 DRSpectgram 谱阵分析控件的创建和应用示例小程序，功能是在 GUI 界面上插入一个按钮控件、一个 DRSpectgram 谱阵分析控件、一个 DRColormesh 语谱图显示控件和一个用户自定义函数，然后通过直连绑定方式，用按钮控件来驱动用户自定义回调函数产生一段 chirp 信号，然后用 DRSpectgram 计算谱阵，最后用 DRColormesh 语谱图显示控件来显示。 DRSpectgram 谱阵分析控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
from scipy import signal
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle3(v):
    Fs=5120;
    N=50000;
    t=np.arange(N)*(1.0/Fs)
    y=np.sin(2*np.pi*(100+5*t*t)*t)
    abc.setValue2DX(1/Fs,y)
    tt,ff,Sxx=abc.getArray3D()
    mArray.setValue3D(tt,ff,Sxx)


#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x497')
```

```
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,20,20,100,30,txt='Run')
#==插入语谱图显示控件
mArray=dr.DRColormesh(win,20,60,800,400)
#==插入谱阵计算控件
abc=dsp.DRSpectgram(mt=2)
#==绑定用户回调函数
mButton1.addCallBackSingle(muserSingle3)
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DRPdfCdf 概率密度和分布用法

DRPdfCdf 概率密度和分布控件提供一个直方图分析模块，可以对收到的采样信号进行概率密度和分布曲线分析。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRPdfCdf 概率密度和分布控件的命令和语法格式如下:

hl=DRPdfCdf(option=value, ...)

该控件创建时只有 3 个可选的输入参数：

M：幅值等分数，默认值=100

ylow：幅值分割计算的下限，默认值=0

yhigh：幅值分割计算的上限，默认值=0

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。如果 ylow==yhigh，幅值分割范围采用自动量程。

DRPdfCdf 概率密度和分布控件有 3 个属性函数，其功能定义如下：

hl.setValue2DX(dx,yarray)：设定信号的采样间隔和采样数组

hl.setPar(M,ylow,yhigh)：设定分析参数

abin,pdf,cdf=hl.getPdfCdf()：获取当前的概率密度和分布曲线数据

下面是一个 DRPdfCdf 概率密度和分布控件创建实例：

```
hl=dsp.DRPdfCdf()
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRPdfCdf(ylow=-1,yhigh=1)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRPdfCdf 概率密度和分布控件可以通过绑定的方式将信号的概率密度和分布曲线数据传递给绑定的回调函数或其他控件。获取概率密度曲线的绑定方法为：

```
hl.addCallBack2Dpdf(fun)
```

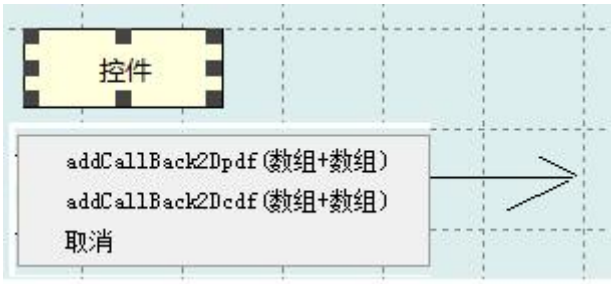
获取功率谱的计算数据绑定方法为：

```
hl.addCallBack2Dcdf(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRPdfCdf 概率密度和分布控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数为 2 个一维数组构成的概率密度曲线数据(addCallBack2Dpdf)，和 2 个一维数组构成的概率分布曲线数据(addCallBack2Dcdf)。



DRPdfCdf 概率密度和分布控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为单值+一维数组的采样信号(setValue2DX)。



下面是 DRPdfCdf 概率密度和分布控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，1 个信号发生器控件、1 个 DRPdfCdf 概率密度和分布计算控件，一个 3 幅曲线的 SubPlot 控件，然后分别通过直连绑定的方式，用一个按钮控件控制信号发生器的启/停和信号波形曲线与概率密度和分布曲线的显示。DRPdfCdf 概率密度和分布控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+10
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    abc.setValue2DX(1/Fs,y)
    abin,pdf,cdf=abc.getPdfCdf()
    mSub.setValue2D1(t,y)
    mSub.setValue2D2(abin,pdf)
    mSub.setValue2D3(abin,cdf)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('1100x600')
win.config(bg="#ddeeee")
#==插入按钮控件
```

```
mButton1=dr.DRButton(win,20,10,100,30,txt='Run/Stop')
#==插入概率密度和分布控件
abc=dsp.DRPdfCdf(ylow=-1,yhigh=1)
#==插入曲线显示控件
mSub=dr.DRSubPlot(win,20,50,950,500,N=3,M=1)
#==信号的直连绑定
mButton1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DRCorr 相关分析用法

DRCorr 相关分析控件提供一个相关分析模块，可以对收到的采样信号进行自相关或互相关分析。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRCorr 相关分析控件的命令和语法格式如下:

hl=DRCorr(option=value, ...)

该控件创建时只有 1 个可选的输入参数：

N：信号计算长度，默认值=0，取全部数据进行计算

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRCorr 相关分析控件有 6 个属性函数，其功能定义如下：

hl.setValue2DX_XX(dx,yarray)：设定自相关分析信号采样间隔和采样数组

hl.setValue3D_XY(dx,xarray,yarray)：设定互相关分析信号采样间隔和 x、y 采样数组

hl.setValue2DX_X(dx,xarray)：设定互相关分析信号采样间隔和 x 采样数组

hl.setValue2DX_Y(dx,yarray)：设定互相关分析信号采样间隔和 y 采样数组

tArray,cArray=hl.getArrayTwo()：获取相关分析曲线数据

tArray,cArray=hl.getArrayTwoCorrect()：获取无偏估计的相关分析曲线数据

下面是一个 DRCorr 相关分析控件创建实例：

```
hl=DRCorr()
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRCorr(N=2048)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRCorr 相关分析控件可以通过绑定的方式将信号的相关分析曲线数据传递给绑定的回调函数或其他控件。获取相关分析曲线的绑定方法为：

```
hl.addCallBack2D(fun)
```

获取无偏估计相关分析曲线绑定方法为：

```
hl.addCallBack2DCorrect(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个二维数组函数，收到的是信号曲线的 x 和 y 数组值。回调函数形式为：

```
def setArrayTwo(xArray,yArray):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRCorr 相关分析控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数为 2 个一维数组构成的相关曲线数据(addCallBack2D)，和 2 个一维数组构成的修正相关曲线数据(addCallBack2Dcorrect)。



DRCorr 相关分析控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为单值+一维数组的自相关分析输入信号(setValue2DX_XX)，单值+一维数组的互相关分析 X 通道输入信号(setValue2DX_X)，单值+一维数组的互相关分析 Y 通道输入信号(setValue2DX_Y)，单值+一维数组+一维数组的互相关分析 XY 通道输入信号(setValue2DX_XY)。



下面是 DRCorr 相关分析控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，1 个信号发生器控件、1 个 DRCorr 相关分析控件，一个 3 幅曲线的 SubPlot 控件，然后分别通过直连绑定的方式，用一个按钮控件控制信号发生器的启/停和信号波形曲线与相关分析曲线的显示。DRCorr 相关分析控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle(v):
    global F
    Fs=5000
    N=512
    F=F+10
    t=np.arange(N)*(1/Fs)
    y=0.5*np.sin(2*np.pi*F*t)
    abc.setValue2DX_XX(1/Fs,y)
    t1,c1=abc.getArrayTwo()
    t2,c2=abc.getArrayTwoCorrect()
    mSub.setValue2D1(t,y)
    mSub.setValue2D2(t1,c1)
```

```
    mSub.setValue2D3(t2,c2)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('1100x600')
win.config(bg="#ddeeee")
#==插入按钮控件
mButton1=dr.DRButton(win,20,10,100,30,txt='Run/Stop')
#==插入相关分析控件
abc=dsp.DRCorr()
#==插入曲线显示控件
mSub=dr.DRSubPlot(win,20,50,950,500,N=3,M=1)
#==信号的直连绑定
mButton1.addCallBackSingle(muserSingle)
#==全局变量===
F=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DRFilter 滤波器用法

DRFilter 滤波器控件提供一个信号带通滤波器模块，可以对收到的采样信号进行带通滤波。该控件为后台控件，在 GUI 界面上无显示。

在 Python 程序 Tkinter GUI 界面中创建一个 DRFilter 滤波器控件的命令和语法格式如下:

hl=DRFilter(option=value, ...)

该控件创建时只 10 个可选的输入参数：

mt：信号滤波算法，默认值=0，0/1/2=FFT/Fir/IIR

Fs：信号采样频率，默认值=44100，实际值=滤波信号采样频率

f1：带通滤波器下限频率，默认值=0，

f2：带通滤波器上限频率，默认值=8000Hz，，消高于 8000Hz 的高频噪声

firM：FIR 滤波器系数长度，默认值 201

firWin：FIR 滤波器设计采用的窗函数，默认值'hamming'。参见 Scipy 库 Window types。

iirMT：IIR 滤波器类型，默认值 0；0/1/2/3=Butter/cheb1/cheb2/ellip

iirP：IIR 滤波器阶次，默认值 8

Irrp：通带邹波幅度，默认值 1db，仅 cheb1 和 ellip 滤波器用到

Iirrs:阻带邹波衰减幅度，默认值，40db，仅 cheb2 和 ellip 滤波器用到

返回值 hl 则是该控件创建后的句柄变量名，可以通过 hl 对创建后的控件进行进一步的设定。

DRFilter 滤波器控件有 6 个属性函数，其功能定义如下：

hl.setValue2DX(dx,yarray)：设定滤波信号采样间隔和采样数组

hl.setType(mt)：设定滤波器类型

hl.setBand(f1,f2)：设定滤波器频率通带

hl.setFIR(firM,firWin)：设定采用 FIR 滤波器时的参数

hl.setIIR(iirMT,iirP,iirrp,iirrs)::设定采用 IIR 滤波器时的参数

dt,yarrayFiltered=hl.getValue2DX()：获取滤波后的信号

下面是一个 DRFilter 滤波器控件创建实例：

```
hl=dsp.DRFilter()
```

通常，可以选择设定部分选择参数的形式进行设定，其余可选参数采用默认值。创建实例如下：

```
hl=dsp.DRFilter(f1=0,f2=500)
```

创建后可以通过返回的控件句柄对其属性函数进行操作。

DRFilter 滤波器控件可以通过绑定的方式将滤波后的信号数据传递给绑定的回调函数或其他控件。获取数据的绑定方法为：

```
hl.addCallBack2DX(fun)
```

被绑定的回调函数或另一控件的 set 函数是一个单值+一维数组函数，收到的是信号曲线的采样间隔 dt 和 y 数组值。回调函数形式为：

```
def setArrayTwoX(dt,yArray):
    ...
```

当采用 DRPython 进行图形化编程时，控件的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。DRFilter 滤波器控件可以做为字符串或单数值输出控件，在连线状态下在控件上点击鼠标，则弹出端选择；选择完成后，点击另一控件，则弹出输入端选择；若输出端和输入端类型一致，则完成连线。输出参数为单值+一维数组构成的滤波信号数据(addCallBack2DX)。



DRFilter 滤波器控件也可以做为单数/字符串输人控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。输入参数为单值+一维数组的输入信号(setValue2DX)。



下面是 DRFilter 滤波器控件的创建和应用示例小程序，功能是在 GUI 界面上插入 1 个按钮控件，1 个信号发生器控件、1 个 DRFilter 滤波器控件，2 个 Plot 控件，然后分别通过直连绑定的方式，用按钮控件驱动信号发生器产生扫频波信号，驱动滤波器控件进行带通滤波，然后进行显示。DRFilter 滤波器控件应用示例小程序代码如下：

```
#==引用的包
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#用户自定义的回调函数
def muserSingle(v):
    global f1
    Fs=5120;
    N=1024;
    t=np.arange(N)*(1.0/Fs)
    x=np.sin(2*np.pi*(100+2000*t)*t)
    mPlot1.setValue2D(t,x)
    abc.setType(2)
    f2=f1+200
    abc.setBand(f1,f2)
    abc.setValue2DX(1/Fs,x)
    dt,x1=abc.getValue2DX()
```

```
    mPlot2.setValue2DX(dt,x1)
    f1=f1+100
    if f1>800: f1=800

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('1100x600')
win.config(bg="#ddeeee")
#==插入控件
mButton1=dr.DRButton(win,20,10,100,30,txt='Run')
abc=dsp.DRFilter()
mPlot1=dr.DRPlot(win,20,60,800,210)
mPlot2=dr.DRPlot(win,20,300,800,210)
#==信号的直连绑定
mButton1.addCallBackSingle(muserSingle)
#==Global
f1=0
#==主程序循环
win.mainloop()
```

pyqt 版本中的用法说明：

DRVI 有继承自 Tkinter 和继承自 pyqt 两个不同库，除引用的 GUI 库(Tkinter/pyqt)和框架程序略有不同外，DRVI 部分控件的创建、布局和连线是完全相同的。因均继承自 Numpy 和 Scipy，该函数在两个版本中用法无区别。

DRUserFunction 支持直连绑定的用户自定义函数用法

在 DRVI 控件构成的 Python 程序中，支持直接用控件的回调函数对另一控件的属性进行设定，称为直连绑定。为实现属性参数的直接设定，按信号分析类程序的特点，我们对控件/模块间传递的参数进行了标准化，并分为如下几类：

String：字符串，用于在控件间交换一段文字信息。

Single：数字，用于在控件间交换一个浮点数字。

2DArray：数组，用于在控件间交换二个一维的浮点数组，如二维曲线的 x 坐标和 y 坐标数据。

2DXArray：数组，用于在控件间传递一个浮点数字+一个一维浮点数组，如一条二维曲线中的采样间隔和采样信号数组值。在传递采样信号类等间隔采样的数据时，可只传递一个浮点数组，比 2DArray 的效率高一些。

3DArray：数组，用于在控件间交换三个浮点数组，如谱阵图、语谱图的 x、y、z 坐标的数组数据。

Bitmap：图象，用于在控件间传递一幅图象的数据块。

只要控件间 get/set 方法的参数类型一致，就可以直接将上一控件的输出，作为输入传递给下一控件的属性设定方法。这样，可以极大地减少因控件取值、赋值所需的回调函数数量，使程序结构变得更加简明、高效；同时，可以通过控件的直连，实现数据流驱动的编程。

1. UserfunSingle(v)用户自定义函数

UserfunSingle(v)是输入参数为 single 变量的自定义函数，实现形式如下：

```
def userSingle(v):
    x=v;
    ......
```

UserfunSingle(v)自定义函数可以用可以用 drvi 控件的 addCallBackSingle 方法进行直连绑定，直接接收该控件的 single 输出，做为函数的输入，并驱动函数执行。

当采用 DRPython 进行图形化编程时，*支持直连的用户自定义函数控件*的创建代码和直连绑定代码的生成均可以由机器实现；控件创建只需要将所需控件拖拽到 GUI 布局区，而直连绑定则需要在 GUI 布局区的控件间进行连线操作。支持直连的用户自定义函数控件为输入控件，在连线状态下当上一控件已选输出函数后，在控件上点击鼠标，则弹出连线选择菜单；选择完成后，若输出端和输入端类型一致，则完成连线。下面是 UserfunSingle(v)自定义函数是控件的直连菜单。



下面是 UserfunSingle(v)自定义函数与滑块控件进行直连绑定，接收滑块控件位置的 Python 代码样例：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
```

```
#==用户自定义函数
def muserSingle1(v):
  x=v
  print(v)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入滑块控件
mHSlider1=dr.DRHSlider(win,77,49,260,30,'#444444','#880000','#008822',8,20,0,100,20)
#==直连绑定
mHSlider1.addCallBackSingle(muserSingle1)
#==主程序循环
win.mainloop()
```

2. UserfunString(txt)用户自定义函数

UserfunString(txt)是输入参数为 string 变量的自定义函数，实现形式如下：

```
def userString(txt):
    tt=txt;
    ......
```

UserfunString(txt)自定义函数可以用 drvi 控件的 addCallBackString 方法进行直连绑定，直接接收该控件的 string 输出，做为函数的输入，并驱动函数执行。下面是 UserfunString(txt)自定义函数与文本框控件进行直连绑定，接收文本框控件中用户输入信息的 Python 代码样例：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr

#==用户自定义函数
def muserString2(txt):
  tt=txt
  print(txt)

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入文本框
mEntry1=dr.DREntry(win,74,55,100,30,'#ffffff','#000000','ABC',12)
#==直连绑定
mEntry1.addCallBackString(muserString2)
#==主程序循环
```

```
win.mainloop()
```

3. Userfun2DArray(x,y)用户自定义函数

Userfun2DArray(x,y)是输入参数为 2 个等长的数组变量的自定义函数，其中 x 是信号采样的横坐标值，通常是时间，y 是信号的采样值。实现形式如下：

```
def user2DArray(x,y):
    xArray=x;
    yArray=y;
    ......
```

Userfun2DArray(x,y)自定义函数可以用 drvi 控件的 addCallBack2D 方法进行直连绑定，直接接收该控件的 2 个数组变量输出，做为函数的输入，并驱动函数执行。下面是 Userfun2DArray(x,y)自定义函数与信号发生器框控件进行直连绑定，接收信号发生器输出信号数组的 Python 代码样例：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#==用户自定义函数
def muserArrayTwo(xArray,yArray):
  print("data ready")

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入信号发生器
mGenerator1=dsp.DRGenerator(0,44100,4096,0.6,100,0,250)
#==插入按钮
mButton2=dr.DRButton(win,43,27,100,40,'#00aacc','#eeee00','Button',12,101)
#==直连绑定
mButton2.addCallBackSingle(mGenerator1.setVSOnOff)
mGenerator1.addCallBack2D(muserArrayTwo)
#==主程序循环
win.mainloop()
```

4. Userfun2DXArray(dx,y)用户自定义函数

Userfun2DArray(dx,y)是输入参数为 1 个单值变量和 1 个数组变量的自定义函数。其中单值变量为信号离散采样间隔，数组变量为离散采样信号的数值。实现形式如下：

```
def user2DXArray(dx,y):
    dt=dx;
    yArray=y;
    ......
```

Userfun2DXArray(dx,y)自定义函数可以用 drvi 控件的 addCallBack2DX 方法进行直连绑定，直接接收该控件的 1 个单值变量+1 个数组变量输出，做为函数的输入，并驱动函数执行。下面是 Userfun2DXArray(dx,y)自定义函数与信号发生器框控件进行直连绑定，接收信号发生器输出信号数组的 Python 代码样例：

```
#==引用的包
import tkinter as tk
import drvi.drviControlls as dr
import drvi.drviDSP as dsp

#==用户自定义函数
def muserArrayTwoX(dx,yArray):
  print("data ready")

#==创建 Tkinter GUI 窗口
win= tk.Tk()
win.geometry('900x500')
win.config(bg="#ddeeee")
#==插入信号发生器
mGenerator1=dsp.DRGenerator(0,44100,4096,0.6,100,0,250)
#==插入按钮
mButton2=dr.DRButton(win,43,27,100,40,'#00aacc','#eeee00','Button',12,101)
#==直连绑定
mButton2.addCallBackSingle(mGenerator1.setVSOnOff)
mGenerator1.addCallBack2DX(muserArrayTwoX)
#==主程序循环
win.mainloop()
```

5. UserfunTri(xarray,yarray,zarray)用户自定义函数

UserfunTri(xarray,yarray,zarray)是输入参数为 3 个数组变量的自定义函数。其实现形式如下：

```
def userTri(array1,array2,array3):
    x=array1;;
    ......
```

UserfunTri(xarray,yarray,zarray)自定义函数可以用 drvi 控件的 addCallBack3D 方法进行直连绑定，直接接收该控件的数组变量输出，做为该函数的输入，并驱动函数执行。

6. UserfunImage(im)用户自定义函数

UserfunImage(im)是输入参数为 1 个图像对象或数组变量的自定义函数。其实现形式如下：

```
def userImage(im):
```

```
pic=im;
.....
```

UserfunImage(im)自定义函数可以用 drvi 控件的 addCallBackImage 方法进行直连绑定，直接接收该控件的图像输出，做为该函数的输入，并驱动函数执行。

2026.9.30

于喻园